Thesis info

Lenker

(annet - forslag - unyttig?)

[Norsk]
https://huggingface.co/ltg/norbert3-base - Norsk språk encoder (?)
https://github.com/stopwords-iso/stopwords-no?tab=readme-ov-file - Filterering av unødvendige ord (Norsk) (preprocessing)
https://github.com/hunspell/hunspell - Feilsjekking av ord
https://tekstlab.uio.no/obt-ny/ - Tagging

[Annet]
https://www.geeksforgeeks.org/snowball-stemmer-nlp/ - Finne stammen av ord

Innhold

- Thesis ting
- Artikkel info
- Instruksjoner / kjøring
- Essay
- Benchmark
- Accuarcy
- Fine tuning

API

https://koala.zooverse.dev/ (koala)
https://refined.zooverse.dev/ (refined)
API_KEY=refined_7xQv9KfR2mL8zTn4YpWc6BqHs3UdEe1aJrV0Xt5CyZ

Thesis description

This thesis aims to explore and evaluate the ReFinED entity linking tool, available on GitHub at https://github.com/amazon-science/ReFinED and documented in the research paper available at https://arxiv.org/pdf/2207.04108. The focus will be on assessing its performance using datasets from the SemTab challenge, specifically those used to validate the Alligator approach (https://ieeexplore.ieee.org/abstract/document/10350082). These datasets include Round_T2D, Round3, Round4, HardTableR2, and HardTableR3. **(and 2T)**

The ultimate goal is to provide insights into the capabilities of ReFinED, identify potential improvements, and demonstrate its practical application in a user-friendly environment through Koala-UI.

The thesis project is structured as follows:

1. **Baseline Evaluation**: The initial phase will involve testing the ReFinED tool in its current state, analyzing its performance, and comparing it to the benchmark results established by the Alligator approach.

2. **Performance Improvement:** If feasible, we will investigate and implement strategies to enhance the performance of ReFinED on the selected datasets.

3. **System Integration:** The final phase will integrate ReFinED into a backend service, which will be connected to the Koala-UI front-end interface (https://github.com/enRichMyData/koala_ui). This integration will enable visualization and interaction with the entity linking results.

Skrive-standarder

UIO latex mal
> Bibtex på latex

1. Lag reference.bib
2. legg inn kilde-info
3. bruk “\cite{...}” eller “\footnote{...}”

Fotnote:
- fotnote ETTER punktum og komma

ReFinED artikkel

https://arxiv.org/pdf/2207.04108

ReFinED uses:

Mention detection

The process of finding pieces of text (mentions) that refers to entities in a knowledge base, like Wikidata or Wikipedia.

e.g. “Steve Jobs founded Apple in 1976”
Mention detection would pick out “Steve Jobs”, “Apple”, “1976” and mark as mentions, candidates for linking entities.

Labeling is not performed, meaning these mentioned are not yet decided on.

- Consists of 2 sequential labeling steps:
    - First step (*mention detection*)
      Responsible for identifying a sequence of words / chunk of words (morphemes) describing an entity
        - similar to named entity recognition (NER), detects entities in text and finds their categories as well. (e.g. finds Mr. Trump and predicts category PERSON)
        - NB! Mention detection varies based on language. English is tokenized (words are separated, nouns are capitalized, etc), while japanese is harder)

          2 Problemer som artikkelen går over, og foreslår løsning for:
          1. Eksisterende verktøy kan ikke gi god nok nøyaktighet for entities next over lange setninger
          2. Kategori “labels” brukt i sekvensiell labeling ikke passer
             (for mange = noen kategorier kan ha for lite info til å være nyttig
             for få = modellen blir upresis på hva som nevnes)

    - Second step (*word sense disambiguation*)
      Responsible for associating the word sequence with its referent entity.
      Verifies each chunk is an entity or not

Fra artikkelen (entity linking):
“Entity linking is a process of associating a name with its referent entity, usually an entry or record of a certain database (e.g., Wikipedia)”
(side 1 under I. INTRODUCTION)

“Mention detection is equivalent to named entity recognition if we ignore category labels”
(side 1 under II. RELATED WORK)

Fine-grained entity typing

- Fine-Grained Entity Typing (FET) is the task of assigning detailed semantic types to entity mentions in text. Unlike coarse entity types like "person" or "organization," FET aims to classify entities into more specific categories (e.g., "politician" or "tech company") based on their context. This allows machines to better understand and reason about the roles entities play in a given sentence, improving performance in tasks like question answering and information extraction.

  ReFinED uses a zero-shot approach where the model can link entity mentions to entities that were not included in training data. Instead of relying on annotated training data for each possible entity, it uses the context of the sentence along with pretrained language model knowledge to understand and classify the correct entity to the correct categories.

(bare random direkte kopieringer med nyttig stuff, skal fjærne dem etterpå)

-Entity disambiguation with fine-grained entity typing In Onoe and Durrett (2020) and Raiman and Raiman (2018) ED is formulated as an entity typing problem. A fine-grained entity typing model is trained on a distantly-supervised dataset consisting of over 10k types derived from Wikipedia categories (e.g. movies released in a specific year). The entity typing model is then used to link entities. We extend their approach to Wikidata, by using a subset of Wikidata triples for providing types instead of Wikipedia categories. -ReFinED artikkelen

-https://arxiv.org/pdf/1909.12079 -fine-grained entity typing med entity linking

ʟGiven a piece of text and the span of an entity mention in this text, fine-grained entity typing (FET) is the task of assigning fine-grained type labels to the mention (Ling and Weld, 2012). The assigned labels should be context dependent (Gillick et al., 2014). For example, in the sentence “Trump threatens to pull US out of World Trade Organization,” the mention “Trump” should be labeled as /person and /person/politician, although Donald Trump also had other occupations such as businessman, TV personality, etc. This task is challenging because it usually uses a relatively large tag set, and some mentions may require the understanding of the context to be correctly labeled. Moreover, since manual annotation is very labor-intensive, existing approaches have to rely on distant supervision to train models (Ling and Weld, 2012; Ghaddar and Langlais, 2018). Thus, the use of extra information to help with the classification process becomes very important.

- Fine-grained entity typing (FET) is the task of identifying specific entity types at a fine-grained level for entity mentions based on their contextual information -https://arxiv.org/pdf/2310.07795

https://ojs.aaai.org/index.php/AAAI/article/view/6380

> “Fine-grained Entity Typing for Domain Independent Entity Linking”

> Tillater linking mellom mentions og entities som ikke forekommer i treningsdata eller knowledge bases

“Zero-shot” = using no training data for the task

(note: det kan hende disse begrepene er generelle og trenger dermed ikke å referere alt, men sikkert nice å ha med noe. dvs. vi kan nok sikkert bare skrive hva de er uten direkte sitat.)
(Ps. Husk å skrive på det faktiske essayet, dette er bare for å samle info fra hovedartikkelen, er flere artikler også)

Entity disambiguation

(disambiguation vil si at man forsøker å spisse inn en betydning - gjøre noe klarere)

Entity disambiguation refers to the process of making detected entities mentioned less ambiguous in meaning, and attempts to find the meaning behind said entity through context clues and candidates from a knowledge base.

It is the process that occurs after mention detection has been done, linking mentions and entities. Which entities each mention refers to.

Mention detection:
- “Apple has a store in Paris”
- “Apple” and “Paris” as mentions

Entity disambiguation
- Decides:
    - “Apple”, meaning fruit? Apple the company?
    - “Berlin”, meaning Berlin, Germany, Berlin, New Hampshire?

Generating candidate entities from a knowledge base such as Wikidata / DBpedia
Uses context around the mention (surrounding words, sentence structuring, topic)
Applies machine learning / neural models to pick best match

.bib fil

https://ieeexplore.ieee.org/document/8885241

```
@article{refined2022,
  title={ReFinED: An Efficient, Transformer-based System for End-to-End Entity Linking of Text},
  author={Ayoola, Ibukun and Ramachandran, Gokul and De-Arteaga, Maria and others},
  journal={arXiv preprint arXiv:2207.04108},
  year={2022}
}
```

ReFinED forts.

“Currently, EL systems use deep learning methods to learn representations for entities and mentions (Ganea and Hofmann, 2017; Le and Titov, 2018)”
https://aclanthology.org/P11-1055/
https://aclanthology.org/P18-1148/

Representation used to be done from only text, relying on similar entries and similar context elsewhere to link mentions and entities - becomes a problem as knowledge bases grow increasingly bigger over time.

Consists of 2 main parts:

1. Mention Detection (MD)
- Recognizing mentions of entities in a given text

2. Entity Disambiguation (ED)
- Assigning knowledge base (KB) entities to each mention
- Function: f : M -> E hvor M = set of mentions, E = set of entities in KB

Entity description:

- Traditionally formed by combining entity with description sources from wikipedia, such as the first sentence in its article

Metode

(Mention Detecting MD)

- Mention encoding done using BIO-tagging format with 3 mentions:
    - B = beginning of a mention
    - I = inside a mention
    - O = outside a mention

> “England won the Fifa World Cup”

B O O B I I

EL inference

- uses predicted mention spans and KB entities with highest combined scores
- (Predicted mention spans = “where mention detected is n text, from x to y”)

ED inference

- Uses provided gold mention spans (= correct mention boundaries)

Experiments

Performance measurements:

- Zero-shot ED:
    - WikiLinksNED unseen mention dataset

- non-zero-shot ED:
    - Pretrain on wikipedia, then use AIDA-CoNLL + mange flere

- non-zeros-hot EL
    - GERBIL platform to evaluate
    - Pretrain on wikipedia, then use AIDA-CoNLL

Praktisk

(anbefalt fremgang?)

1. Setup and Familiarization

Tools and Repositories:

- ☑ ~~Clone the ReFinED GitHub repository and set up its environment.~~
  - ☑ ~~Follow the installation instructions carefully.~~
  - ☑ ~~Test basic functionality with sample datasets provided in the repository.~~
- ☐ Familiarize with Koala-UI to understand its backend/frontend architecture.

Papers and Datasets:

- Study ReFinED research paper to understand the tool's methodology and technical specifics.
- Read the Alligator paper to understand the benchmark methodologies for SemTab datasets.

Dataset Preparation:

- Download the SemTab datasets you plan to use:
  - **Round_T2D, Round3, Round4, 2T-2020, HardTableR2, HardTableR3**.
  - Preprocess the datasets to match the input format expected by ReFinED.
  - Familiarize with the ? schema and challenges of these datasets.

2. Baseline Evaluation

Running Tests:

- Use the pre-trained ReFinED model to perform entity linking on SemTab datasets.
- Measure performance metrics:
    - Precision, recall, and F1-score for Cell Entity Annotation (CEA).
    - Analyze Column Type Annotation (CTA) and Column Property Annotation (CPA) if supported.

Benchmark Comparison:

- Compare ReFinED's results with the benchmark results from Alligator using:
    - Performance metrics provided in the SemTab evaluation scripts.
    - Identify strengths and weaknesses in different dataset challenges (e.g., noisy data in HardTableR2).

Documentation:

- Record detailed observations and potential areas where ReFinED performs sub-optimally.
- Document dataset-specific issues (e.g., ambiguous entities, missing links).

3. Performance Improvement

Analyze Results:

- Focus on cases where ReFinED struggles:
    - Ambiguous mentions or missing context.
    - Errors in linking noisy or incomplete data.

Potential Improvements:

- **Contextual Features**: Enhance ReFinED with better context-awareness for tabular data.
- **Fine-Tuning**: Fine-tune the model using domain-specific or SemTab datasets.
- **Pre/Post-Processing**: Develop custom preprocessing pipelines for table normalization or additional post-processing heuristics.

4. Integration with Koala-UI

Backend Service:

- Wrap ReFinED as an API service:
    - Use frameworks like Flask or FastAPI to expose ReFinED's functionality.
    - Design endpoints for table uploads, entity linking results, and performance metrics.

Frontend Integration:

- Connect the backend service to Koala-UI:
    - Ensure seamless communication via API endpoints.
    - Use Koala-UI to visualize results, highlighting linked entities and their knowledge graph connections.

Demonstration:

- Develop a user-friendly demo showcasing:
    - Entity linking results for uploaded tables.
    - Visualization of metrics and linked entities in Koala-UI.

5. Continuous Evaluation and Refinement

- Test each phase iteratively with small datasets before scaling.
- Gather feedback from your supervisor or peers for improvements.

Tools and Tips:

- Use **Jupyter Notebooks** for quick experimentation and debugging.
- Leverage **Docker** to containerize your setup for easy sharing and deployment.
- Automate evaluation using scripts for consistent benchmarking across datasets

Use of the Alligator approach

1. Comparative Benchmarking (Essential Use)

The Alligator approach provides established benchmark results for SemTab datasets, making it critical for:

- **Performance Comparison**:
    - Use Alligator's results as a baseline to evaluate ReFinED's effectiveness on metrics like precision, recall, and F1-score.
-

- **Dataset Insights**:
    - Analyze how Alligator handles specific challenges in the SemTab datasets, such as noisy or incomplete tables, and compare that to ReFinED's performance.

Why it's essential: It ensures your evaluation is grounded in meaningful, state-of-the-art benchmarks, making your results more credible.

2. Learning from the Alligator Methodology (Optional but Valuable)

The Alligator approach's methodologies can inspire enhancements to ReFinED:

- **Preprocessing**:
    - Study how Alligator handles table normalization, missing values, or ambiguous data and adapt similar strategies to improve ReFinED.
-
- **Graph Traversal**:
    - Alligator uses graph-based methods to refine linking results. Consider whether such techniques could enhance ReFinED, especially for resolving ambiguous entities.

Why it's valuable: Incorporating proven techniques can strengthen ReFinED's performance and offer insights into specific areas where it currently lags.

3. Extending the Comparison to Broader Insights (Highly Recommended)

- **Error Analysis**:
    - Compare the types of errors made by ReFinED and Alligator (e.g., false positives/negatives).
    - Identify whether ReFinED struggles in areas where Alligator excels, or vice versa.
-
- **Qualitative Analysis**:
    - Use specific examples from the datasets to showcase cases where ReFinED outperforms or fails compared to Alligator.

Why it's recommended: It allows you to present a nuanced evaluation and clearly articulate the strengths and limitations of ReFinED.

4. Integration or Hybridization (Advanced/Optional)

- If Alligator provides strong performance in areas where ReFinED struggles, you could explore hybrid approaches:
    - Use Alligator's preprocessing or graph-based linking as a pre-step for ReFinED.
    - Combine outputs from both tools using an ensemble or fallback strategy.

Why it’s advanced: While this adds complexity, it could lead to superior results and demonstrate practical ingenuity in your thesis.

ReFinED git info

bViktige filer og mapper

README.md	Generell bruksinstruksjoner Info om ReFinED + hvordan bruke
requirements.txt	“Dependencies” Ekstra pakker som kreves for å kjøre, bl.a. numpy, cython, …
TRAINING.md	Hvordan trene modellen på wikipedia hyperlinks dataset
FINE_TUNING.md	Hvordan fintune modellen på standard og custom datasets
example_scripts fint sted å forbedre, e.g. - Custom input/output formats - Profile inference speed (?) - Teste endringer i modell-kode - Evaluere egne sample-teksts egen Test-Suite i folder /tests test_accuracy.py = automatisere evaluering mot gold labels test_performance.py = måling av timing og minne test_edge_cases.py = prøving av “rare” eller ekstreme inputs test_consistency.py = sjekke hvis refaktorering gir samme resultat	Mappe med klare python-scripts for demonstrasjon av ulike funksjoner **entity_disambiguation.py** = hvordan bruke tity disambiguation på en tekst **coreference_resolution.py** = bruk av modell med coreference resolution, dvs. håndtering av pronouns og referanser til tidligere entiteter **wikidata_disambiguation.py** = bruking av wikidata varianter av ReFinED istedenfor wikipedia **batch_processing.py** = hvordan prosessere flere tekster samtidig (batch mode) **custom_model_loading.py** = eksempel på loading av model fra custom checkpoint / directory **ner_and_disambiguation.py** = kjøring av både NER (Named Entity Recognition) og Disambiguation med ReFinED
test/test_refined.py	test for å sjekke om det går ann å kjøre
test/run_refined.py	fil for visning av funksjoner
src/refined	Source code

Hvordan kjøre / teste

1. Opprett Python virtuell miljø

```
python -m venv .venv
```

2. Aktiver virtuell miljø til Python (venv)

Windows (cmd)

```
.\venv\Scripts\activate
```

Windows (PowerShell)

```
.\venv\Scripts\Activate.ps1
```

Linux/Mac (bash)

```
source venv/bin/activate
```

3. Installer dependencies

```
pip install -r requirements.txt
```

4. Intaller pytest

```
pip install pytest
```

5. Installer pakken i utviklingsmodus

```
pip install -e .
```

6. Sett PYTHONPATH

hvis kjøring uten (5.), kun om pakken ikke er installert

```
export PYTHONPATH=src
```

7. Kjør tester

```
python -m pytest test/test_refined.py -v
```

Thesis structure

Forside (bruk mal)

- navn
- Linje, studiepoeng
- Institutt for Informatikk

Sammendrag / Abstrakt

Deklarasjon av bruk av KI

- nevne at KI har blitt brukt og til spesifikt hva.

Innholdsfortegnelse

- Sider / kapitler
- Liste av bilder
- Liste av tabeller / data
  (aka hvor det ble brukt i tesen, sidetall osv)

Introduksjon

- Motiver
  (ta fra essay)
- Forskningsspørsmål
  (lag basert på tekst side 2)
- Struktur

Bakgrunn / Relatert arbeid

Implemetasjon
(punkt 3. ideer)
(punkt 4. ideer)

Design / Eksperimentering / testing
(punkt 1. ideer)

Evaluering
(punkt 1. ideer)
(punkt 4. ideer)

Diskusjon

Videre arbeid i fremtiden og konklusjon
(punkt 2. ideer)

Lenker

info

“En lang masteroppgave (60 studiepoeng) burde være på ca. 100 (+/- 20) sider. “

Generelt:
https://www.mn.uio.no/its/studier/master/skrive-en-masteroppgave.html
https://www.mn.uio.no/studier/master/

inspirasjon / andre oppgaver:

www.duo.uio.no/handle/10852/3/discover?rpp=100&sort_by=dc.date.issued_dt&order=DESC

Av Jonas og Elias (Design) - Samarbeids oppgave
https://bibsys-almaprimo.hosted.exlibrisgroup.com/primo-explore/fulldisplay?docid=DUO10852%2F112663&context=L&vid=UIO&lang=no_NO&search_scope=default_scope&adaptor=Local%20Search%20Engine&tab=default_tab&query=any,contains,jonas%20longva%20pettersen&offset=0

Av Sebastian (Prosa) - Alene oppgave
https://bibsys-almaprimo.hosted.exlibrisgroup.com/primo-explore/fulldisplay?docid=DUO10852%2F114472&context=L&vid=UIO&lang=no_NO&search_scope=default_scope&adaptor=Local%20Search%20Engine&tab=default_tab&query=any,contains,Automated%20Clone%20Elimination

LaTeX

overleaf:
https://www.overleaf.com/project

maler:
https://www.uio.no/om/designmanual/profilen-i-bruk/latex/index.html

info:
https://www.mn.uio.no/ifi/tjenester/it/hjelp/latex/uiomasterfp-guide.pdf

Data

tidlig testing på companies, movies, EL
>
https://docs.google.com/spreadsheets/d/1E2h3ukWjGOhwiPadD36T0hWzLHeZ0F73WCrkbnffogc/edit?usp=sharing

Semtab dataset
>
https://docs.google.com/spreadsheets/d/1Iemt3-J4HgdoZn-eT7IM9tCRTax501-XGEbfJjGHmVg/edit?usp=sharing

Kode

https://github.com/amazon-science/ReFinED - ReFinED github side (original)

https://github.com/enRichMyData/ReFinED - ReFinED github ("vår")

https://github.com/enRichMyData/koala_ui - Koala UI
https://github.com/enRichMyData/ReFined - ReFined Repo

Tidlige artikler

https://arxiv.org/pdf/2207.04108 - ReFinED dokumentasjon / artikkel

https://arxiv.org/abs/2207.04106 - ReFinED extension m/ knowledge base info i modellen

https://ieeexplore.ieee.org/abstract/document/10350082 - Alligator dokumentasjon

Kommunikasjon

entitylinking-gwp6496.slack.com - Slack URL

Ideer

Ting å skrive om i selve tesen

1. Skrive om og inkludere benchmarker og resultater vi fikk fra å teste modellen og fine tune med mange ulike parametere.

Future Work / Practical System design / Domain specialization

2. Fine-tuning med forskjellige datamengder (movies, companies) viser gode resultater for **domene-spesifikke modeller.**
    - En modell trent på filmdata brukes for filmrelevant input
    - En modell trent på selskapsdata brukes for selskapsrelevant input
    - Kan utvides til en “Front-end with Domain Experts” arkitektur:
        - Generell front-end modell som tar imot input og ruter det automatisk til riktig domeneekspert (modell)
        - “Ekspertene” returnerer resultater som samles for sluttbrukeren.
        - Fordelen er at det gir høy presisjon per domene, skalerbarhet og enkel brukerinternasjon.
        - Filtrerer ut feil skapt av trening på irrelevante data.

- (Videre arbeid i fremtiden og konklusjon) på Thesis-siden

3. Skrive om testene lagd i python, hvordan det ble implementert og hva det ble brukt til.

4. Performance improvements (step 2 i master plan)
    - Her kan skrives om hvordan fine-tuning og tester for å forbedre presisjonen på spesifikke data som filmer og selskap ble brukt, med betydelig bedre resultater.
    - Også hvordan benchmarking og tester bidro til å finne hyperparametere / konfigurasjoner for enklere og kjappere kjøring.

5. Skrive om Cell-level vs Row-level accuracy
    - gitt en linje “aaa, bbb, ccc, ddd” med label til “ggg” for “aaa”
    - cell level: predict fra “aaa” (bruk kun main)
    - row-level: predict fra “aaa, bbb, ccc, ddd” (bruk hele kontekst)

Kommunikasjon

Veilder info:

Titi Roman / Dimitru Romanu
Dumitru.Roman@sintef.no

Martin Giese
martingi@ifi.uio.no

Professor Roberto Avogadro
roberto.avogadro@sintef.no

Kommunikasjon

entitylinking-gwp6496.slack.com - Slack URL

TODO

TODO

Ting som må gjøres, fra mest til minst viktig

~~1. Koala JSON / API~~

- ~~se (møtereferat 10.11)~~

2. OLLAMA, evt. andre LLM og undersøke Entity linking brukbarhet

- Ollama har egen branch med egen test-fil.
- Henvis til slack for bruk og modeller
- (ikke gjort pga mindre scope)

~~3. Data fra Crocodile / Alligator~~

- ~~Finne og samle data fra crocodile og/eller alligator benchmark så vi har noe å sammenligne med.~~
- ~~Helst Alligator, siden dette er spesifisert i beskrivelsen.~~
- ~~Kan finne info her:~~
  - ~~https://ieeexplore.ieee.org/abstract/document/10350082~~
- ~~VELDIG VIKTIG~~

~~4. Samle tekst fra docs, legg til overleaf~~

- ~~samle inn alt tekst herifra som kan være nyttig og brukt i masteroppgaven~~
- ~~Fordele det utover riktige deler / kapitler i masteroppgaven på overleaf~~
- ~~muligens skrive om / endre på ting slik at det passer~~

5. Utforsk og integrere LamAPI

- lamapi_demo_2023 ← token
- Egen fil: my_tests/lamapi_test.py
- Bytte ut tidligere wikidata call i filer som bl.a. my_tests/run_refined.py
- https://lamapi.hel.sintef.cloud/
- (ikke tid + mindre relevant for oppgaven)

6. Fine-tune modell på relevant semtab-data (?)

- https://sem-tab-challenge.github.io/2023/
- Semtab info viser mulighet for finetuning av semtab-relatert data.
- Kan hjelpe med resultater i evalueringen potensielt
- (ingen tid, kunne vært sinnsykt nyttig :/)

~~7. Bli mer kjent med Koala UI~~

- ~~Step 3 i “master-planen”~~
- ~~Mest sannsynligvis senere etter vi er ferdig m/ refined~~
- ~~må integrere funksjonalitet til koala fra refined~~
- ~~https://koala.zooverse.dev/login~~
- ~~SKRIVE OM I MASTEROPPGAVEN !~~

~~8. Dypere benchmark av ReFinED~~

- ~~inkludere sammenligning av modeller, CPU/GPU, batching/ikke batching~~
- ~~Dypere analyse og evaluering og sett inn i benchmark-rapport~~
- ~~kan bytte ut imdb-delen~~
- ~~mye er allerede laget under essay - Benchmark og Accuracy~~

~~9. Skrive om benchmarking / evaluering / fine tuning~~

- ~~Dette er stuff som kan og burde brukes i selve tesen~~
- ~~Kanskje gjøre senere, men definitivt viktig~~

TODO

- [x] ~~Clone ReFinED og gjøre det til en delt repo begge kan teste~~
- [x] ~~Clone Koala (?) ^~~
- [x] ~~Lese / summere artikler som ReFinED~~
- [x] ~~Forstå ReFinED linking og teste~~
- [ ] Numba / Cython for ytelsesforbedring (?)
  Mixed programming (Webvisning)
- [x] ~~Skrive i Google docs -> overfør til latex ?~~

Møtenotater

05.09.2025

Hovedmål:

- [x] ~~Overføre filer og tekst fra (https://github.com/borgebj/ReFinED_Master) til main repo (https://github.com/enRichMyData/ReFined)~~
- [x] ~~Benchmark refined - simple tables~~
    - measure time - evaluate algorithm
    - from start to done
- [x] ~~https://lamapi.hel.sintef.cloud/ LAMAPI for (QID) -> (Description)~~
- [x] ~~Output TYPE for entity links~~
- [x] ~~test.csv file (IKKE TRAIN)~~
- [x] ~~filter out (companies_train - bruk companies_test ?)~~
- Candidate threshold - if top candidate has low score skip etc
    - 2 tests:
        - filter out nil-cases

(etterhvert)
Semtab

- https://sem-tab-challenge.github.io/2025/

for integration:

- format på json:
- https://github.com/enRichMyData/koala_ui (står nedeover)

(SENERE)
lage api for ReFinED

- post, get, delete, put



SPARQL queries med QID

- query wikidata

koala page:

- http://koala.zooverse.dev/

17.09.2025
Hovedmål

Fra fil: `el_companies_gt_wikidata`

- bruk pandas til å lese alle linjer som inkluderer/starter med: "Companies_test" keep rows with companies_test.

- Store refined results (main predicted) in csv file

```
for span in spans:
    pred_ent = span.predicted_entity # <-- main predicted entity
    pred_qid = getattr(pred_ent, "wikidata_entity_id", None)
```

associate corresponding row to companies_test

Refined row 0 with test row 0

Do with:
- "companies"
- "movies"
- "SN" tables

Also test performance without GPU - using CPU
Compare GPU and CPU

(etterhvert) Improve with batching

Prioritize accuracy - comparing with gt values

- Finne ut om mulig "switch" fra bruke GPU til CPU

Første mention, aka "Company" er tydeligvis hoved-entitetet som skal sammenlignes dvs for companies_test

```
Mention: [WeoGeo]  →  Predicted entity: [WeoGeo (QID: Q7983092)]
  Type: MENTION
  Model confidence: 100.00%
  Top-k candidate entities:
    - WeoGeo (QID: Q7983092), score: 100.00%
```

nr1

```
Mention: [Indiana Public Retirement System]  →  Predicted entity: [Indiana Public Retirement System (QID: Q16847362)]
  Type: MENTION
  Model confidence: 100.00%
  Top-k candidate entities:
    - Indiana Public Retirement System (QID: Q16847362), score: 100.00%
```

nr2

Disse sammensvarer med de 2 første QID i gt-filen:



TODO

1. Kjør refined, lagre hoved-entity (company) i csv fil, vil se lignende ut:
    a. linje1: QID1
    b. linje2: QID2
    c. linje3: QID3

2. les GT-fil (ground truth) og extract linjer med “companies_test” fra 0 til 999
    a. extract QID etter wikidata link “wikidata.org/…/QID”

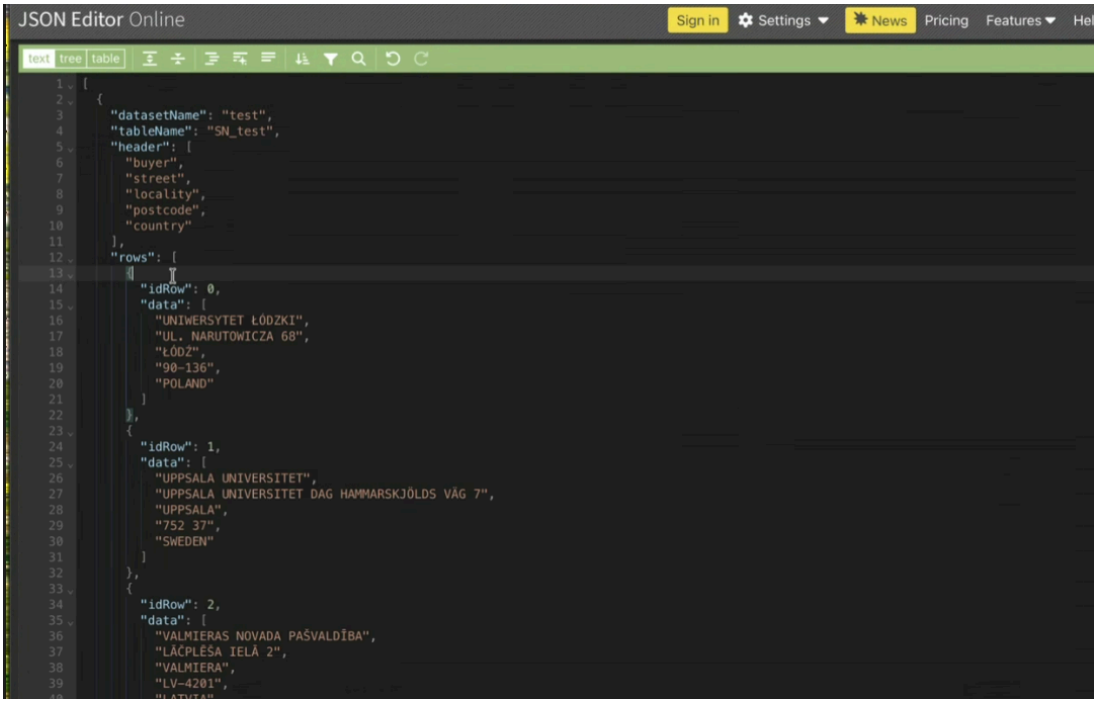
3. Sammenlign denne QID med refined QID
    a. nb! vær sikker på at linjenr aka rad sammensvarer
        i. dvs. rad 1 i refined.csv == rad 1 i gt.csv (eller noe sånt)
    b. Score ved å telle antall som er like

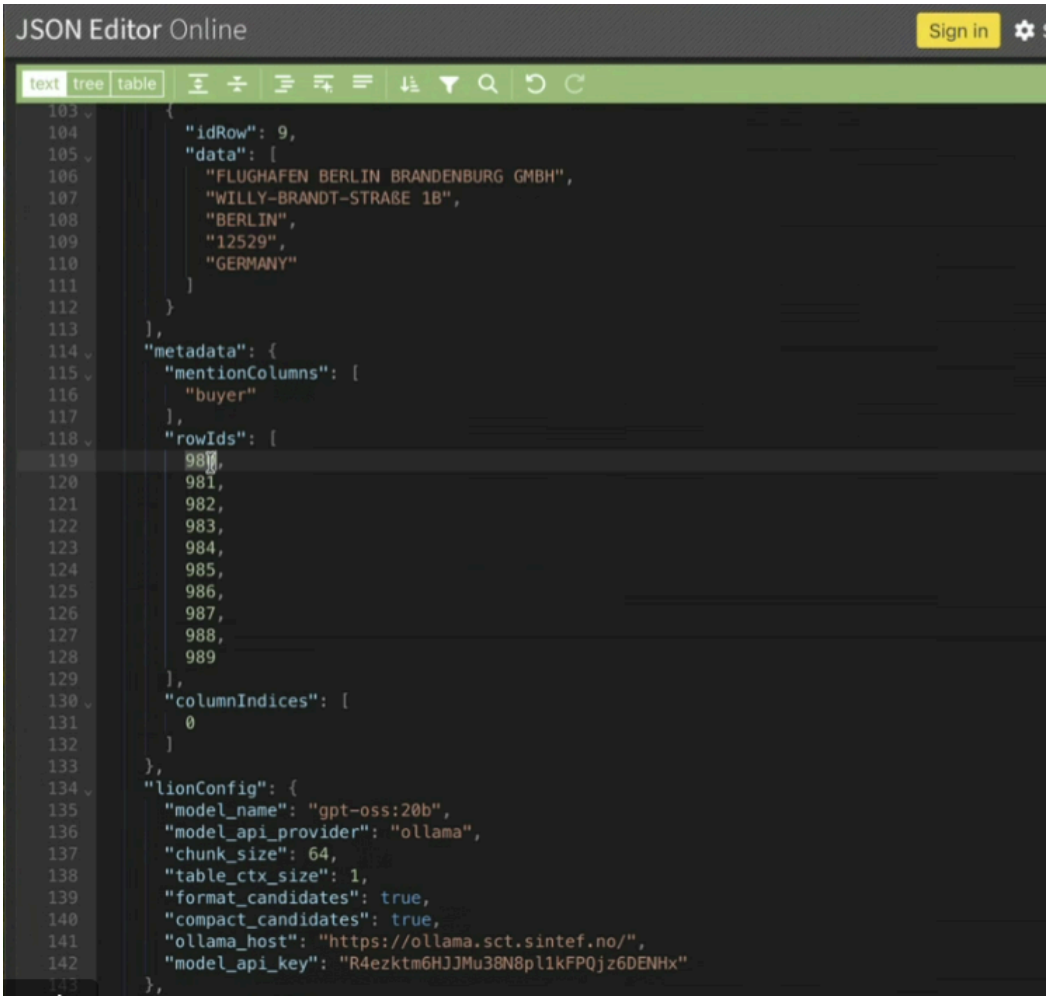
- Gjør dette med filer:
    - companies
    - movies
    - SN

- Gjør i egen ryddig fil, hent dette: (kun første for hver span)

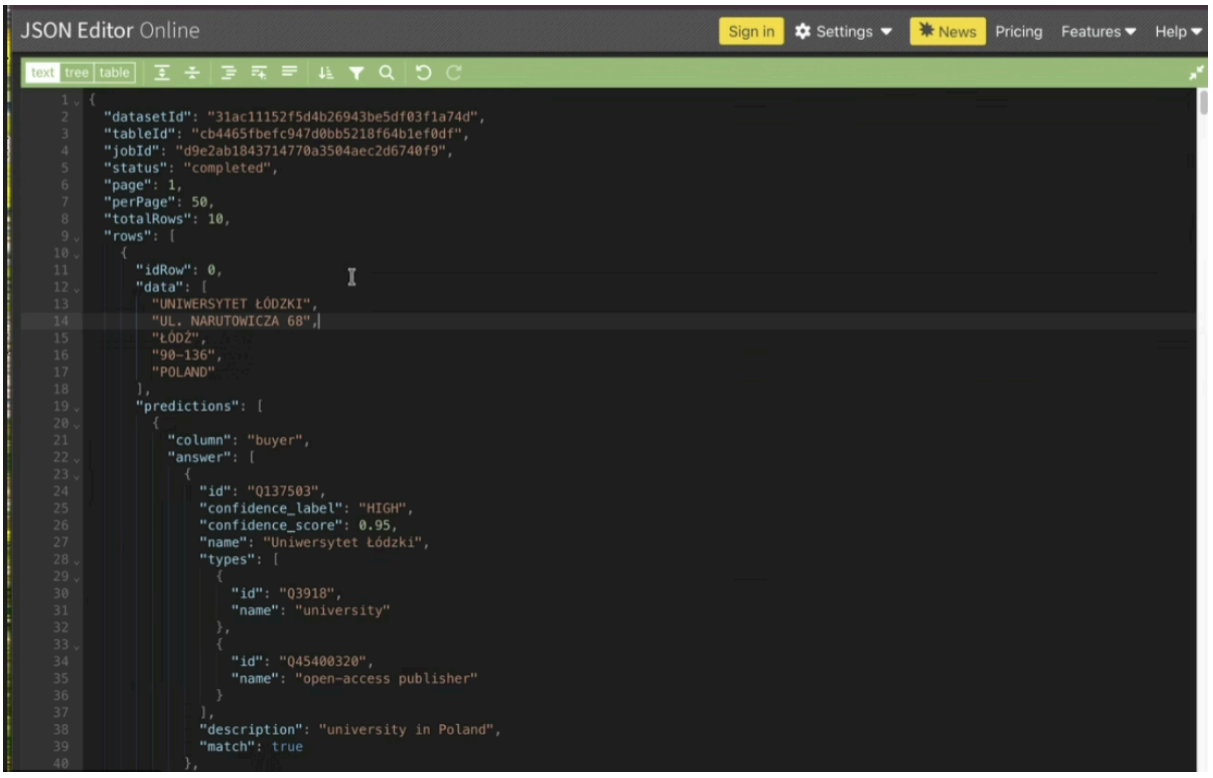
```
for span in spans:
    pred_ent = span.predicted_entity # <-- main predicted entity
    pred_qid = getattr(pred_ent, "wikidata_entity_id", None)
```

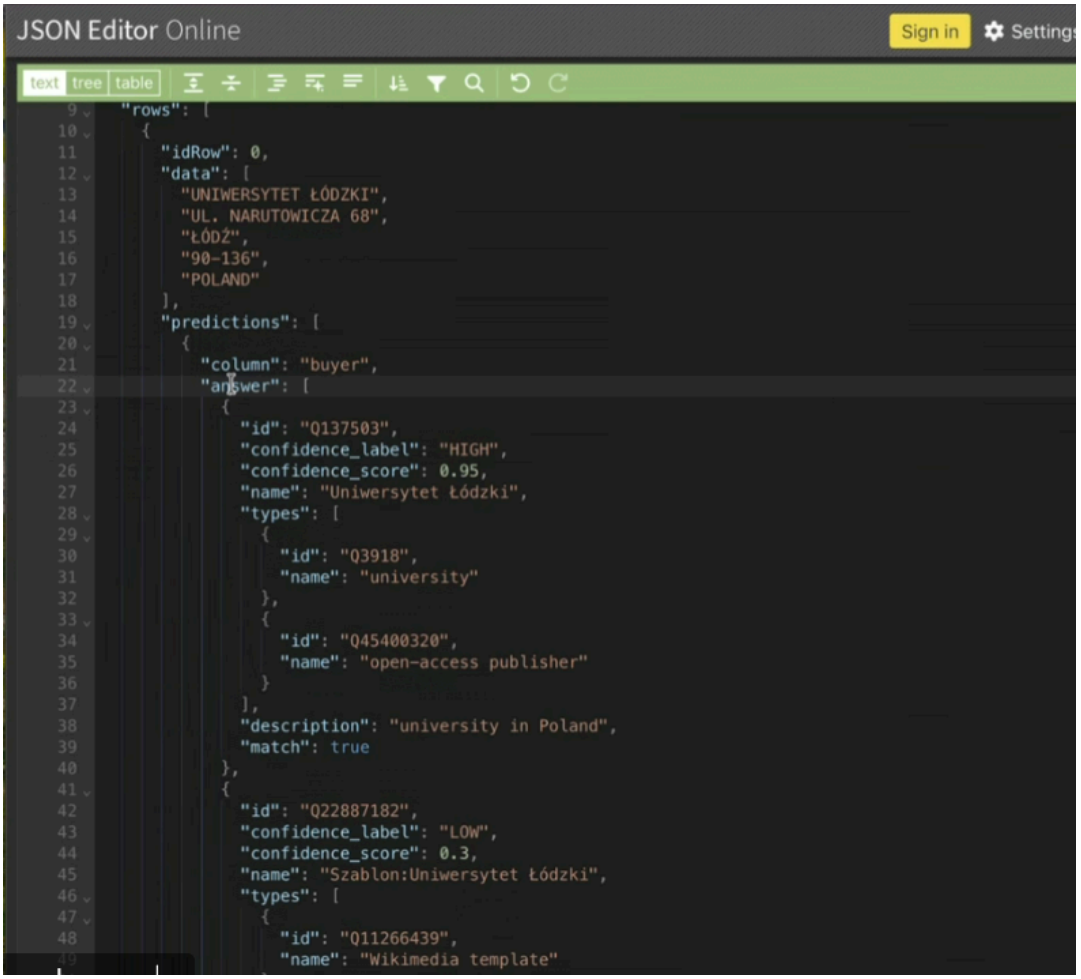
10.11.25
JSON input





Output





MATCH = True if highest confidence



1. ideer til paper
2. Ideer til bruk av AI / LLM
3. Måle ulike datasets (dataset.zip)
4. Fine tune på alle tables -> teste på vanskeligere
5. Gjøre klar API for bruk av ReFined til Koala (se standarder under)

W3C entity linking specs (standarder)

28.01.2026

stateless
> lion.zooverse.dev/docs#/default/create_job_jobs_post

ideally: use same structure, slightly changed for ReFinED

send a job (linking)

status endpoint

link columns = which columns to link - what is being detected/linked

top_k = how many candidates from the link

retriever = is not needed by ReFinED - done at the same time
- source where candidates are gotten from

inputs - specifically CSV files
API - uses JSON (generate JSON that is to be sent)

moose does NER - specific with STI (semantic table i...) identifying columns in a table

(finner lignende på koala github)

"col", 0,
"cell_id": "0.0",
"mention": "...",
"candidate_ranking:" [

  # én candidate

  "id": "...",
  "name": "...",
  "types": { { "id": "...", "name": "..." }],
  "description":, "...",
  "confidence_label": "HIGH",
  "confidence_score": 0.095,
  "match: true

]

FastAPI endpoint <--- use AI

if process 1000 rows not return 1000 rows
pagination system ?
- if several pages, sync results for UI

Skrive info

Innhold

Academic writing..... 1
Introductions..... 1
Structure..... 1
Free writing:.....3
Referencing..... 5

Academic writing

Academic writing is when one is writing for the purpose of gathering, studying, and portraying information in a way that is presentable to a general audience such that eventual findings or points come across.

Introductions

A relatively short start to a broader text where one explains and goes through what the actual topic is about.
Often relates background information and related topics.

2 Models:

1) **CARS (Creating a research space)**

- Establish the territory (larger space)
- Identify a niche (something specified, something we need to look at)
- Occupy the niche (establish expectation with reader)

2) **ABT (And but therefore)**

- AND (establish context, “This is how things are”)
- BUT (establish tension, a conflict, something to be resolved)
- THEREFORE (propose a solution, a resolution, a way forward)

The idea is you must actually write these three words, as markers for presentational sequence

Structure

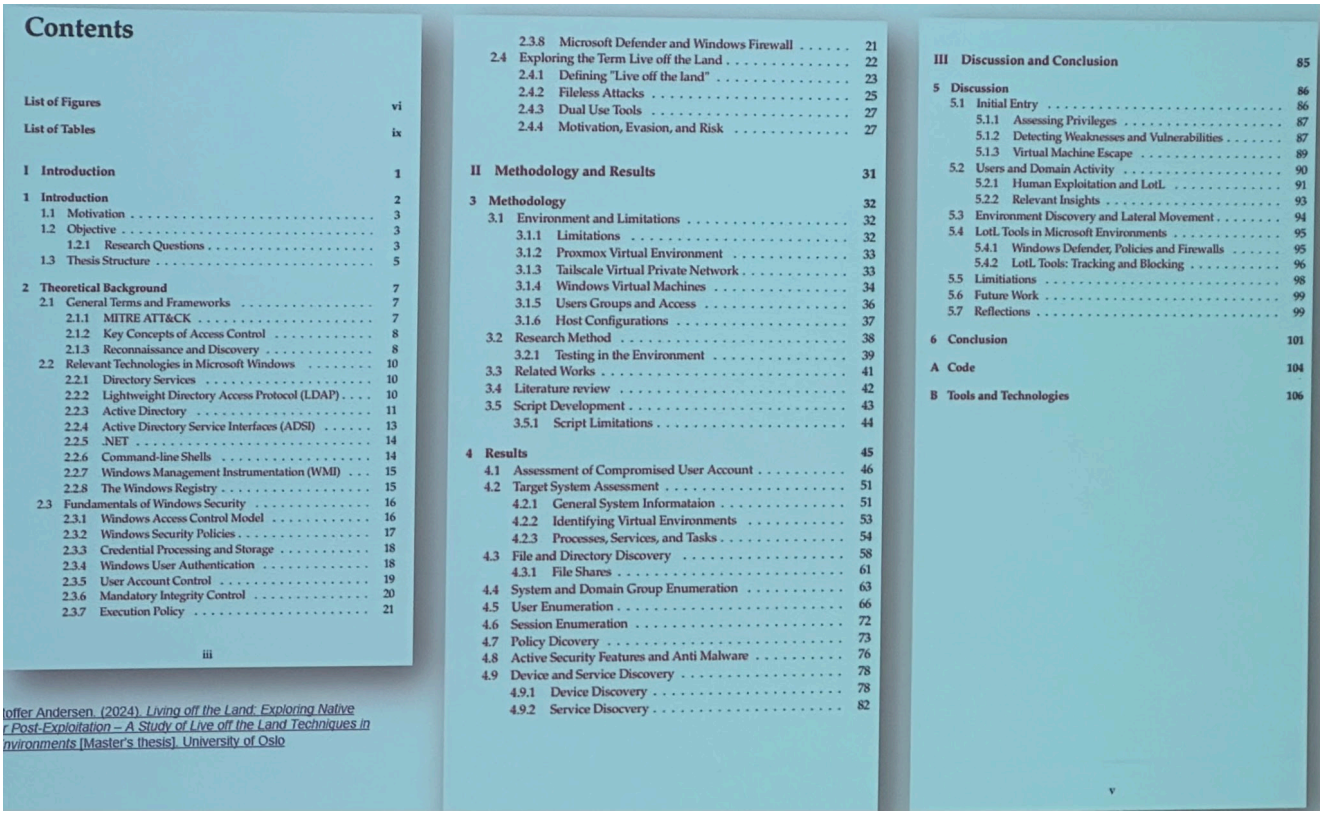
Basic structure common to most ifi theses:

- IMRaD
    - Introduction
    - Methods
    - Results (and)
    - Discussion

Example

1. Introduction
    a. motivation
    b. Objective
    c. Thesis structure
2. Theoretical background’
    a. General terms and frameworks
    b. Relevant technologies …
    c. Fundamentals of …
    d. Exploring …
3. Methodology
    a. Environment and limitations ?
    b. Research method
    c. Related work
    d. Litterature review
4. Results
    a. Assessment …
    b. Target system assessment ?
    c. ….
5. Discussion
6. Conclusion

- Code
- Tools and technologies



Contents

List of Figures vi

List of Tables ix

I Introduction 1

1 Introduction 2
1.1 Motivation 3
1.2 Objective 3
1.2.1 Research Questions 3
1.3 Thesis Structure 5

2 Theoretical Background 7
2.1 General Terms and Frameworks 7
2.1.1 MITRE ATT&CK 7
2.1.2 Key Concepts of Access Control 8
2.1.3 Reconnaissance and Discovery 8
2.2 Relevant Technologies in Microsoft Windows 10
2.2.1 Directory Services 10
2.2.2 Lightweight Directory Access Protocol (LDAP) 10
2.2.3 Active Directory 11
2.2.4 Active Directory Service Interfaces (ADSI) 13
2.2.5 .NET 14
2.2.6 Command-line Shells 14
2.2.7 Windows Management Instrumentation (WMI) 15
2.2.8 The Windows Registry 15
2.3 Fundamentals of Windows Security 16
2.3.1 Windows Access Control Model 16
2.3.2 Windows Security Policies 17
2.3.3 Credential Processing and Storage 18
2.3.4 Windows User Authentication 18
2.3.5 User Account Control 19
2.3.6 Mandatory Integrity Control 20
2.3.7 Execution Policy 21
2.3.8 Microsoft Defender and Windows Firewall 21
2.4 Exploring the Term Live off the Land 22
2.4.1 Defining "Live off the land" 23
2.4.2 Fileless Attacks 25
2.4.3 Dual Use Tools 27
2.4.4 Motivation, Evasion, and Risk 27

II Methodology and Results 31

3 Methodology 32
3.1 Environment and Limitations 32
3.1.1 Limitations 32
3.1.2 Proxmox Virtual Environment 33
3.1.3 Tailscale Virtual Private Network 33
3.1.4 Windows Virtual Machines 34
3.1.5 Users Groups and Access 36
3.1.6 Host Configurations 37
3.2 Research Method 38
3.2.1 Testing in the Environment 39
3.3 Related Works 41
3.4 Literature review 42
3.5 Script Development 43
3.5.1 Script Limitations 44

4 Results 45
4.1 Assessment of Compromised User Account 46
4.2 Target System Assessment 51
4.2.1 General System Informataion 51
4.2.2 Identifying Virtual Environments 53
4.2.3 Processes, Services, and Tasks 54
4.3 File and Directory Discovery 58
4.3.1 File Shares 61
4.4 System and Domain Group Enumeration 63
4.5 User Enumeration 66
4.6 Session Enumeration 72
4.7 Policy Dicovery 73
4.8 Active Security Features and Anti Malware 76
4.9 Device and Service Discovery 78
4.9.1 Device Discovery 78
4.9.2 Service Disocvery 82

III Discussion and Conclusion 85

5 Discussion 86
5.1 Initial Entry 86
5.1.1 Assessing Privileges 87
5.1.2 Detecting Weaknesses and Vulnerabilities 87
5.1.3 Virtual Machine Escape 89
5.2 Users and Domain Activity 90
5.2.1 Human Exploitation and LotL 91
5.2.2 Relevant Insights 93
5.3 Environment Discovery and Lateral Movement 94
5.4 LotL Tools in Microsoft Environments 95
5.4.1 Windows Defender, Policies and Firewalls 95
5.4.2 LotL Tools: Tracking and Blocking 96
5.5 Limitiations 98
5.6 Future Work 99
5.7 Reflections 99

6 Conclusion 101

A Code 104

B Tools and Technologies 106

loffer Andersen. (2024). *Living off the Land: Exploring Native r Post-Exploitation – A Study of Live off the Land Techniques in nvironments* [Master's thesis]. University of Oslo

Free writing:

The image is of a red big apple showing visible spots of yellow in itself. It envelops the entire screen infront of us, as a way of portraying something of the unknown. The topic is the actual apple, but it is unknown if there is any purpose of the picture itself. No additional information was provided with the actual apple, due to this the interpretation of the reader is required to understand anything at all. There is a visible tiny speck of black beneath the apple, showing its shadow casted from above. It is a full white background, almost as if the apple is in a photoshoot where light is casted from every angle. Because of this only a small amount of shadow if visible of the apple.
The apple itself is a food, and because of this the thought of other food can come to ones mind. An example of a food is the delicious substance known to man as “Zita hamburger”

located in the town of Lierbyen. This is the objective of the current weekend to come, acquire the zita burger.
It is almost as the apple itself removes the focus of the reader to another topic completely, due it its small amount of information. There is not much to write about a single apple in itself, and therefore you must come up with something. That is the goal of the apple. On the topic of zita burgers, it is a noteworthy objective to acquire it and it must be done in a fast manner. Together with the zita burger one must not forget the pizza, an essential ingredient of a full weekend.
writing this feels not entirely fulfilling as it is a giant spew of random text about a single yet useless piece of apple.

Free writing 2:

Semantic technology is about connecting data. Basically on the internet and digitally data exists everywhere, but the relationship between units and information is not an easy task. Information is often complex and not always as easy as saying “x is y” or something exists and this is that. Information is constructed from general knowledge, from the thoughts and findings of people. These constructions can be interpreted in a thousand different ways, making it both hard to generalize, convey and understand by both human and machine.

Semantic technologies is about creating technology in a way that understands the semantics of information, meaning the way information works together, what they express and such. To achieve this, specialized tools and knowledge is needed, specifically tying information together. The topic of data linking is vital when it comes to this. Data linking is about what the name implies, linking relevant data together in a way that makes it understandable mostly by machines, making it easier to process digitally and therefore also store for further actions such as lookups.

Data linking can happen in various ways. The subject IN4060 goes through technology and tools that make this possible, utilizing tools such as OWL, RDF, SPARLQ and such to create complex expressions and make complex actions happen.

Free writing 3

What i really want to explore is namely the tool usage when it comes to the action of data linking digitally. The need for tools to be produced to process this efficiently i greatly needed. In this day and age there are already massive amounts of information available online for many purposes such as this. Therefore we wish to explore already existing tools such as Refined and Alligator in a way that we explore its functionality through various benchmarks and “challenges”, measuring their performance on a large set available online. This benchmark helps us establish a start point for our investigation, as it is the benchmark results we look at when we will further explore how to deal with potential issues and even possibly rework and remake functionality as to better its performance.

Free writing 4

I need to know more about the entire field of semantics as well as the tools available for us online in order for the exploration and study to be successful. This is crucial, as a baseline

knowledge of the topic is needed to grasp the functionality of these tools is needed for us to start working on it and potentially improve already existing features.

Additionally, if we are to integrate large language models, this would also be a field in need of exploring, as to integrate a new service to an already existing model requires knowledge about the usage of and the range of functionality this other service provides. Integrating large language models is not an easy task in itself, as we need a base understanding of how to use it as a developer, and not a user.

Referencing

All claims in scientific writing (except general knowledge) must be **substantiated**

- … by the authors own work
- … by citing others

Reference styles:

- Author-date references, ex. APA 7th
- Numerical references, ex. IEE, Vancouver

Speak with supervisor about which style to use early in process
Two critical points on referencing:

1. Be consistent with reference style
2. Always cite your sources

Citing

- Cite all sources you have used
  (dont copy or repeat text)
- Refer correctly by citing source in the text and giving complete information about the source in the list of references at the end
  (include date in reference)
- Don't cite sources not used
- Citation must identify the source

Basic rules

- Read UIO web page on so
- Use “Søk og Skriv” and read through your reference list
  (https://www.sokogskriv.no/)

Verktøy:

- https://www.zotero.org/

Citing code

- AUthors name
- Date
- Title of program / source code

- Code version
- Type (e.g. computer program, source code)
- web address or publisher (eg. program publisher, URL) (Link to github)

Using others figures

- illustrations, figures, plots … protected by copyright
- “open access” != right to re use
- Check if it has a license for reuse / ask rights holder for permission
- Check if you have permission to re-use or adapt (sometimes a license for re use exists)

How to ask for permission

- on web page -> look for contact details
- “figure re-use permission” or similar
- E-mail to author
- Books: find book on publishers web-page

Essay

Essay ideer:

- Utkast av intro til thesis / methodology
  - ☑ ~~Bakgrunn til semantisk teknologi~~
  - ☑ ~~Bakgrunn av hvordan internett samler / kobler / knyttet data~~
  - ☐ Bruksområdet til dette
  - ☐ eksempler, wikipedia / wikidata

- Skrive om Refined / Alligator etc
  - ☐ lese / oppsummere artikler, presentere

- Skrive om verktøy vi skal bruke, og detaljert bakgrunn
  - ☐ intro til ReFinED
  - ☐ Intro til Koala UI
  - ☐ Intro til Alligator ?
  - ☐ Statistikk / Kjøretid / Semtab
  - ☐ Forbedringer

- Grafer og bilder

Tutorial:

- Titler:
  - Format -> Avsnittstitler
- Kilder:
  - Verktøy -> Kildehenvisninger
- Artikler:
  - https://scholar.google.com/

NB! Skrevet først, så strukturert med GPT (ide på form og struktur), dvs. rettskriving og et par termer. Selvfølgelig rettet opp etter også.

TODO:

1. Legg til / skriv om verktøy og teknologi (Viktig !)
2. Delkapittel om “Metodologi” ? (burde ha med i masteren)
3. Skrive om hva artikkelen handler om (kun master thesis)
4. Flere delkapitler
5. Legg til wikidata (?) (med mindre dette ikke skal være i introduksjonen)
6. Definere og utdype:
   a. Reasoning
   b. Entailment/inference
   c. Knowledge graph
7. Overfør over til latex (etterhvert)
8. Spør om kildehenvisning til veileder
   a. **svar: bibliotex i LaTeX**

From Data to Knowledge: Entity Linking in semantic information systems

Introduction

In today's digital age, the ability to connect and link relevant information across diverse sources is more critical than ever. As global infrastructure expands and the volume of publicly available data ever increases, a fundamental challenge emerges: how can both humans and machines make sense of this information?

Interpretation and connection of data and information often comes naturally to us humans. We use prior knowledge, intuition, and context to understand what we read or see, but machines operate differently. While they can process vast quantities of data in milliseconds, they lack the innate understanding of meaning, context and nuance. This gap presents a core challenge in the field of artificial intelligence: enabling machines to not only read data but to *understand* it in a way that allows for meaningful reasoning and decision making.

This is the basis of semantic technologies and intelligent systems. From search engines returning contextually relevant results, to recommendation systems predicting what we might like based on subtle patterns, to digital assistants responding to natural language queries - these systems all rely on the ability to model, link, and interpret information in a human-like way. Even more fundamental are knowledge graphs like Google's Knowledge Graph for Microsoft's Satori, which connect facts, concepts, and entities into structured networks of meaning.[1] These serve as the backbone for everything from answering trivia questions to understanding user intent in search queries.

As the scale and complexity of data increases, so does the need for tools that support these systems in linking, categorizing, and disambiguating information. This is where semantic entities linking tools such as ReFinED[2] and Koala UI[3] contribute to this field.

---

[1] https://blog.google/products/search/introducing-knowledge-graph-things-not/
[2] https://github.com/amazon-science/ReFinED/tree/main.
[3] https://github.com/enRichMyData/koala_ui.

Among the various tools developed to address the challenges of entity linking, this thesis focuses on a detailed exploration and evaluation of the ReFinED entity linking tool. Specifically, its performance will be assessed using datasets from the SemTab Challenge,[4] including Round_T2D, Round3, Round4, 2T-2020, HardTableR2, and HardTableR3, which were previously used to benchmark the Alligator approach.[5] The ultimate goals of this research are to provide insights into the capabilities of ReFinED, identify potential improvements, and demonstrate its practical applications in a user-friendly environment through integration with the Koala-UI front-end interface. This essay is structured in three main phases: first, a baseline evaluation of ReFinED's performance; second, an investigation into potential performance improvements; and third, the integration of ReFinED with a back-end service connected to Koala UI for enhanced visualization and interaction with entity linking results.

Background of semantic technologies

While traditional databases are well-suited for storing structured data in tables such as numbers, names, or clearly defined categories, much of the data that surrounds us today is not structured in this way. Data from news articles, scientific papers, websites, and social media posts often lacks a consistent schema. This makes it harder to analyze, integrate, and reason within an automated process. Structured data is organized in a clear and defined format, making it easy to analyze by tools and humans, whereas unstructured data is lacking in format and form. Sources such as emails, text documents and video files can be seen as unstructured[6], as often a mix of media elements such as text and images, and, in the case of video, a stream of audio and visual data. As the complexity and increasing volume of this data grows, the gap between structured and unstructured data has become a central challenge in fields like artificial intelligence, information retrieval, and web development.

To address this, researchers and standards organizations have worked over the years to develop frameworks that process raw data with semantics, giving meaningful descriptions of what the data represents and how it relates to other information. The goal is not just to allow machines to read data, but to understand it in a limited and useful way. This is the foundation of semantic technologies: systems and standards that give data explicit meaning, allowing

[4] https://www.cs.ox.ac.uk/isg/challenges/sem-tab/
[5] https://ieeexplore.ieee.org/abstract/document/10350082
[6] https://www.ibm.com/think/topics/structured-vs-unstructured-data

machines to perform reasoning. draw inferences, and connect information across disparate contexts.

One of the core building blocks of this movement is the **Resource Description Framework (RDF)**. Originally introduced by the W3C in the late 1990s, RDF was designed as a general model for representing information in a machine-processable way. Its conceptual simplicity is part of its power: RDF expresses knowledge in the form of **triples**, each consisting of a *subject*, *predicate*, and *object*. These triples form the atomic units of meaning in RDF and can be used to describe virtually any relationship.[7]

For example, to represent the statement *"Berlin is the capital of Germany"*, an RDF triple might look like:

<uri/Berlin> <uri/capitalOf> <uri/Germany>

Here, the URIs act as globally unique identifiers for each entity and relationship, making it possible to merge and interlink data from different sources without ambiguity. The structure naturally forms a **graph** of interconnected facts—a "web" of data, where each node can point to or refer to any other node on the web. This model forms the technical foundation of the **Linked Data** movement.[8]

With the adoption of RDF, clearer and richer semantic capabilities were required to support more complex knowledge. This led to the development of additional layers on top of RDF, beginning with RDFS. RDFS introduced the ability to define classes and properties, allowing data to be grouped into hierarchies described with greater structure. From the previous example, this could mean that *<uri/Berlin>* was a member of the *<uri/City>* class, and *<uri/Germany>* was a member of the <uri/Country> class. For further expressiveness of knowledge, OWL was introduced. The Web Ontology Language extended both RDF and RDFS, allowing expressions containing equivalence and cardinality to be used. It was here we could express "Every x is y" for example. To work with this language, additional tools were developed, making it easier to use for developers. Specifically SPARQL worked as a SQL-type language where one could make queries and ask for specific information from

[7] https://www.w3.org/TR/rdf11-primer/
[8] https://www.w3.org/DesignIssues/LinkedData.html

these graphs.[9] In programming languages such as Java, Apache Jena was made, providing libraries and services for creating RDF graphs, defining ontologies, and running SPARQL queries.

It is with this background that tools like ReFinED and Koala UI were developed, solutions of newer generations for linking text to structured knowledge bases.

Over time layers have also been built on top of and extended the use of RDF, enriching its usage and extending its coverage. The RDFS includes a schema as well, providing capabilities for defining classes, properties, and hierarchies, allowing for grouping into categories.[10] For more expressive modelling OWL (web Ontology Language) was developed upon these, allowing complex relationships and constraints such as defining equivalence between concepts or setting cardinality of statements.

Building upon this foundation of semantic technologies and the structured representation of knowledge through RDF, RDFS, and OWL, a crucial process emerges for bridging the gap between raw data and this structured understanding; **Entity Linking (EL)**. At its core, Entity Linking is the task of identifying entities mentioned in a text - often referred to as *mentions* - and linking them to a unique identifier in a knowledge base.[11] This process can be conceptually viewed as a function, $f : M \rightarrow E$, where M represents the set of mentions identified in the text, and E represents the set of entities within the target knowledge base. This seemingly straightforward mapping is, in reality, a complex process, and one of its most significant challenges is **ambiguity.**

[9] https://www.w3.org/TR/rdf-sparql-query/ (Berners-lee 2006)
[10] https://www.w3.org/TR/rdf-schema/
[11] https://www.ontotext.com/knowledgehub/fundamentals/what-is-entity-linking/

Ambiguity

The prevalent issue of ambiguity arises from our ever expanding vocabulary where numerous words with multiple meanings emerge and gain widespread use, contributing to linguistic phenomena such as homonymy and polysemy becoming increasingly common. Homonymy refers to situations where a word or sequence of words has the same spelling, but also multiple *unrelated* meanings, (e.g., "bat" the animal vs. the club). Polysemy, on the other hand, involves words spelt the same with multiple *related* meanings (e.g., "line as a row, a mark, or a connection). These phenomena, homonymy and polysemy, are prime examples of the ambiguity that constitutes one of the most significant challenges when it comes to entity linking.

This inherent ambiguity in human language poses a substantial challenge for machines attempting to understand and process text. Unlike humans, whose disambiguation of phenomena such as homonymy and polysemy occurs often intuitively through understanding of the context behind the text as well as prior knowledge, entity linking requires explicit mechanisms and access to structured information to resolve such complexities. A number of modern entity linking systems use deep learning techniques to analyze the context surrounding mentions and predicting its most accurate entity in a knowledge base.[12] These approaches often involve training neural networks to create numerical representations of both mentions and entities, where similarity in representations suggests a likely link. For example, techniques like Recurrent Neural Networks (RNNs)[13] and Transformers[14] are used to process the surrounding text of a mention, capturing the relationships between words. Attention mechanisms within these models allow the system to weigh the importance of different contextual words for determining the correct entity link.

Initial techniques learned representations from the text where entities actually appeared, which would limit their ability to link unseen entities. Building upon this landscape of entity linking methods, this thesis focuses further on ReFinED as an entity linking tool using a zero-shot approach, exploring its specific approach and potential, as well as its integration with koala later touched upon.

---

[12] https://aclanthology.org/P18-1148/, https://aclanthology.org/D17-1277/
[13] https://www.ibm.com/think/topics/recurrent-neural-networks
[14] https://www.ibm.com/think/topics/transformer-model

Beyond the problem of ambiguity, several other hurdles complicate the development of robust and accurate entity linking systems. These include the variability in how entities are referred to in text, occasional scarcity of required contextual information, the emergence of novel entities, and the growing demands for scalability in processing vast amounts of data.

Naming variations

There exists a wide range of forms certain meanings may be portrayed in. For example: ‘USA’, ‘United States’, and ‘The United States of America’ all point to the same entity in a knowledge base, yet spelt portrait differently. Simple matching algorithms may fail in recognizing these, hence the need for better solutions.

Lack of context

A critical factor when accurately resolving ambiguous entity mentions is its surrounding context. When a mention appears in isolation or with little text, it becomes very difficult for an entity linking system to determine its intended entity. An apple by itself may mean many things: The fruit, the company, or even a video game. Surrounding words provide important clues needed to narrow the accuracy of a predicted entity.

The importance of context is particularly important when dealing with ambiguity. As discussed earlier with homonyms and polysemes, it is the contextual words that allow both human and machine to discern the various potential meanings of a single word. Without context, even deep learning models with sufficiently trained data struggle to produce an accurate prediction.

Introduction of novel entities

As the world constantly evolves in many areas not just language but also globalisation, people, organizations, products, and much more emerge to continuously expand our vocabulary. While many knowledge bases may be vast with information, they're not always up-to-date with the latest information. Because of this, they may lack information about new entities, possibly never heard of before which would not be present in the knowledge base. Many entity linking systems therefore struggle with the challenge of non-nil entity linking, referring to entities outside its knowledge base.

Identifying these novel entities is therefore important in real-world applications, especially those dealing with current events or rapidly changing domains. Linking every mention to an existing entity, even if incorrect, can lead to the spreading of misinformation and flawed analysis, introducing even further problems beyond entity linking.

OVER DETTE ER BRUKT I OVERLEAF KAPITTEL 1 / 2 !

==========

Introduction of ReFinED

The challenges of linguistic ambiguity, limited contextual information, naming variability, and the continuous emergence of novel entities, highlight the limitations of traditional entity linking approaches that rely heavily on surface-level matching or fixed knowledge base coverage. While earlier methods addressed ambiguity by learning representations from contexts in which entities explicitly appeared, such approaches struggle to generalize to unseen entities and sparse or evolving domains. This creates a need for entity linking systems that can robustly disambiguate mentions using contextual understanding while remaining adaptable to entities not encountered during training.

ReFinED directly addresses these challenges through a unified, context-aware, and zero-shot entity linking approach. ReFinED is an entity linking (EL) system that links entity mentions in documents to corresponding entities in large-scale knowledge bases such as Wikipedia and Wikidata. It is designed to operate efficiently on web-scale datasets, achieving high accuracy while maintaining relatively low computational requirements, although GPU acceleration is recommended for optimal performance.

ReFinED leverages Transformer-based neural architectures to jointly perform mention detection, fine-grained entity typing, and entity disambiguation within a single forward pass.

ReFinED is therefore explored as a central entity linking solution due to its ability to mitigate ambiguity, operate under limited context, and adapt to previously unseen entities. Its architecture and zero-shot capabilities form the foundation for later integration with Koala, where human interaction further enhances the reliability and interpretability of entity linking results

.

Introduction and utilization of Wikidata

To support this zero-shot entity linking process, ReFinED relies on large-scale structured knowledge bases, most notably Wikidata, which provides the semantic foundation required for candidate retrieval, comparison, and evaluation. Entity linking relies on a structured data source where entities already exist with links between them. These knowledge bases allow systems to compare segments of text with other existing entities and find desired candidates from relevant comparisons. Wikidata is one of these knowledge bases. It's a free and open knowledge base with a wide range of entities, with related data from their other projects, such as Wikipedia, linked together through unique identifiers (QIDs). ReFinED makes use of QIDs to find candidates based on context, and to compare found candidates with desired candidates to calculate accuracy, which is essential to measure performance. To utilize Wikidata in this matter, we use Datasets with queries to measure these accuracies and runtimes

Datasets

To benchmark results with ReFinED and compare them to competing solutions, the same set of data needs to be used to compare results on the same ground. This is crucial to understand where the ReFinED solution excels or lacks. To simply benchmark some capabilities, ReFinED was tested on some simple custom datasets to make an estimation of how well it performed, before testing against the main datasets from the SemTab challenge.

SemTab

Our goal is to evaluate how well our approach performs on the SemTab challenge datasets. The SemTab challenge focuses on benchmarking systems for semantic table interpretation, specifically targeting the problem of matching tabular data to knowledge graphs. From this challenge some tables that's being tested include Round_T2D, Round3, Round4, 2T-2020, HardTableR2, HardTableR3, from the 2020 and 2022 SemTab Challenge. The T2D and 2T tables are tough tables, which refers to a “hard” dataset used in SemTab 2020 that contains particularly challenging tables for entity linking and semantic annotation, often with noisy column headers, sparse metadata, and difficult context for linking. The Round 3 and 4

include some variety of challenges, stressing entity linking, typing and identification, with the round 4 also including some entries from the tough tables. The hard tables are also supposed to challenge these factors.

Custom datasets

Using custom datasets allows ReFinED to be adapted and optimized for specific domains or applications where general data (like Wikipedia) may not fully represent the target content. While ReFinED's pretrained model performs well on broad, open-domain text, specialized fields often contain unique terminology, entity types, or relationships that are not covered in standard datasets. The datasets Companies, Movies and SN are more-so to test capabilities of the ReFinED solution, while EL_challenge is to really test with some unlabeled form of data. To calculate accuracy we test results of each chosen candidate to a ground truth answer sheet for each of the datasets.

- **Companies**

  This dataset contains a table of organizations and some describing attributes; Company, Founded Year, Website, X (Twitter), Employees, Coordinates, Founded by, Industry, Country, Headquarters
- **Movies**

  This dataset contains a table of movies and some describing attributes; Title, Release_Date, Duration, Genre, Country, Language, Director
- **SN**

  This dataset is more revolved around free-text in different languages rather than concrete and direct data. It contains these attributes: Buyer, Street, Locality, Postcode, Country
- **EL_challenge**

  This dataset contains a lot of different tables where columns are nameless and data seems like words and numbers. The words are of some correlation, but the numbers are basically only machine readable without context in most of the tables.

Model Training

The way ReFinED can handle challenging tasks like complex datasets, is by using algorithms and trained models. ReFinED is a Transformer-based end-to-end entity linking model that performs mention detection, fine-grained entity typing, and entity disambiguation within a single forward pass. This joint formulation is central to understanding how the model interacts with both the custom datasets and the SemTab challenge data.

The model is primarily trained using Wikipedia hyperlinks as distant supervision. Hyperlinked spans provide large-scale mention–entity pairs, while associated Wikipedia and Wikidata information supplies entity descriptions and fine-grained type labels. During training, the model learns to identify mention spans, predict detailed semantic types, and match mentions to entities based on contextual similarity to entity descriptions.

Fine Tuning

While the model training gives ReFinED a good ground for entity linking and typing, fine tuning can increase accuracy in more specific settings, without the same large amount of training data model training uses. This is especially useful in our case where complex datasets with strange patterns are being tested.
ReFinED supports fine-tuning to improve performance in such particular use cases or datasets. After being pretrained on large, general data from Wikipedia (covering millions of entity mentions), the model can be further trained on domain-specific or task-specific data to adapt its predictions.
ReFinED's fine-tuning process is designed to be efficient and modular. Users can load the pretrained model (e.g., wikipedia_model) and fine-tune it into a domain-adapted version (e.g., aida_model), allowing for quick customization without retraining from scratch. This is where we can insert custom datasets to be fine tuned on, with different EL Models.

Alligator

When we evaluate the results from testing on those datasets we also compare the results with other EL approaches that's been tested on the same sets. Alligator is one of those approaches.

Alligator is an open-source Python library developed to support entity linking within structured tabular datasets. Entity linking is the process of identifying and disambiguating entities so that references across different tables and data sources can be connected to the same real-world identifier. This task is central to data integration, knowledge graph construction, and advanced analytics because it enables coherent interpretation of disparate small bits of information across columns and tables.

Designed with scalability in mind, Alligator offers features including automatic detection of named-entity and literal column types, multiprocessing support, and machine learning-based ranking and reranking of candidate entities. It can be integrated into data pipelines to improve the quality of downstream tasks like data cleaning, fusion, search, and semantic enrichment. Alligator typically uses external entity retrieval services and caching mechanisms to improve performance over large datasets.

Crocodile

With the recent addition of crocodile, which is a somewhat easier to use approach based on the concept of Alligator, we can also compare against results of this approach. Crocodile is a companion Python library with a focus similar to Alligator: it implements efficient entity linking for tabular datasets. Like its counterpart, Crocodile assists in resolving ambiguous textual references in tables to canonical entity identifiers, enabling richer, semantically-aware data processing. The repository emphasizes smooth integration into data workflows and includes both a command-line interface (CLI) and a Python API for user flexibility.

Crocodile's functionality includes scalable entity linking operations, explicit configuration for column types (including which fields should be treated as named entities versus literals), and support for connecting to external entity retrieval endpoints. By providing tools to connect raw input tables with structured knowledge sources, Crocodile helps bridge the gap between raw data and entities recognized across systems, improving interoperability and analytical accuracy.

Koala UI

To simplify comparing results, accuracy and so on, we use Koala UI to visualize the different approaches. Koala_UI is an open-source, user-friendly web application designed to visualize and interact with entity linking results over tabular data. Where many entity linking tools focus solely on back-end algorithms, Koala_UI fills an important gap by offering an intuitive front-end environment that makes linking results accessible and inspectable by human users. This is particularly important in workflows where data quality, error correction, and human-in-the-loop verification improve the robustness of downstream analytics and decision-making.

Ollama

To integrate the ReFinED approach to Koala UI we have to use an API, and Ollama is the tool we will be using for that. Ollama is an open-source platform and runtime environment that enables users to run large language models (LLMs) locally on their own machines instead of relying on cloud-based services. Designed to support a variety of open-source models, such as LLaMA, Mistral, Gemma, and DeepSeek. Ollama provides a simple yet powerful interface to download, manage, and interact with these models via a command-line interface (CLI), REST API, or associated libraries.

==========

Typisk essay struktur:

1. Introduksjon
    a. Introdusere tema (semantisk teknologi i dette tilfellet)
    b. Forklare hva det er, hvorfor det er “viktig” / spiller noen rolle
    c. Forklare fokuset på essayet, “ideen” som blir utforsket på (thesis statement ?)
2. Hoveddel (body)
    a. Paragrafer som tar for seg en ide som støtter fokusområdet
    b. Inneholder forklaringer, analyser, bevis, eksempler, ..

    c. Utforsker temaet
3. Konklusjon
    a. Summere / konkludere hovedpoengene
    b. Gjenta essay fokuset / ideen på en "ikke-gjentagede" måte
    c. I noen tilfeller, ender slik at leseren "tenker videre" på et eller annet. Typ et ubesvart spørsmål.

==========

Evaluering:

- cell-level prediction vs row-level prediction
  (står under ideer også)

(andre mulige delkapitler)

The Role of Entity Linking

Dette delkapitlet kan gå mer inn i det "formelle" mtp definisjoner og forklaring av hva entity linking faktisk er.

- ~~"Hva er entity linking"~~

- "Hvorfor er det essensielt i moderne semantiske / informasjon systemer"
    - Context, ambiguity, knowledge graph population, improved search, data integration

- Entity linking process overview
    - detection, candidate generation, disambiguation, linking

- eksempler (f.eks. filmene vist i koala)
    - (kan bruke bilder ?)

- Formål / Purpose
    - redegjøre og forklare rollen som tillater semantisk reasoning, anbefalinger, analyse…
    - “foundational link between unstructured and structured knowledge bases”

Understanding Key Semantic Concepts

Delkapittel for videre utdyping og forklaring av diverse begreper som er viktig.

- Knowledge graphs
    - hva de er, hvordan entities connects, hvordan triples lagrer konsepter

- ~~Entity Disambiguation~~
  ~~(dvs å spisse inn betydning og gjøre det klarere vha. kontekst og annet)~~
  ~~(i.e. Apple er et selskap, ikke frukten)~~

- RDF / RDFS / OWL (?)
    - liten recap / kjapp summering av formelle definisjoner?

- Reasoning
- Entailment / Inference
    - definisjoner og eksempler

- Linked data principles
    - f.eks. “Tim Berners-Lee’s 4 rules -> use URIs, link out, … etc”

- Hvorfor disse begrepene er viktig for forståelsen og bruken av entity linking og tilsvarende systemer

Challenges in Entity Linking for Information Systems

limitasjoner, utdype problemer som bl.a. nevnt ovenfor
Forslag:

- ~~**Ambiguity and context**~~
- ~~dvs, at noe kan ha flere betydninger / tolkningen~~
    - ~~+ hvordan betydningen er sterkt knyttet til kontekst, og hvordan reasoning/entailment/inference eller whatever bruker konteksten~~
    - ~~i.e. “The Joker” kan bety både filmen og kortet.~~
    -
- ***domain-specific difficulties***
- Data mangel i visse “knowledge baser” for veldig spesifikke temaer
    - hvis mye data mangler (eller det er mye, men kritisk data mangler) så kan konsekvensene være at ingenting gir mening elns

- Konsekvensene av problemene.
    - bl.a. spredning av feil informasjon (“misinformation propagation”)

- Hvorfor ordentlig data linking er viktig og kreves av blant annet AI, knowledge graphs, data analytikere, osv

Overview of Entity Linking Approaches

(disse vet vi ikke mye om, men forslag som kan søkes opp og undersøkes)

- String-based matching
- Statistical / ML-based linking
- Neural models / Transformers for EL
- Hybrid and knowledge-based systems

- Plasser og kategoriser våre verktøy i disse og utdyp

Evaluation

Egen (del)kapittel som handler om evaluering av maskingslæringsmodeller som ReFinED (som er spesifikt en Transformer modell). Hva generell evaluering går ut på, ulike måling (F1, Recall, Precision. … in4050), og hvordan vi evaluerte ulike modeller fra ReFinED.

Tools and technologies

Introduser, forklar og sammenlign:

- ReFinED
- Koala UI
- Alligator
- OpenRefine (OpenRefine)
- SPARLQ endpoints (?)
- Ontology editors (?)
- Noe om wikidata?

- Github / andre praktiske verktøy

Use Case Scenarios / Practical Applications of Semantic Technologies

(Basically et delkapittel med eksempler for å vise hvordan semantiske verktøy og entity linking er brukt i forskjellige domener)

Benchmark (time)

“Raw” output from short run (1)

[Environment Info]
Python: 3.8.20 (default, Sep 7 2024, 18:35:08)
[GCC 11.4.0]
PyTorch: 2.4.1+cu121
NumPy: 1.24.4

[Manual Timing & Per-Text Timing]
FutureWarning: torch.cuda.amp.autocast(...) is deprecated.

Processed 100 texts in 5.22s
Average per-text: 0.0522s
Min: 0.0171s, Max: 0.3730s
Peak memory usage: 4.69 MB

[cProfile Profiling]
1969210 function calls (1803504 primitive calls) in 1.506 seconds

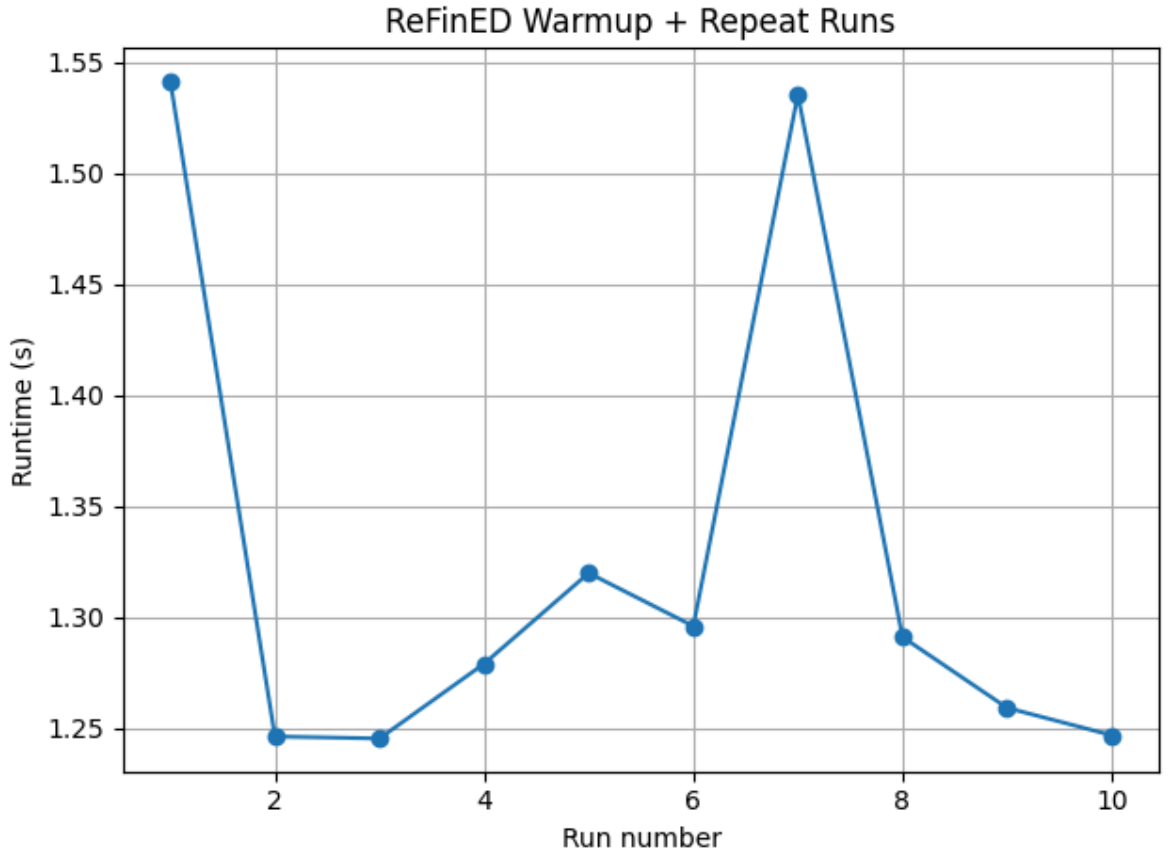
Ordered by cumulative time (top 20 functions):

ncalls	tottime	percall	cumtime	percall	function
1	0.000	0.000	1.513	1.513	benchmark.py:run_refined
1	0.002	0.002	1.513	1.513	benchmark.py:<listcomp>
100	0.002	0.000	1.511	0.015	processor.py:process_text
100	0.029	0.000	1.323	0.013	processor.py:process_tensors
23900/100	0.011	0.000	1.034	0.010	module.py:_wrapped_call_impl
23900/100	0.028	0.000	1.034	0.010	module.py:_call_impl
100	0.004	0.000	1.033	0.010	refined_model.py:forward
100	0.002	0.000	0.691	0.007	modeling_roberta.py:forward
100	0.003	0.000	0.640	0.006	modeling_roberta.py:forward
1200	0.004	0.000	0.631	0.001	modeling_roberta.py:forward
1200	0.003	0.000	0.422	0.000	modeling_roberta.py:forward
1200	0.045	0.000	0.324	0.000	modeling_roberta.py:forward

7700	0.007	0.000	0.308	0.000 linear.py:forward
7700	0.298	0.000	0.298	0.000 {built-in method torch._C._nn.linear}
1200	0.005	0.000	0.202	0.000 modeling_utils.py:apply_chunking_to_forward
200	0.000	0.000	0.189	0.001 module.py:eval
56200/200	0.041	0.000	0.189	0.001 module.py:train
100	0.022	0.000	0.186	0.002 refined_model.py:_identify_entity_mentions
1200	0.002	0.000	0.166	0.000 modeling_roberta.py:feed_forward_chunk
57400	0.046	0.000	0.096	0.000 module.py:__setattr__

[Repeat Runs Timing]
Run 1: 1.21s
Run 2: 1.61s
Run 3: 1.31s
Average over 3 runs: 1.38s

[Warmup + Repeat Runs Timing] (10 runs)
Warmup Run (Run 1): 1.51s
Run 2: 1.31s
Run 3: 1.28s
Run 4: 1.23s
Run 5: 1.31s
Run 6: 1.25s
Run 7: 1.51s
Run 8: 1.24s
Run 9: 1.25s
Run 10: 1.32s
Average (excluding warmup) over 9 runs: 1.30s



(warmup_runs_imdb_top_100.csv.png

“Raw” output from long run (2)

[Environment Info]
Python: 3.8.20
PyTorch: 2.4.1+cu121
NumPy: 1.24.4

[Manual Timing & Per-Text Timing]
FutureWarning: torch.cuda.amp.autocast(...) is deprecated.

Processed 1000 texts in 21.38 s
Average per-text: 0.0214 s
Min: 0.0109 s, Max: 0.5865 s
Peak memory usage: 102.71 MB

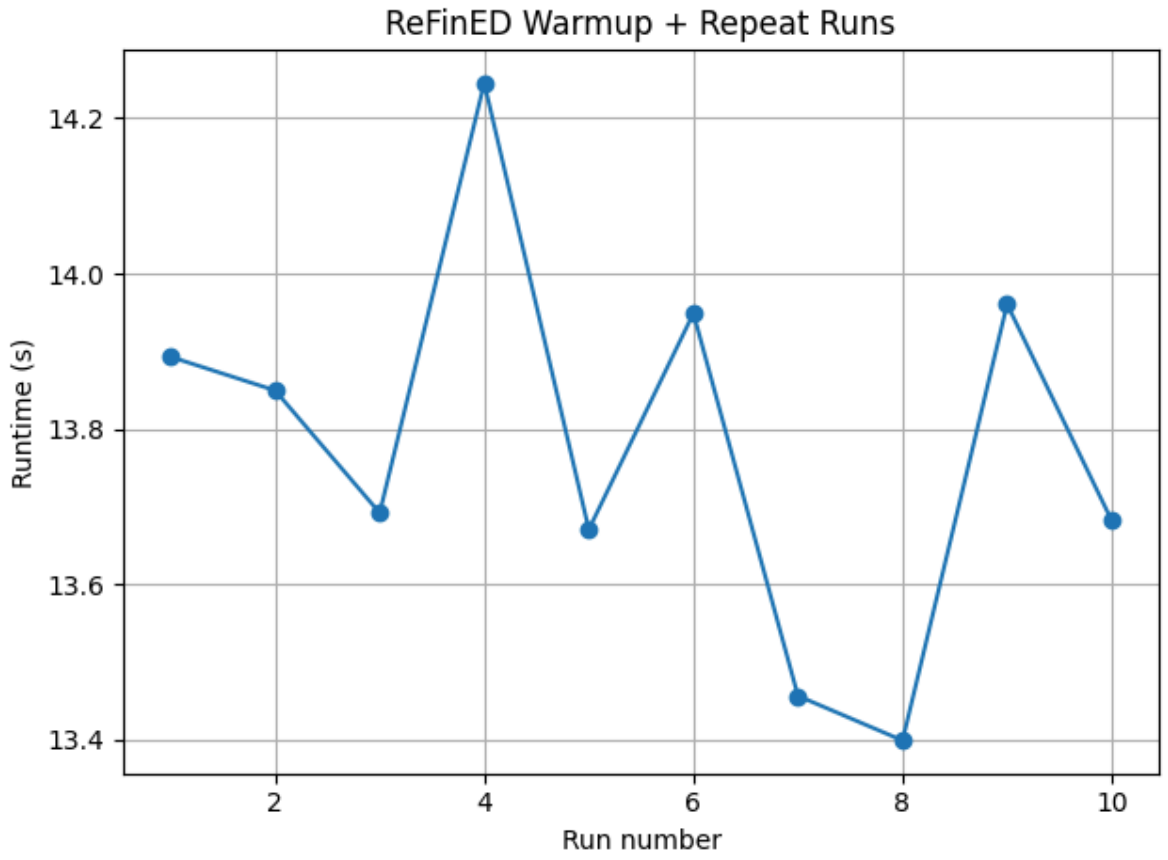
[cProfile Profiling]
21990891 function calls (20364554 primitive calls) in 15.128 seconds

Ordered by: cumulative time
List reduced from 446 to 20 due to restriction <20>

ncalls	tottime	percall	cumtime	percall	function
1	0.000	0.000	15.445	15.445	benchmark.py:run_refined
1	0.010	0.010	15.445	15.445	benchmark.py:<listcomp>
1000	0.021	0.000	15.436	0.015	processor.py:process_text
1000	0.489	0.000	14.093	0.014	processor.py:process_tensors
239000/1000	0.101	0.000	10.153	0.010	module.py:_wrapped_call_impl
239000/1000	0.259	0.000	10.152	0.010	module.py:_call_impl
1000	0.034	0.000	10.144	0.010	refined_model.py:forward
1000	0.013	0.000	6.457	0.006	modeling_roberta.py:forward
1000	0.025	0.000	6.030	0.006	modeling_roberta.py:forward
12000	0.033	0.000	5.958	0.000	modeling_roberta.py:forward
12000	0.023	0.000	3.977	0.000	modeling_roberta.py:forward
12000	0.423	0.000	3.058	0.000	modeling_roberta.py:forward
77000	0.066	0.000	2.947	0.000	linear.py:forward
77000	2.858	0.000	2.858	0.000	{built-in method torch._C._nn.linear}
1000	0.202	0.000	2.266	0.002	refined_model.py:_identify_entity_mentions
12000	0.042	0.000	1.918	0.000	modeling_utils.py:apply_chunking_to_forward
2000	0.001	0.000	1.752	0.001	module.py:eval
562000/2000	0.381	0.000	1.751	0.001	module.py:train
12000	0.020	0.000	1.582	0.000	modeling_roberta.py:feed_forward_chunk
7192	0.695	0.000	1.346	0.000	processor.py:<listcomp>

[Repeat Runs Timing]
Run 1: 13.32 s
Run 2: 13.14 s
Run 3: 13.43 s
Average over 3 runs: 13.30 s

[Warmup + Repeat Runs Timing] (10 runs)
Warmup Run (Run 1): 13.89s
Run 2: 13.85s
Run 3: 13.69s
Run 4: 14.25s
Run 5: 13.67s
Run 6: 13.95s
Run 7: 13.46s
Run 8: 13.40s
Run 9: 13.96s
Run 10: 13.68s
Average (excluding warmup) over 9 runs: 13.77s



(warmup_runs_companies_test.csv.png)

(FORMATTER DETTE PLS ! (gjelder også benchmark description, er direkte kopiert fra kode-fil så er egentlig ikke riktig)
. f.eks. riktig tabs / indents, fet skrift, kursiv., mellomrom osv

Benchmark description:

The benchmark (benchmark.py) is a script for measuring ***runtime performance and memory usage*** of the ReFinED entity linking pipeline
on a subset of the IMDb dataset (first 100 rows) and a longer data file (companies_test.csv), but works on any given csv.
It focuses specifically on the core pipeline:

1. `Refined.from_pretrained(...)`
- Loads the pre-trained ReFinED model for entity linking.
- Includes transformer-based components (RoBERTa) and numerical normalization layers.
- Only loaded once; initialization time is not included in per-text processing time.

2. `refined.process_text(text)`
- Main entity linking function tested per text.
- Converts raw text into tokenized tensors.
- Passes tokens through the model to detect and classify entity mentions.
- Produces spans with predicted entities, candidate entity rankings, coarse types (e.g., DATE, NUMERIC), and normalized values.)
- This function is the *primary hotspot* measured in this benchmark.

3. Timing Measurements
- **Manual timing** using `time.perf_counter()` for total and per-text runtime.
- Measures wall-clock time for CPU execution of 100 texts.
- Also prints min/max/average per-text timings for insight into variability.

4. Detailed Profiling with `cProfile`
- Collects **function-level profiling data** including_
  * Total calls
  * Cumulative time
  * Per-call time.
- Highlights major bottlenecks:
  * Transformer forward passes (RoBERTa)
    - Transformers perform self-attention across all tokens. For n tokens, attention is roughly $O(n^2)$ operations.
    - Most of the time benchmark spends in this step.

  * Linear layers (fully connected layers dominate runtime)
    - Tasks like Mention detection, Entity classification, Scoring candidate entities.
    - Expensive as it involves matrix multiplication, computationally heavy tasks especially on larger embedding dimensions.

* Preprocessing steps (`process_tensors`) in ReFinED
- Converting raw token IDs and features to tensors for the model.
- Noticeable in cProfile due to pure Python-level work, as apart from linear layers running in native code.

- Captures Python-level computation time, not full native memory usage of PyTorch tensors.

5. Repeat Run Analysis
- Runs the same batch multiple times to account for initialization overhead and caching effects.
- Provides a stable average per-run runtime.

6. Notes
- **Excluded operations**: Online wikidata queries and candidate label fetching are deliberately omitted
to focus purely on **local model inference performance**.
- Memory profiling via `tracemalloc` captures Python allocations only, not full PyTorch tensor memory.
- This benchmark is suitable for **relative comparisons**, e.g. testing different model sizes, batch sizes, or hardware.

Purpose:
- Provides precise runtime and profiling data for **entity linking performance** of ReFinED.
- Identifies computational bottlenecks.
- Forms the basis for further optimization or resource planning

Benchmark Report:
ReFinED RoBERTa Entity Linking

Environment

Component	Version
Python	3.8.20
PyTorch	2.4.1 + CUDA 12.1
NumPy	1.24.4
System	Linux / GCC 11.40 (Ubuntu)

System info

1. **CPU:**
    a. 13th Gen Intel(R) Core(™) i5-13600k,
    b. 3500 MHz,
    c. 14 Core(s),
    d. 20 Logical Processor(s)
2. **GPU:**
    a. NVIDIA GeForce RTX 4070,
    b. 12.3 GB VRAM,
    c. CUDA 12.9,
    d. Driver 576.88
3. **RAM**:
    a. 32 GB,
    b. Speed: 3600 MHz
4. **OS**:
    a. **Host OS:** Windows 10 Pro, 64-bit build 19045
    b. **Runtime OS:** Ubuntu 22.04 LTS (WSL2)
    c. **WSL Version:** WSL:2

1. Short Run: 100 IMDB Texts

Benchmark Setup

Dataset: First 100 lines of `imdb_top_100.csv`

Model: `ReFinED` with RoBERTa-based entity linker (`wikipedia_model_with_numbers`)

Runs: Single-text sequential processing for manual timing, repeated 3 times for warm runs

Measured Metrics:

- Per-text processing time
- Total throughput
- Peak Python memory usage
- Function-level CPU profiling
- Repeat-run timing for consistency

Manual Timing Results

Metric	**Value**
Total time per 100 text	5.22 s
Average per text	0.052 s
Min / Max per text	0.017 s / 0.373 s
Peak Python memory	4.69 MB

Comment: Variability in min/max per text is due to different text lengths. Peak memory only tracks Python heap, not GPU VRAM.

Repeated Runs Timing

Run	**Time (s)**
1	1.21
2	1.61
3	1.31
Average	1.38

Comment: Warm runs are faster per 100 texts due to caching / GPU kernel initialization.

Profiler Highlights (cProfile)

- Most time spent inside **RoBERTa forward passes** → 1.3 s of cumulative time.

- Significant CPU time is also spent in **linear layers**, which is expected for transformers inference.
- Overhead from pipeline / preprocessing functions is minimal (<5%).

Comment: Profiling confirms **model inference is the main bottleneck**, not Python code or I/O

Summary and Key Observations, run 1

- Sequential processing of 100 texts is **fast** (~52 ms per text) for a RoBERTa-based entity linker.
- Memory footprint is small on the Python side. Actual GPU usage not measured here.
- Warm runs improve throughput (1.38 s per 100 texts vs initial 5.22 s).
- Profiling identifies clear focus for optimization: **batched processing** and **mixed precision** for faster GPU inference.

Comment:
- GPUs are optimized for parallel computation, whereas here many small operations are run sequentially.
- Better utilization of GPU could be e.g. forwarding batches of texts, e.g. 16 at a once.

5. Long Run: 1000 lines of company info (company_test.csv)

Benchmark Setup

Dataset: 1000 rows of `companies_test.csv`

Model: `ReFinED` with RoBERTa-based entity linker (`wikipedia_model_with_numbers`)

Runs: Single-text sequential processing for manual timing, repeated 3 times for warm runs

Measured Metrics:

- Per-text processing time
- Total throughput
- Peak Python memory usage
- Function-level CPU profiling
- Repeat-run timing for consistency
-

Manual Timing Results

Metric	Value
Total time per 1000 text	21.38 s
Total time per 100 text	2.138 s
Average per text	0.0214 s
Min / Max per text	0.0109 s / 0.5865 s
Peak Python memory	102.72 MB

Comment: Variability in min/max per text is due to different text lengths. Peak memory only tracks Python heap, not GPU VRAM.

Repeated Runs Timing

Run	Time (s)
1	13.32
2	13.14
3	13.43
Average	13:30

Comment: Warm runs are faster per 100 texts due to caching / GPU kernel initialization.

Profiler Highlights (cProfile)

- **Total calls**: ~21.99M
- **Cumulative CPU time:** 15.13 s
- **Top bottlenecks:**
    - `process_text` / `process_tensors`: 15.4 / 14.1
    - RoBERTa forward passes: 6.5 / 6.0 / 5.96 / 3.98 s
    - Linear layers: 2.86 / 2.95 s cumulative
    - Entity mention identification: 2.27 s cumulative

Comment: Most CPU time is spent inside RoBERTa and linear layers. Preprocessing overhead is minimal (<5%)

Summary and Key Observations, run 2

- Sequential processing of 1000 texts is **fast** (~21 ms per text)

- Python memory footprint remains modest. Actual GPU memory not tracked.
- Warm runs reduce runtime (13.30 s vs initial 21.38 s).
- Profiling confirms **model inference is still the main bottleneck.**

Comment:

- The sequential approach underutilizes GPU parallelism. **Batch processing** (e.g. 16-32 texts at once) and **mixed precision** could further reduce runtime and improve throughput.

Bottlenecks

1. **RoBERTa forward passes (main bottleneck)**
    - for the long run: ~6-7 s per 1000 texts.

    <u>What is going on</u>
    - Text is tokenized and run through a transformer.
    - Forward pass involves multi-head self-attention + feed-forward layers
        - Multi-head self-attention (MHSA)
            - tokens look at other tokens in sequence, compute relevance scores through attention algorithm.
            - Multi-head as it is run in parallel each with a «head» that focuses on different relationships.
            - Bottleneck due to large matrix multiplications.
        - Feed-Forward Network (FFN)
            - Two linear layers with an “activation” in between where first layer expands a tokens vector
            - Activation adds non-linearity to model complex patterns
            - second layer shrinks tokens vector back down
            - Applied to every token, but no “communication” like in MHSA

    <u>Why costly</u>
    - transformers scale roughly quadratically with sequence length due to attention, so longer texts / batches get expensive fast

- In a shorter run, alone accounts for ~45% of time. In the long run still the largest single cost.

  <u>Takeaway</u>
    - A big spot for potential improvements or area of interest. Batching and mixed precision could give a potential speedup result.
    - Running one forward per text wastes GPU parallelism

2. **Entity mention identification**
    - Relevant function: `refined_model.py:_identify_entity_mentions`
    - Cumulative time: ~2.2 s per 1000 texts

    What is going on
    - Pipeline scans output to find spans in text that could be entities, done after the transformer runs.
    - Involves logic, indexing, and math.

    Why costly
    - Python-driven (looping, conditional logic…) not optimized tensor code.
    - CPU-bound, unlike linear layers.
    - In the short run, smaller bottleneck, but in the long run it adds up.

    Takeaway
    - Parallelizing or vectorizing in this step (e.g. process multiple texts in one batch rather than per text) would help. Mixed precision wont.

3. **Warmup overhead (first run vs repeated runs)**

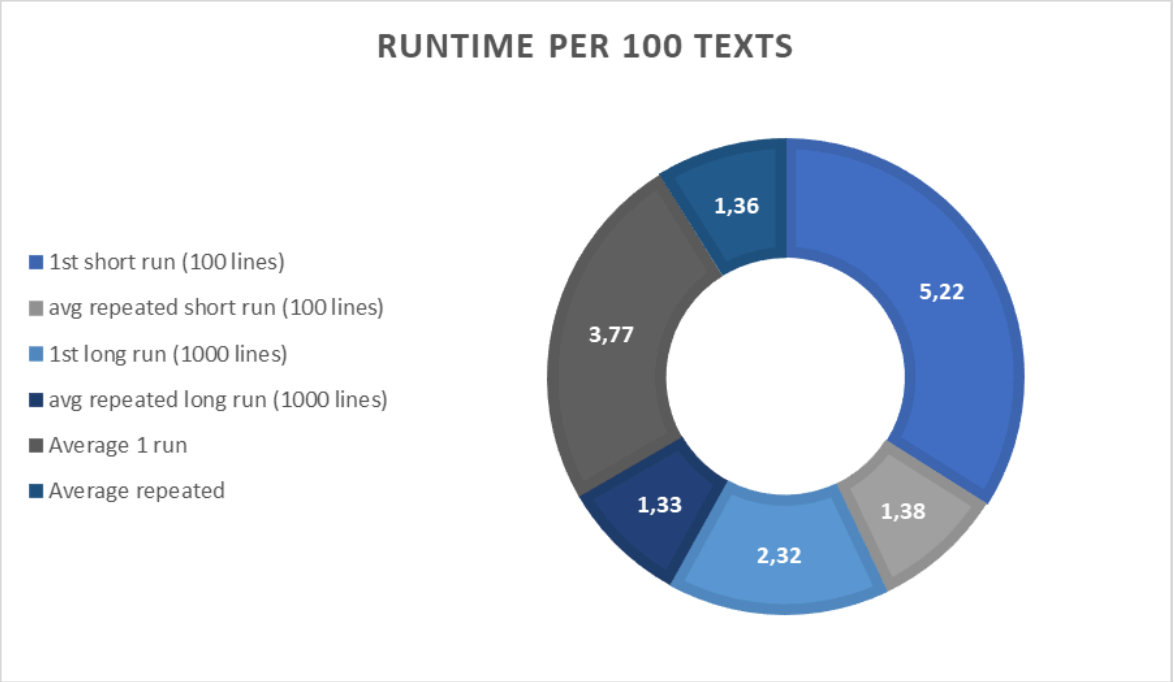
    What is going on
    - As in results of e.g. short run, the first run of 100 texts gave 5.22 s, whereas later runs gave ~1.3s.
        - The 5.22 s first run is inflated by setup overhead.
        - The ~1.3 s repeated runs show true steady-state throughput.

    Why costly
    - Costly due to the first run makes the system have to do extra one-time setup work that doesn't happen in later runs.
    - Specifically:
        - Model weights transfer to GPU:
            - Copying MBs of RoBERTa/ReFinED parameters from CPU RAM to GPU VRAM
        - CUDA kernel compilation / initialization
            - GPU libraries needed to build or load optimized kernels for hardware
        - Cache filing
            - things like tokenizer vocabularies lookups, indexing entities or other initializations

    Takeaway
    - Usually, always discard the first run in benchmarks, as the “true” throughput is shown in the repeated runs.

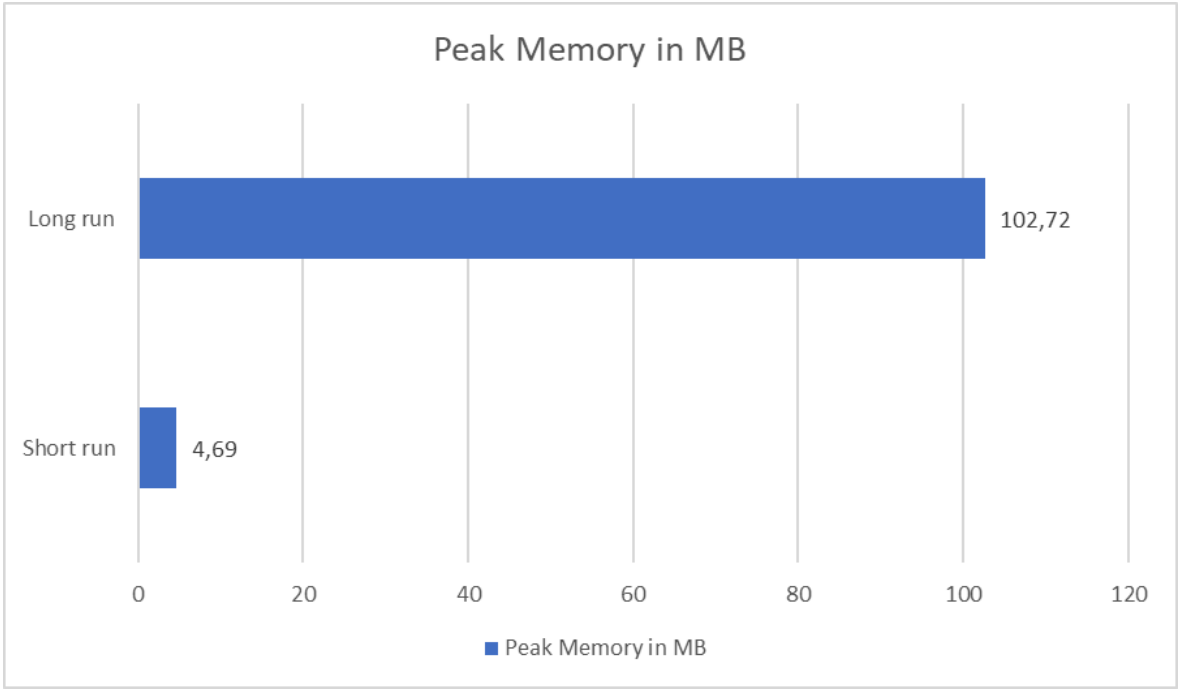
Summary

Comparison harts



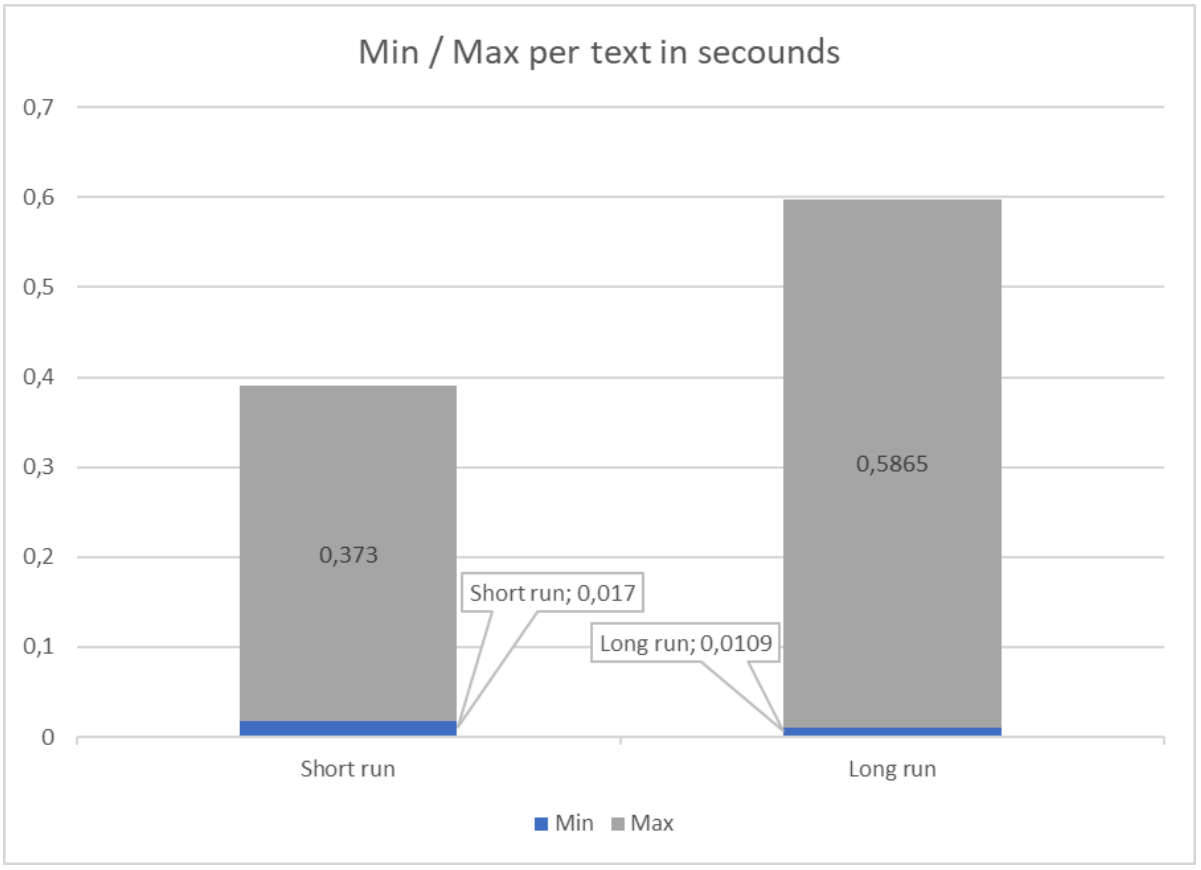
Values shown in seconds

As previously mentioned, the first run will yield slower results, mainly due to empty cashes / kernel initialization. For warm runs, the difference for each 100 texts is not that noticeable, but with larger runs the average time is slightly lower.



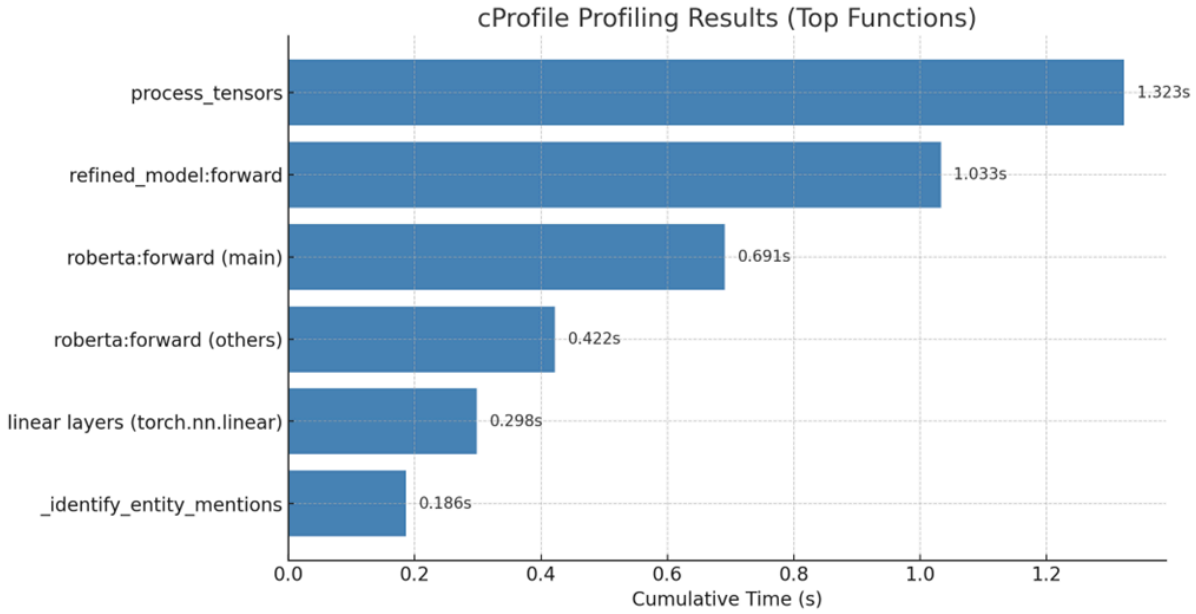
Values shown in MB (Megabytes)

As expected a longer run takes more memory, due to factors such as cashing, kernel initialization and more data to handle. Long runs handles 1000 lines, which is 10x more than shorter runs, but memory for long runs is actually ~22 times greater than short runs!



Values shown in seconds

The variations from max and min times is due to different sized texts, and the reason for a difference from short to long run is likely due to the long run having more text samples.



made with gpt

The profiling shows that most runtime is spent in tensor preparation (process_tensors) and the model forward passes (refined_model.forward and RoBERTa layers). Linear layers and entity mention identification are smaller but still notable costs. This confirms the main bottleneck is transformer inference, not Python overhead.

CODE

```python
# ========================================
# ReFinED Benchmarking Script
# ========================================

from my_tests.process_file import process_csv
from refined.inference.processor import Refined
import sys
import os
import time
import cProfile, pstats
import tracemalloc
import sys
import torch
import numpy as np



# ================== CONFIG ==================
REPEAT_RUNS = 3           # Repeat runs to account for cold start
TOP_STATS = 20            # Number of cProfile functions to show
DEFAULT_DATA_FOLDER = "my_tests/data"

# ================== HANDLE COMMAND-LINE ARGUMENT ==================
if len(sys.argv) < 2:
    print(f"Usage: python {sys.argv[0]} <input_file>")
    print(f"Supported files:\n- 'imdb_top_100.csv'\n-
'companies_test.csv'")
    sys.exit(1)

TEXT_FILE = sys.argv[1]

if not os.path.exists(TEXT_FILE):
    TEXT_FILE = os.path.join(DEFAULT_DATA_FOLDER, TEXT_FILE)
    if not os.path.exists(TEXT_FILE):
        print(f"File not found: {TEXT_FILE}")
        sys.exit(1)
print(f"[INFO] Using input file: {TEXT_FILE}")


# ================== LOAD DATA ==================
texts = process_csv(TEXT_FILE)


# ================== LOAD MODEL ==================
refined = Refined.from_pretrained(
   model_name='wikipedia_model_with_numbers',
```

```
    entity_set="wikipedia"
)


# ================== HELPER FUNCTION ==================
def run_refined(texts_subset):
    """Processes a list of texts through ReFinED."""
    return [refined.process_text(t) for t in texts_subset]


# ================== ENVIRONMENT INFO ==================
print("\n[Environment Info]")
print("Python:", sys.version)
print("PyTorch:", torch.__version__)
print("NumPy:", np.__version__)


# ================== MANUAL TIMING ==================
print("\n[Manual Timing & Per-Text Timing]")

per_text_times = []
start_total = time.perf_counter()

for t in texts:
    start = time.perf_counter()
    refined.process_text(t)
    per_text_times.append(time.perf_counter() - start)

end_total = time.perf_counter()
total_time = end_total - start_total

print(f"Processed {len(texts)} texts in {total_time:.2f}s")
print(f"Average per-text:
{sum(per_text_times)/len(per_text_times):.4f}s")
print(f"Min: {min(per_text_times):.4f}s, Max:
{max(per_text_times):.4f}s")


# ================== PEAK MEMORY ==================
tracemalloc.start()
run_refined(texts)
_, peak_memory = tracemalloc.get_traced_memory()
print(f"Peak memory usage: {peak_memory / 1e6:.2f} MB")
tracemalloc.stop()
```

```
# ================== CPROFILE PROFILING ==================
print("\n[cProfile Profiling]")
profiler = cProfile.Profile()
profiler.enable()
run_refined(texts)
profiler.disable()

stats = pstats.Stats(profiler).sort_stats('cumtime')
stats.print_stats(TOP_STATS)


# ================== REPEAT RUNS ==================
print("\n[Repeat Runs Timing]")
repeat_times = []
for i in range(REPEAT_RUNS):
    start = time.perf_counter()
    run_refined(texts)
    end = time.perf_counter()
    repeat_times.append(end - start)
    print(f"Run {i+1}: {end - start:.2f}s")

print(f"Average over {REPEAT_RUNS} runs:
{sum(repeat_times)/len(repeat_times):.2f}s\n")
```

==========

Forslag til senere:

1. **Mixed precision**
2. Mixed precision:
    a. “deep learning teknikk” hvor kombinasjon av ulike numeriske presisjoner er brukt dvs. 16-bit og 32-bit floats. kan gjøres for å øke treningen og inference hastigheten og accuracy samtidig.
    b. Hvordan kan *vi* gjøre dette?
        i. PyTorch sin `torch.amp.autocast()’ - noe med denne

==========

```
from torch.cuda.amp import autocast

def run_refined_mixed_precision(texts_subset):
    """Processes a list of texts using mixed precision (FP16 where
safe)."""
    results = []
    refined.eval()  # Make sure model is in eval mode
    with autocast(device_type='cuda', dtype=torch.float16):  # FP16
        for t in texts_subset:
            results.append(refined.process_text(t))
    return results
```

==========

3. **GPU usage**

==========

```
refined = Refined.from_pretrained(...).to('cuda')
```

==========

4. **Batched inference**

==========

```
BATCH_SIZE = 16
for i in range(0, len(texts), BATCH_SIZE):
    batch = texts[i:i+BATCH_SIZE]
    run_refined_mixed_precision(batch)
```

==========

NB!!!!!

- Ingenting av dette (koden) er bekreftet til å fungere
- kode forslag fra GPT
- Testes senere kanskje

Info

Benchmark data for 4 different configurations:

1. CPU + No batching
2. CPU + Batching
3. GPU + No batching
4. GPU + Batching

Companies
Movies
SN

- Contains data such as
    - batch size
    - peak memory usage
    - average over 50 runs
    - + diagram for 8 runs each
-

Summary

- Contains short summary of fastest and slowest runtime between all 3 files
- Includes diagrams for average runs based on batch size

1

======= START BENCHMARK =======

[Environment Info]
Python: 3.8.20 (default, Sep 7 2024, 18:35:08)
[GCC 11.4.0]
PyTorch: 2.4.1+cu121
NumPy: 1.24.4
Running on CPU

[Manual Timing & Per-Text Timing]
Processed 1000 texts in 22.69s
Average per-text: 0.0212s
Min: 0.0123s, Max: 0.0393s

[Peak Memory Usage]
Peak memory usage: 33.08 MB

[cProfile Profiling]
16404710 function calls (14778694 primitive calls) in 23.712 seconds

Ordered by: cumulative time
List reduced from 447 to 20 due to restriction <20>

ncalls tottime percall cumtime percall filename:lineno(function)
1 0.000 0.000 23.754 23.754 /mnt/d/Dokumenter/GitHub/ReFinED/my_tests/refined_utils.py:160(run_refined_single)
1 0.009 0.009 23.754 23.754 /mnt/d/Dokumenter/GitHub/ReFinED/my_tests/refined_utils.py:162(<listcomp>)
1000 0.016 0.000 23.745 0.024 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/inference/processor.py:136(process_text)
1000 0.127 0.000 22.289 0.022 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/inference/processor.py:274(process_tensors)
238960/1000 0.123 0.000 20.270 0.020 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/module.py:1549(_wrapped_call_impl)
238960/1000 0.295 0.000 20.268 0.020 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/module.py:1555(_call_impl)
1000 0.035 0.000 20.261 0.020 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/model_components/refined_model.py:164(forward)
1000 0.017 0.000 18.545 0.019 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:696(forward)

1000 0.033 0.000 18.171 0.018 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:457(forward)
12000 0.047 0.000 18.073 0.002 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:381(forward)
76985 0.075 0.000 14.671 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/linear.py:116(forward)
76985 14.558 0.000 14.558 0.000 {built-in method torch._C._nn.linear}
12000 0.049 0.000 10.829 0.001 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/modeling_utils.py:1721(apply_chunking_to_forward)
12000 0.029 0.000 10.409 0.001 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:444(feed_forward_chunk)
12000 0.027 0.000 7.160 0.001 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:311(forward)
12000 0.364 0.000 5.394 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:175(forward)
12000 0.036 0.000 5.276 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:345(forward)
12000 0.153 0.000 5.026 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:359(forward)
2000 0.001 0.000 2.001 0.001 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/module.py:2459(eval)
562000/2000 0.445 0.000 2.000 0.001 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/module.py:2437(train)

[Repeated Runs]
Warmup Run: 21.54s
Run 1: 21.85s
Run 2: 22.68s
Run 3: 22.23s
Run 4: 22.03s
Run 5: 22.62s
Run 6: 22.21s
Run 7: 20.65s
Run 8: 21.03s
Average (excluding warmup): 21.91s

[Accuracy]

Accuracy: 72.00% (720/1000 correct)

======= END BENCHMARK =======

2

======= START BENCHMARK =======

[Environment Info]
Python: 3.8.20 (default, Sep 7 2024, 18:35:08)
[GCC 11.4.0]
PyTorch: 2.4.1+cu121
NumPy: 1.24.4
Running on CPU
Using Batched mode

[Manual Timing & Per-Text Timing]
Processed 1000 texts in 14.08s (batch mode)
Per-text timing skipped (batch processes them together)

[Peak Memory Usage]
Peak memory usage: 50.84 MB

[cProfile Profiling]
4028719 function calls (3960967 primitive calls) in 13.574 seconds

Ordered by: cumulative time
List reduced from 443 to 20 due to restriction <20>

ncalls tottime percall cumtime percall filename:lineno(function)
1 0.001 0.001 13.601 13.601 /mnt/d/Dokumenter/GitHub/ReFinED/my_tests/refined_utils.py:164(run_refined_batch)
1 0.003 0.003 13.599 13.599 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/inference/processor.py:198(process_text_batch)
63 0.043 0.001 13.449 0.213 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/inference/processor.py:274(process_tensors)
15057/63 0.015 0.000 12.951 0.206 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/module.py:1549(_wrapped_call_impl)
15057/63 0.035 0.000 12.951 0.206 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/module.py:1555(_call_impl)
63 0.003 0.000 12.950 0.206 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/model_components/refined_model.py:164(forward)
63 0.001 0.000 11.781 0.187 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:696(forward)

63 0.005 0.000 11.718 0.186 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:457(forward)
756 0.007 0.000 11.706 0.015 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:381(forward)
4851 0.009 0.000 10.774 0.002 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/linear.py:116(forward)
4851 10.762 0.002 10.762 0.002 {built-in method torch._C._nn.linear}
756 0.006 0.000 7.604 0.010 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/modeling_utils.py:1721(apply_chunking_to_forward)
756 0.005 0.000 7.557 0.010 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:444(feed_forward_chunk)
756 0.003 0.000 4.092 0.005 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:311(forward)
756 0.006 0.000 3.797 0.005 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:345(forward)
756 0.056 0.000 3.745 0.005 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:359(forward)
756 0.071 0.000 3.116 0.004 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:175(forward)
63 0.033 0.001 0.991 0.016 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/model_components/refined_model.py:382(_identify_entity_mentions)
756 0.026 0.000 0.966 0.001 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:278(forward)
1000 0.000 0.000 0.315 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/doc_preprocessing/preprocessor.py:300(add_candidates_to_spans)

[Repeated Runs]
Warmup Run: 13.18s
Run 1: 13.13s
Run 2: 13.07s
Run 3: 13.45s
Run 4: 13.50s
Run 5: 13.39s
Run 6: 13.28s

Run 7: 12.65s
Run 8: 13.23s
Average (excluding warmup): 13.21s

[Accuracy]

Accuracy: 72.00% (720/1000 correct)

======= END BENCHMARK =======

3

======= START BENCHMARK =======

[Environment Info]
Python: 3.8.20 (default, Sep 7 2024, 18:35:08)
[GCC 11.4.0]
PyTorch: 2.4.1+cu121
NumPy: 1.24.4
Running on GPU: NVIDIA GeForce RTX 4070

[Manual Timing & Per-Text Timing]
Processed 1000 texts in 12.90s
Average per-text: 0.0095s
Min: 0.0064s, Max: 0.0197s

[Peak Memory Usage]
Peak memory usage: 33.08 MB

[cProfile Profiling]
16399629 function calls (14773613 primitive calls) in 11.775 seconds

Ordered by: cumulative time
List reduced from 444 to 20 due to restriction <20>

ncalls tottime percall cumtime percall filename:lineno(function)
1 0.000 0.000 12.061 12.061 /mnt/d/Dokumenter/GitHub/ReFinED/my_tests/refined_utils.py:160(run_refined_single)
1 0.005 0.005 12.061 12.061 /mnt/d/Dokumenter/GitHub/ReFinED/my_tests/refined_utils.py:162(<listcomp>)
1000 0.016 0.000 12.056 0.012 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/inference/processor.py:136(process_text)
1000 0.196 0.000 10.922 0.011 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/inference/processor.py:274(process_tensors)
238960/1000 0.090 0.000 8.571 0.009 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/module.py:1549(_wrapped_call_impl)
238960/1000 0.230 0.000 8.570 0.009 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/module.py:1555(_call_impl)
1000 0.029 0.000 8.563 0.009 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/model_components/refined_model.py:164(forward)
1000 0.012 0.000 6.103 0.006 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:696(forward)

1000 0.024 0.000 5.703 0.006 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:457(forward)
12000 0.032 0.000 5.634 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:381(forward)
12000 0.022 0.000 3.769 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:311(forward)
12000 0.406 0.000 2.897 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:175(forward)
76985 0.058 0.000 2.784 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/linear.py:116(forward)
76985 2.704 0.000 2.704 0.000 {built-in method torch._C._nn.linear}
12000 0.041 0.000 1.807 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/modeling_utils.py:1721(apply_chunking_to_forward)
2000 0.001 0.000 1.639 0.001 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/module.py:2459(eval)
562000/2000 0.354 0.000 1.638 0.001 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/module.py:2437(train)
12000 0.019 0.000 1.488 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:444(feed_forward_chunk)
1000 0.163 0.000 1.148 0.001 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/model_components/refined_model.py:382(_identify_entity_mentions)
574000 0.394 0.000 0.830 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/module.py:1731(__setattr__)

[Repeated Runs]
Warmup Run: 10.12s
Run 1: 9.78s
Run 2: 9.86s
Run 3: 9.81s
Run 4: 9.53s
Run 5: 9.87s
Run 6: 9.73s
Run 7: 9.96s
Run 8: 9.56s
Average (excluding warmup): 9.76s

[Accuracy]

Accuracy: 72.00% (720/1000 correct)

======= END BENCHMARK =======

4

======= START BENCHMARK =======

[Environment Info]
Python: 3.8.20 (default, Sep 7 2024, 18:35:08)
[GCC 11.4.0]
PyTorch: 2.4.1+cu121
NumPy: 1.24.4
Running on GPU: NVIDIA GeForce RTX 4070
Using Batched mode

[Manual Timing & Per-Text Timing]
Processed 1000 texts in 2.57s (batch mode)
Per-text timing skipped (batch processes them together)

[Peak Memory Usage]
Peak memory usage: 50.86 MB

[cProfile Profiling]
4030053 function calls (3962301 primitive calls) in 2.627 seconds

Ordered by: cumulative time
List reduced from 440 to 20 due to restriction <20>

ncalls tottime percall cumtime percall filename:lineno(function)
1 0.001 0.001 2.651 2.651 /mnt/d/Dokumenter/GitHub/ReFinED/my_tests/refined_utils.py:164(run_refined_batch)
1 0.003 0.003 2.649 2.649 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/inference/processor.py:198(process_text_batch)
63 0.042 0.001 2.258 0.036 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/inference/processor.py:274(process_tensors)
15057/63 0.007 0.000 1.255 0.020 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/module.py:1549(_wrapped_call_impl)
15057/63 0.017 0.000 1.255 0.020 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/torch/nn/modules/module.py:1555(_call_impl)
63 0.003 0.000 1.254 0.020 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/model_components/refined_model.py:164(forward)
63 0.034 0.001 0.735 0.012 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/model_components/refined_model.py:382(_identify_entity_mentions)

3026 0.574 0.000 0.693 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/inference/processor.py:378(<listcomp>)
63 0.001 0.000 0.419 0.007 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:696(forward)
63 0.002 0.000 0.391 0.006 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:457(forward)
756 0.002 0.000 0.386 0.001 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:381(forward)
1000 0.001 0.000 0.318 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/data_types/doc_types.py:46(from_text)
1000 0.001 0.000 0.317 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/doc_preprocessing/preprocessor.py:211(tokenize)
1000 0.002 0.000 0.295 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/tokenization_utils_base.py:2359(encode_plus)
1000 0.001 0.000 0.292 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/gpt2/tokenization_gpt2_fast.py:161(_encode_plus)
1000 0.002 0.000 0.292 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/tokenization_utils_fast.py:425(_encode_plus)
1000 0.247 0.000 0.285 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/gpt2/tokenization_gpt2_fast.py:152(_batch_encode_plus)
756 0.001 0.000 0.262 0.000 /mnt/d/Dokumenter/GitHub/ReFinED/.venv/lib/python3.8/site-packages/transformers/models/roberta/modeling_roberta.py:311(forward)
63 0.000 0.000 0.246 0.004 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/doc_preprocessing/preprocessor.py:249(get_classes_idx_for_qcode_batch)
63 0.000 0.000 0.246 0.004 /mnt/d/Dokumenter/GitHub/ReFinED/SRC/refined/doc_preprocessing/class_handler.py:124(get_classes_idx_for_qcode_batch)

[Repeated Runs]
Warmup Run: 1.99s
Run 1: 1.98s
Run 2: 1.69s
Run 3: 1.79s
Run 4: 1.70s
Run 5: 2.06s
Run 6: 1.72s

Run 7: 1.74s
Run 8: 1.75s
Average (excluding warmup): 1.80s

[Accuracy]

Accuracy: 72.00% (720/1000 correct)

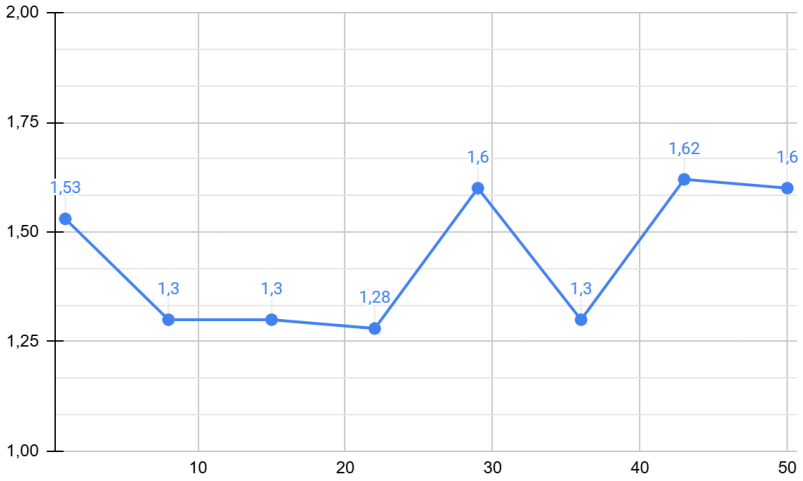
======= END BENCHMARK =======

Companies

GPU: RTX 4070 12GB VRAM
warmup NOT included in avg
accuarcy: 72.0%

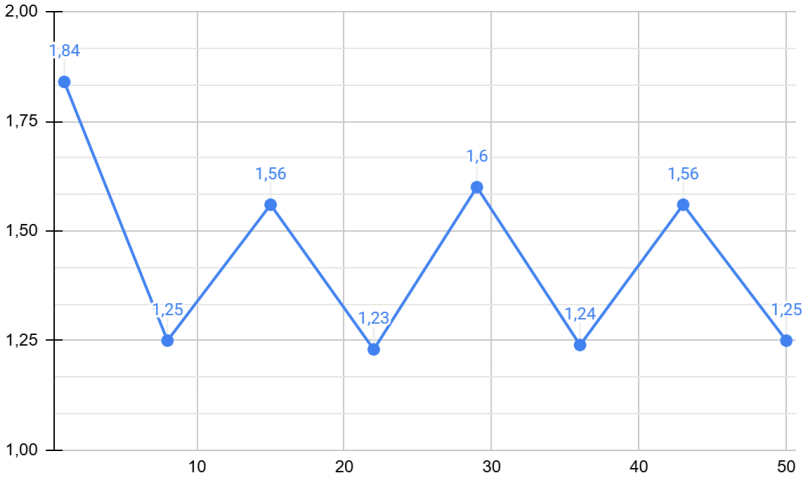
Batch size: **512**
peak memory usage: **65.05 MB**
Average over 50 runs: **1.47s**

Run	Time
1	1,53
8	1,3
15	1,3
22	1,28
29	1,6
36	1,3
43	1,62
50	1,6



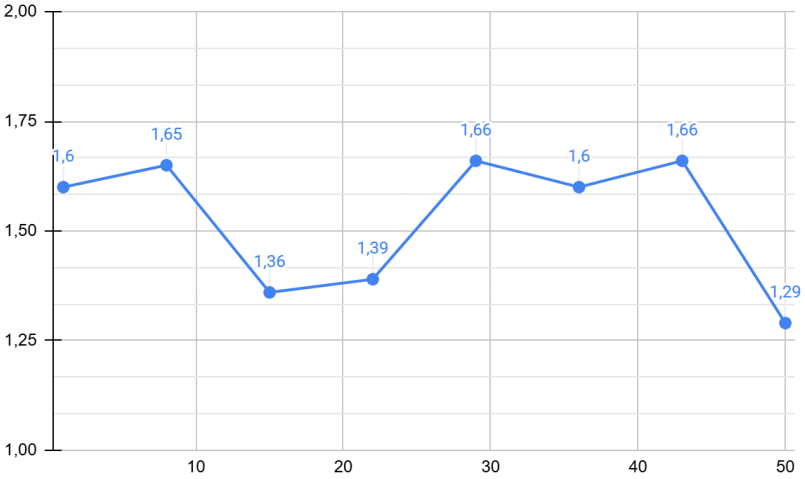
Batch size: **256**
peak memory usage: **56.74 MB**
Average over 50 runs: **1.43s**

Run	Time
1	1,84
8	1,25
15	1,56
22	1,23
29	1,6
36	1,24
43	1,56
50	1,25



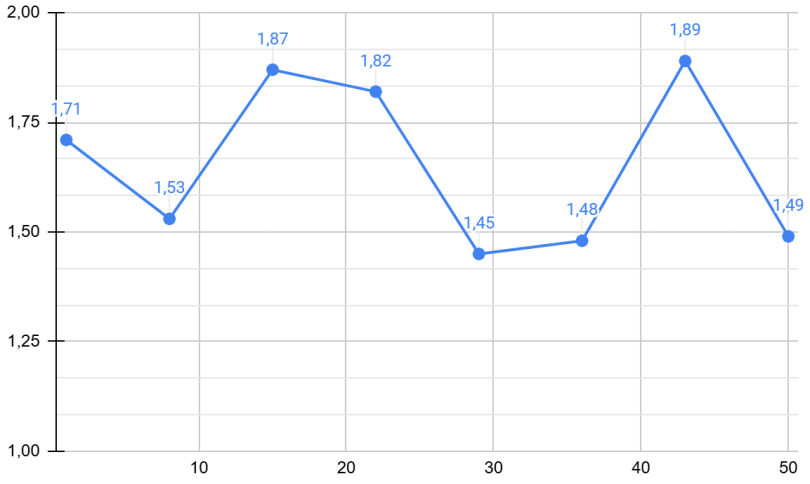
Batch size: **128**
peak memory usage: **53.36 MB**
Average over 50 runs: **1.54s**

Run	Time
1	1,6
8	1,65
15	1,36
22	1,39
29	1,66
36	1,6
43	1,66
50	1,29



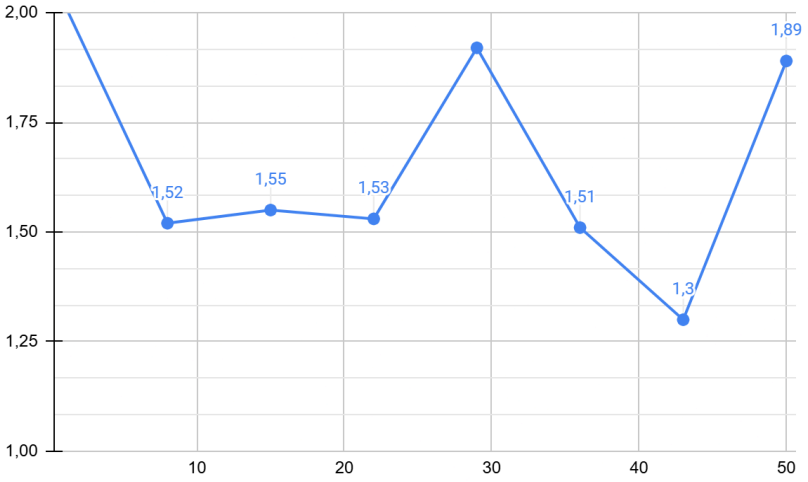
Batch size: **64**
peak memory usage: **51.63 MB**
Average over 50 runs: **1.67s**

Run	Time
1	1,71
8	1,53
15	1,87
22	1,82
29	1,45
36	1,48
43	1,89
50	1,49



Batch size: **32**
peak memory usage: **51.28 MB**
Average over 50 runs: **1.72s**

Run	Time
1	2,02
8	1,52
15	1,55
22	1,53
29	1,92
36	1,51
43	1,3
50	1,89



Conclusion:

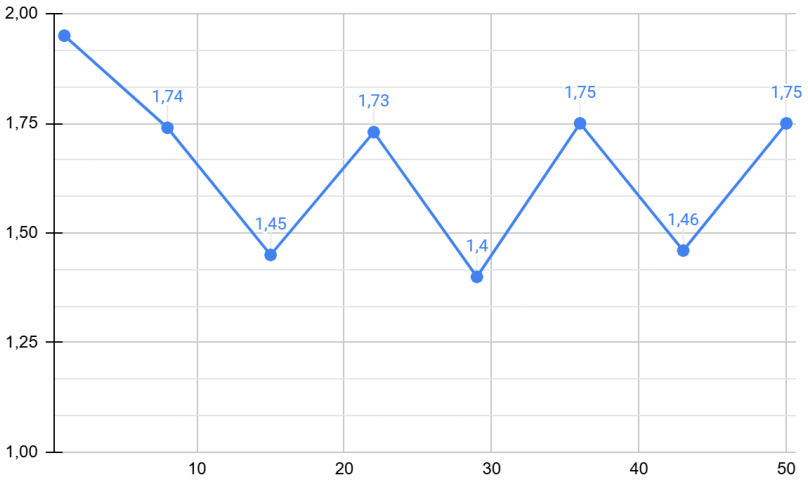
- A steady increase in performance the bigger batch size, whereas the peak of 512 batch size often differed, mostly being slower.
- Fastest: **1.43s avg,** with batch size = **256**
- Slowest: **1.72s avg**, with batch size = **32**

Movies

GPU: RTX 4070 12GB VRAM
warmup NOT included in avg
accuarcy: 70.2%

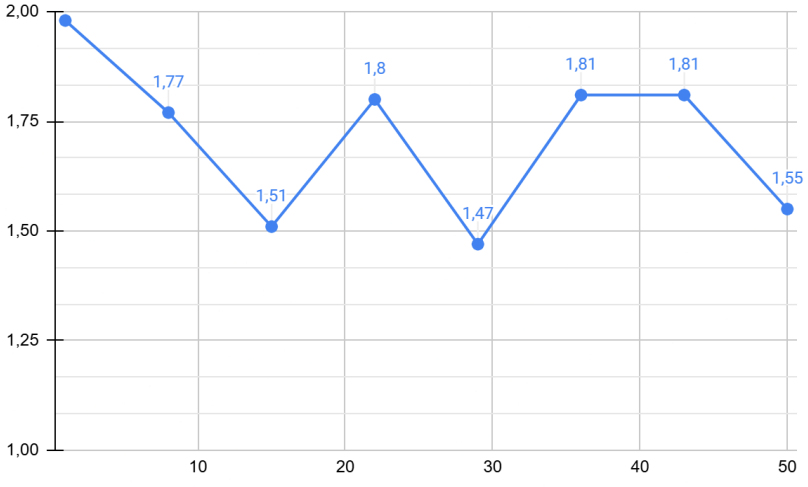
Batch size: **512**
peak memory usage: **82.63 MB**
Average over 50 runs: **1.63s**

Run	Time
1	1,95
8	1,74
15	1,45
22	1,73
29	1,4
36	1,75
43	1,46
50	1,75



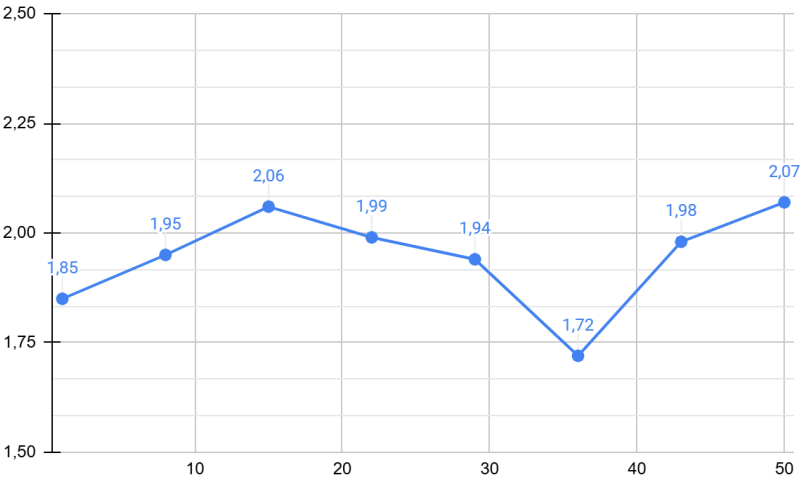
Batch size: **256**
peak memory usage: **71.78 MB**
Average over 50 runs: **1.70s**

Run	Time
1	1,98
8	1,77
15	1,51
22	1,8
29	1,47
36	1,81
43	1,81
50	1,55



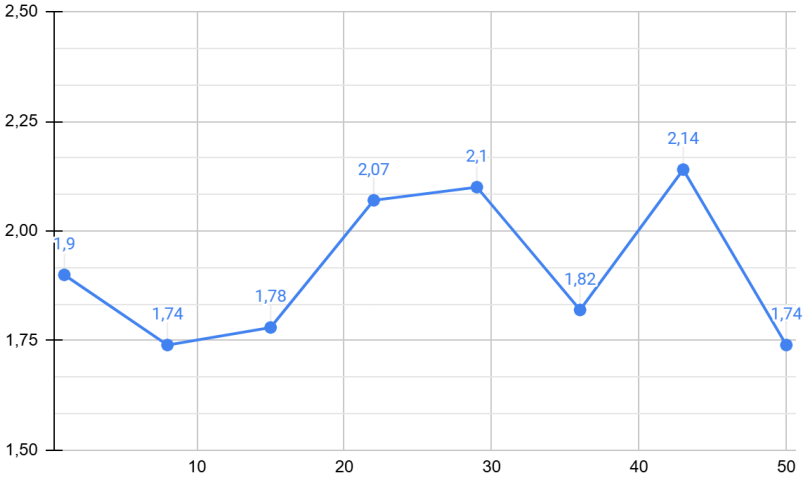
Batch size: **128**
peak memory usage: **66.39 MB**
Average over 50 runs: **1.87s**

Run	Time
1	1,85
8	1,95
15	2,06
22	1,99
29	1,94
36	1,72
43	1,98
50	2,07



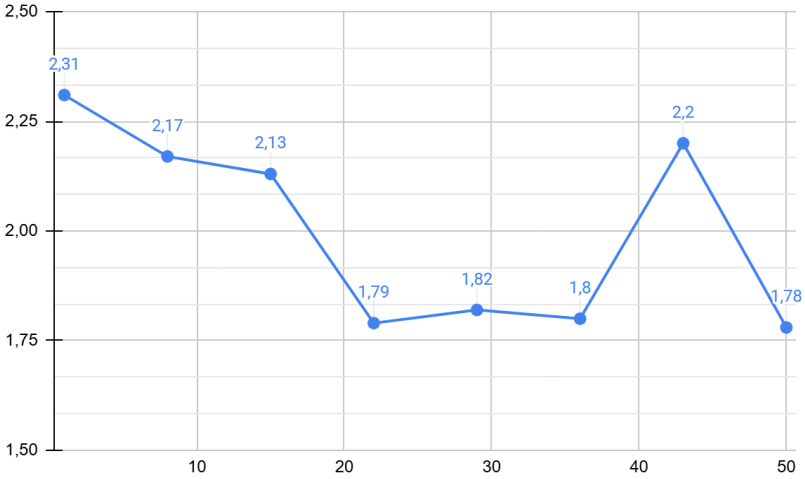
Batch size: **64**
peak memory usage: **63.93 MB**
Average over 50 runs: **1.98s**

Run	Time
1	1,9
8	1,74
15	1,78
22	2,07
29	2,1
36	1,82
43	2,14
50	1,74



Batch size: **32**
peak memory usage: **63.52 MB**
Average over 50 runs: **2.03s**

Run	Time
1	2,31
8	2,17
15	2,13
22	1,79
29	1,82
36	1,8
43	2,2
50	1,78



Conclusion:

- A steady increase in performance the bigger batch size.
- Faster than movies and SN on 512 batch size, although that seems like the peak.
- Fastest: **1.63s avg,** with batch size = **512**
- Slowest: **2.03s avg**, with batch size = **32**

SN

GPU: RTX 4070 12GB VRAM
warmup NOT included in avg
accuarcy: 22.50%

Batch size: **512**
peak memory usage: **95.39 MB**
Average over 50 runs: **2.49s**
1: 2.91
8: 2.53
15: 2.52
22: 2.50
29: 2.51
36: 2.55
43: 2.54
50: 2.51

Batch size: **256**
peak memory usage: **79.79 MB**
Average over 50 runs: **2.26s**
1: 2.76
8: 2.25
15: 2.25
22: 2.28
29: 2.07
36: 2.33
43: 2.23
50: 2.34

Batch size: **128**
peak memory usage: **72.74 MB**
Average over 50 runs: **2.12s**
1: 2.62
8: 2.16
15: 2.23
22: 2.06
29: 2.18
36: 2.16
43: 2.15
50: 2.16

Batch size: **64**
peak memory usage: **69.42 MB**
Average over 50 runs: **2.09s**
1: 2.59
8: 2.14
15: 2.13
22: 2.12
29: 2.10
36: 2.20
43: 2.15
50: 2.12

Batch size: **32**
peak memory usage: **68.74 MB**
Average over 50 runs: **2.14s**
1. 2.23
8: 1.82
15: 2.18
22: 2.24
29: 1.84
36: 2.16
43: 2.25
50: 2.20

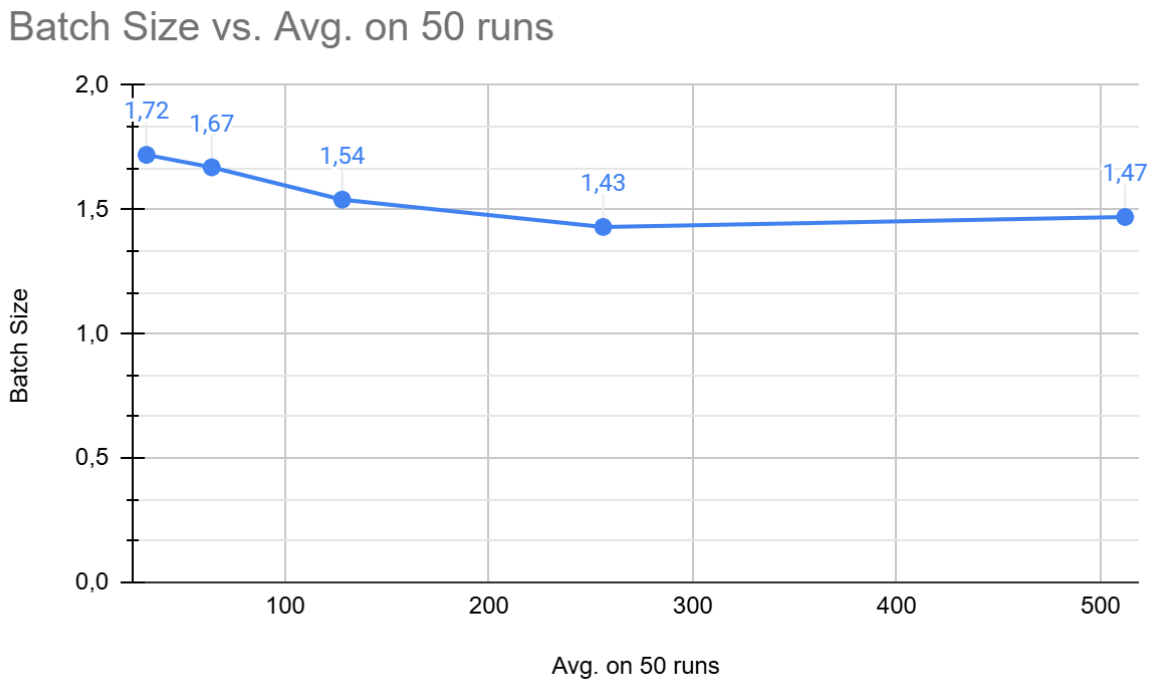
Conclusion:

- Surprisingly, the SN dataset runs faster the smaller the batch size it seems.
- Fastest: **2.09s avg,** with batch size = **64**
- Slowest: **2.49s avg**, with batch size = **512**

Summary

Companies

Batch Size	512	256	128	64	32
Avg. on 50 runs	1.47s	1.43s	1.54s	1.67s	1.72s



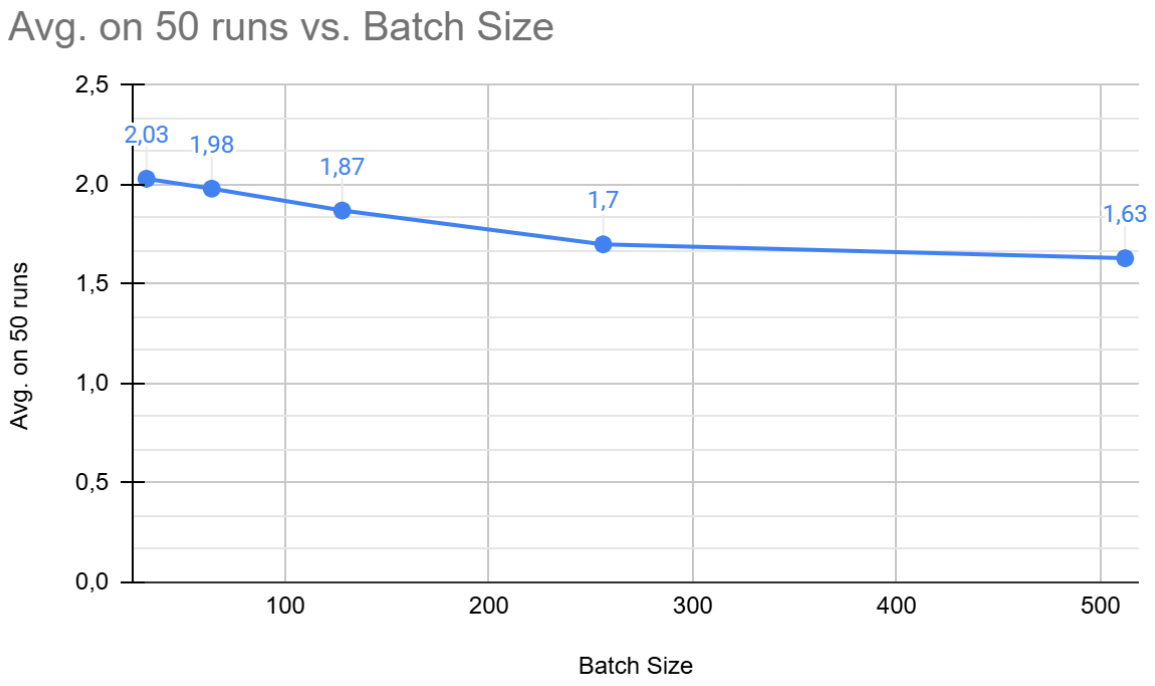
Conclusion:

- A steady increase in performance the bigger batch size, whereas the peak of 512 batch size often differed, mostly being slower.

  Fastest: 1.43s avg, with batch size = 256
  Slowest: 1.72s avg, with batch size = 32

Movies

Batch Size	512	256	128	64	32
Avg. on 50 runs	1.63s	1.70s	1.87s	1.98s	2.03s



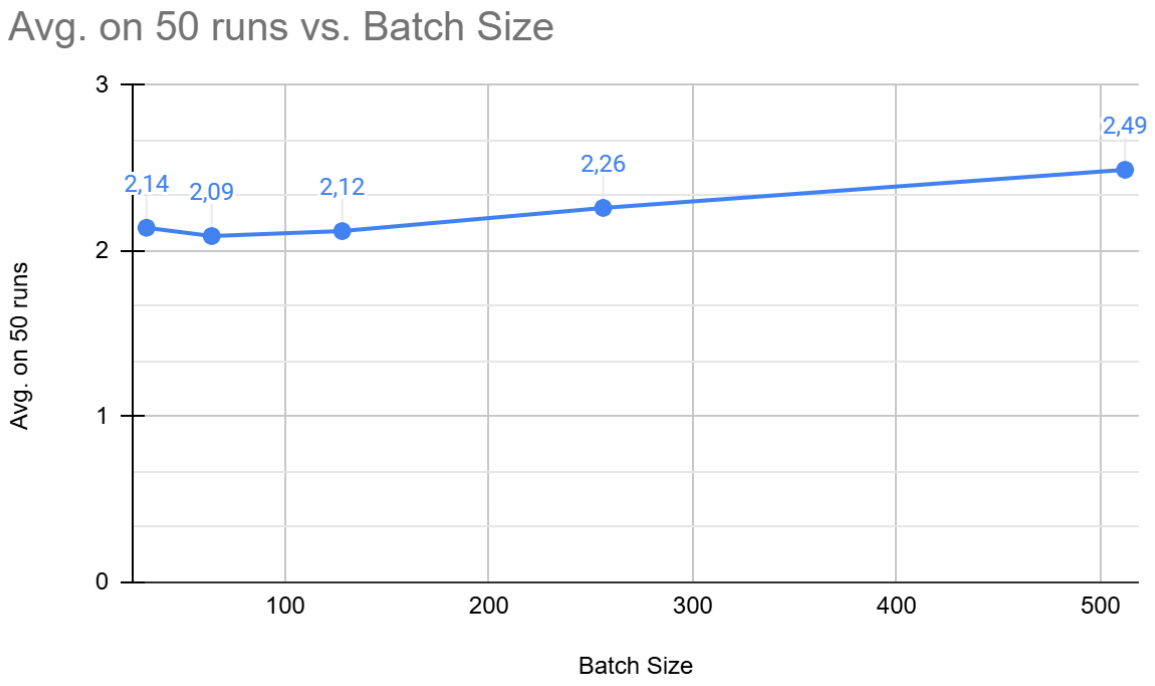
Conclusion:

- A steady increase in performance the bigger batch size.
- Faster than movies and SN on 512 batch size, although that seems like the peak.

Fastest: 1.63s avg, with batch size = 512
Slowest: 2.03s avg, with batch size = 32

SN

Batch Size	512	256	128	64	32
Avg. on 50 runs	2.49s	2.26s	2.12s	2.09s	2.14s



Conclusion:

- Surprisingly, the SN dataset runs faster the smaller the batch size it seems.

  Fastest: 2.09s avg, with batch size = 64
  Slowest: 2.49s avg, with batch size = 512

Fine Tuning

Models:

- 3 models with merged data (10k, 60k, full)
- 2 models with full from each (companies, movies)

Entity set:

- Wikidata

Batch size:

- 128

1. merged_10k

- Consists of merged data from both ‘movies_test.csv’ og ‘companies_test.csv’
- Uses 5000 lines from both files (total 10.000)
- roughly ~15 min training time (Batch 128, 4070ti)
- Results:

Companies:

Custom accuracy results:

```
========-EVAL METRICS-==========
Accuracy:  0.7750   77.50   (775/1000)
Precision: 0.9639
Recall:    0.7750
F1 Score:  0.8592
================================
```

```
f1: 0.4015
accuracy: 0.7730
gold_recall: 0.9220
p: 0.2711
r: 0.7730
num_gold_spans: 1000
```

- decrease from 77.6% to 77.5% **(-0.1%)**
- Refined accuracy: 77.3% **(-0.3%)**

Movies:

Custom accuracy results:

```
========-EVAL METRICS-==========
Accuracy:  0.8860   88.60   (886/1000)
Precision: 0.9683
Recall:    0.8860
F1 Score:  0.9253
================================
```

```
f1: 0.3848
accuracy: 0.8860
gold_recall: 0.9090
p: 0.2458
r: 0.8860
num_gold_spans: 1000
```

- Increase from 69.1% to 88.6% **(+19.5%)**
- Refined accuracy: 88.6% **(+19.5%)**

2. merged_60k

- Consists of merged data from both ‘movies_test.csv’ og ‘companies_test.csv’
- Uses 30.000 lines from both files (total 60.000)
- roughly 1 hour 37 minutes (Batch 128, 4070ti)
- Results:

Companies:

Custom accuracy results:

```
========-EVAL METRICS-==========
Accuracy:  0.7780   77.80   (778/1000)
Precision: 0.9798
Recall:    0.7780
F1 Score:  0.8673
==============================
```

```
f1: 0.4486
accuracy: 0.7810
gold_recall: 0.9220
p: 0.3147
r: 0.7810
num_gold_spans: 1000
```

- Increase from 77.6% to 77.8% **(+0.2%)**
- Refined accuracy: 78.10% **(+0.5%)**

Movies:

Custom accuracy results:

```
========-EVAL METRICS-==========
Accuracy:  0.8920   89.20   (892/1000)
Precision: 0.9791
Recall:    0.8920
F1 Score:  0.9335
==============================
```

```
f1: 0.4592
accuracy: 0.8920
gold_recall: 0.9090
p: 0.3092
r: 0.8920
num_gold_spans: 1000
```

- Increase from 69.1% to 89.2% **(+20.1%)**
- Refined accuracy: 89.2% **(+20.1%)**

3. companies_full

- Consists of full data from only ‘companies_test.csv’
- Uses 67.500 lines
- roughly 1 hour 34 minutes (Batch 128, 4070ti)
- Results:
-

Companies:

Custom accuracy results:

```
========-EVAL METRICS-==========
Accuracy:  0.7900   79.00   (790/1000)
Precision: 0.9887
Recall:    0.7900
F1 Score:  0.8783
==============================
```

```
f1: 0.4510
accuracy: 0.7890
gold_recall: 0.9220
p: 0.3157
r: 0.7890
num_gold_spans: 1000
```

- Increase from 77.6% to 79% **(+1.4%)**
- Refined accuracy: 78.9% **(+1.3%)**

Movies:

Custom accuracy results:

```
========-EVAL METRICS-==========
Accuracy:  0.3830   38.30   (383/1000)
Precision: 0.8201
Recall:    0.3830
F1 Score:  0.5222
==============================
```

```
f1: 0.1624
accuracy: 0.3940
gold_recall: 0.9090
p: 0.1023
r: 0.3940
num_gold_spans: 1000
```

- Decrease from 69.1% to 38.3% **(-30.8%)**
- Refined accuracy: 39.4% **(-29.7%)**

4. movies_full

- Consists of full data from only ‘movies_test.csv’
- Uses 67.500 lines
- roughly 1 hour 21 minutes (Batch 128, 4070ti)
- Results:

Companies:

Custom accuracy results:

```
========-EVAL METRICS-==========
Accuracy:  0.0790   7.90   (79/1000)
Precision: 0.8876
Recall:    0.0790
F1 Score:  0.1451
================================
```

```
f1: 0.0866
accuracy: 0.0970
gold_recall: 0.9220
p: 0.0783
r: 0.0970
num_gold_spans: 1000
```

- decrease from 77.6% to 7.9% **(-69.7%)**
- Refined accuracy: 9.7% **(-67,9%)**

Movies:

Custom accuracy results:

```
========-EVAL METRICS-==========
Accuracy:  0.8930   89.30   (893/1000)
Precision: 0.9900
Recall:    0.8930
F1 Score:  0.9390
================================
```

```
f1: 0.4815
accuracy: 0.8920
gold_recall: 0.9090
p: 0.3298
r: 0.8920
num_gold_spans: 1000
```

- Increase from 69.1% to 89.3% **(+20,2%)**
- Refined accuracy: 89.2% **(+20.1%)**

5. merged_full

- Consists of full data from both ‘movies_test.csv’ and ‘companies_test.csv’
- Uses (67.500 x 2) = 135.000 lines
- roughly 3 hour 13 min (Batch 128, 4070ti)
- Results:

Companies:

Custom accuracy results:

```
========-EVAL METRICS-=========
Accuracy:  0.7860   78.60   (786/1000)
Precision: 0.9924
Recall:    0.7860
F1 Score:  0.8772
==============================
```

```
f1: 0.4488
accuracy: 0.7840
gold_recall: 0.9220
p: 0.3144
r: 0.7840
num_gold_spans: 1000
```

- Increase from 77.6% to 78.6% **(+1.0%)**
- Refined accuracy: 78.4% **(+0.8%)**

Movies:

Custom accuracy results:

```
========-EVAL METRICS-=========
Accuracy:  0.8920   89.20   (892/1000)
Precision: 0.9966
Recall:    0.8920
F1 Score:  0.9414
==============================
```

```
f1: 0.4195
accuracy: 0.8920
gold_recall: 0.9090
p: 0.2742
r: 0.8920
num_gold_spans: 1000
```

- Increase from 69.1% to 89.2% **(+20.1%)**
- Refined accuracy: 89.2% **(+20.1%)**

Summary

Custom accuracy vs ReFinED metrics:

- ***custom accuracy*** = measures row-level correctness
    - only checks whether first entity each row is predicted correctly
    - Uses whole row for context for prediction, but ignores other entities in row
    - Makes it stricter, coarser metric

- ReFinED metrics = measures entity-level correctness
    - every predicted entity in the row is compared to its gold entity, and metrics like precision, recall, F1 count all entities, not just first.
    - Gives more fine-grained and complete assessment of models overall performance
    - (not used in this assessment)

Merged datasets improve cross-domain performance:

- Small merged datasets (10k) already provide substantial gains, particularly for movies (+19.5% recall)
- Larger datasets (60k and full 135k) give incremental gains while keeping balanced performance.

Domain-specific full datasets excel in own domain, but hurt other:

- 'companies_full' gives a 1.4% increase in companies, but movies drop (-30.8%)
- 'movies_full' gives a 20.2% increase in movies, but companies drop (-69.7%)

Merged full dataset (135k) is the best overall model:

- Custom accuracy for main entity each row shows a decent score with 78-89%
- compared to baseline, it provides a +1.0% gain for companies and +20.1% for movies
- ReFinED evaluation confirms near-perfect precision (**0.997** for movies and **0.992** for companies), making it the most reliable model.

Training considerations:

- Beyond ~60k lines, returns diminish significantly
- Merged dataset recommended for balanced, ***multi-domain performance***
- Single-domain datasets are best for ***domain-specific applications***

Evaluation

Evaluering for andre ting (SemTab ++ ?)

(Fra thesis beskrivelse)

The focus will be on assessing its performance using datasets from the SemTab challenge, specifically those used to validate the Alligator approach (https://ieeexplore.ieee.org/abstract/document/10350082). These datasets include Round_T2D, Round3, Round4, HardTableR2, and HardTableR3.

Grunndata

Disse testene er de “originale” start-testene, dvs

- companies
- movies
- EL

De første datasettene som ble brukt til testing, hovedgrunnlaget

Accuracy (%)

Following tabs use different models and entity sets:

model_name:

1. **'wikipedia_model':** (wikipedia)
- This is the model which matches the setup described in the paper

2. **'wikipedia_model_with_numbers':** (numbers)
- This model extends the above model, to also include detection of SpaCy numerical data types in the mention detection layer ("DATE", "CARDINAL", "MONEY", "PERCENT", "TIME", "ORDINAL", "QUANTITY"). The detected types are available at span.coarse_type. If the coarse_type is detected as "DATE", the date will be normalised to a standard format available at span.date. All non-numerical types will have a coarse_type of "MENTION", and will be passed through the entity disambiguation layer to attempt to resolve them to a wikidata entity.

entity_set:

1. **"wikidata"**:
- resolves against all ~33M (after some filtering) entities in wikidata (requires more memory)

2. "**wikipedia**":
- limits to resolving against the ~6M entities which have a wikipedia page.

measure accuracy

- "Simple" measurer specific for our goal
- - Looks only at first predicted entity in row and compares to gold value
    > input: "a, b, c, d"
    > gold: {1}
    > preds: [1, 2, 3, 4]
- ignores extra predictions (2, 3, 4)
- counts prediction correct if (a) == (1) , first matches truth
- High precision due to not penalizing extra predictions (2, 3, 4), only main prediction

Refined evaluation

- packs input and gold into "Entity" and "Docs" objects - used internally
- looks at ALL predicted spans
- compares each predicted span to all gold spans in doc
- extra preds with no gold are counted as FALSE POSITIVES (FP)
- precision heavily influenced by extra penalties

- True positive (TP) - pred. entity matches gold entity exactly
- False positive (FP) - pred. entity exist but no gold entity (“pred true, but gold is not”)
- False negative (FN) - gold entity exists but no pred. entity (predicted = None)
- True negative (TN) - meaningless

numbers/wikipedia

model_name **= ‘wikipedia_model_with_numbers’**
entity_set **= ‘wikipedia’**

Companies

Best = 50.7% / 44.2%

Measure_accuracy:

- Accuracy = 0.5070 = (50.70)
- Precision = 0.8353
- Recall = 0.5070
- F1 = 0.6310

evaluate:

- Accuracy = 0.4420 = (44.2)
- Precision = 0.1900
- Recall = 0.4420
- F1 = 0.2658

```
[985]   Pred: Q289          Truth: ['Q7314030']     Match: False(Q289)
[986]   Pred: Q1669453      Truth: ['Q1669453']     Match: True(Q1669453)
[987]   Pred: None          Truth: ['Q7670348']     Match: False(None)
[988]   Pred: Q498410       Truth: ['Q498410']      Match: True(Q498410)
[989]   Pred: None          Truth: ['Q92437201']    Match: False(None)
[990]   Pred: Q9779         Truth: ['Q4721851']     Match: False(Q9779)
[991]   Pred: Q28130206     Truth: ['Q28130206']    Match: True(Q28130206)
[992]   Pred: Q7392916      Truth: ['Q7392916']     Match: True(Q7392916)
[993]   Pred: Q6020637      Truth: ['Q6020637']     Match: True(Q6020637)
[994]   Pred: Q7383544      Truth: ['Q7383544']     Match: True(Q7383544)
[995]   Pred: Q5874938      Truth: ['Q5874938']     Match: True(Q5874938)
[996]   Pred: Q921363       Truth: ['Q921363']      Match: True(Q921363)
[997]   Pred: Q2503528      Truth: ['Q2503528']     Match: True(Q2503528)
[998]   Pred: Q144105       Truth: ['Q144105']      Match: True(Q144105)
[999]   Pred: Q7271689      Truth: ['Q7271689']     Match: True(Q7271689)

========-EVAL METRICS-=========
Accuracy:  0.5070   50.70   (507/1000)
Precision: 0.8353
Recall:    0.5070
F1 Score:  0.6310
==============================
```

```
Dataset name: EVAL

****************
************
f1: 0.2658
accuracy: 0.4420
gold_recall: 0.5660
p: 0.1900
r: 0.4420
num_gold_spans: 1000
************
*******MD*****
MD_f1: 0.3173, (p: 0.2038, r: 0.7160)
*****************

******************************
```

Movies

Best = 61.5% / 58.7%

Measure_accuracy:

- Accuracy = 0.6150 = (61.50)
- Precision = 0.8146
- Recall = 0.6150
- F1 = 0.7009

evaluate:

- Accuracy = 0.5870 = (58.70)
- Precision = 0.1687
- Recall = 0.5870
- F1 = 0.2621

```
[985]   Pred: Q3488219      Truth: ['Q3488219']     Match: True     (Q3488219)
[986]   Pred: Q2190650      Truth: ['Q2190650']     Match: True     (Q2190650)
[987]   Pred: Q3212936      Truth: ['Q3212936']     Match: True     (Q3212936)
[988]   Pred: Q972771       Truth: ['Q972771']      Match: True     (Q972771)
[989]   Pred: Q1044944      Truth: ['Q1044944']     Match: True     (Q1044944)
[990]   Pred: Q1089809      Truth: ['Q1089809']     Match: True     (Q1089809)
[991]   Pred: None          Truth: ['Q2713426']     Match: False    (None)
[992]   Pred: None          Truth: ['Q3705949']     Match: False    (None)
[993]   Pred: Q542465       Truth: ['Q542465']      Match: True     (Q542465)
[994]   Pred: Q178417       Truth: ['Q3941726']     Match: False    (Q178417)
[995]   Pred: None          Truth: ['Q3212896']     Match: False    (None)
[996]   Pred: Q3066641      Truth: ['Q3066641']     Match: True     (Q3066641)
[997]   Pred: Q1057433      Truth: ['Q1057433']     Match: True     (Q1057433)
[998]   Pred: Q3602584      Truth: ['Q3602584']     Match: True     (Q3602584)
[999]   Pred: Q1162433      Truth: ['Q1162433']     Match: True     (Q1162433)

========-EVAL METRICS-==========
Accuracy:  0.6150   61.50   (615/1000)
Precision: 0.8146
Recall:    0.6150
F1 Score:  0.7009
==============================
```

```
Dataset name: EVAL

****************
************
f1: 0.2621
accuracy: 0.5870
gold_recall: 0.7670
p: 0.1687
r: 0.5870
num_gold_spans: 1000
************
*******MD*****
MD_f1: 0.2814, (p: 0.1691, r: 0.8390)
*****************


******************************
```

SN

Best = 17,0%

Measure_accuracy:

- Accuracy = 0.1700 = (17.00)
- Precision = 0.6513
- Recall = 0.1700
- F1 = 0.2696

evaluate:

- (not implemented)

```
[985]   Pred: None          Truth: ['NIL']          Match: False   (None)
[986]   Pred: None          Truth: ['Q2927109']     Match: False   (None)
[987]   Pred: None          Truth: ['Q118791749']   Match: False   (None)
[988]   Pred: None          Truth: ['Q12758']       Match: False   (None)
[989]   Pred: Q160556       Truth: ['Q933681']      Match: False   (Q160556)
[990]   Pred: None          Truth: ['NIL']          Match: False   (None)
[991]   Pred: Q7209967      Truth: ['Q7209967']     Match: True    (Q7209967)
[992]   Pred: None          Truth: ['NIL']          Match: False   (None)
[993]   Pred: Q918239       Truth: ['Q918239']      Match: True    (Q918239)
[994]   Pred: None          Truth: ['Q16988947']    Match: False   (None)
[995]   Pred: Q13576286     Truth: ['Q66232114']    Match: False   (Q13576286)
[996]   Pred: None          Truth: ['Q38609561']    Match: False   (None)
[997]   Pred: Q24055697     Truth: ['Q112160997']   Match: False   (Q24055697)
[998]   Pred: Q204234       Truth: ['Q16997843']    Match: False   (Q204234)
[999]   Pred: Q84           Truth: ['Q5657624']     Match: False   (Q84)

========-EVAL METRICS-=========
Accuracy:  0.1700   17.00   (170/1000)
Precision: 0.6513
Recall:    0.1700
F1 Score:  0.2696
==============================
```

wikipedia/wikipedia

model_name **= ‘wikipedia_model’**
entity_set **= ‘wikipedia’**

Companies

Best = 48.8% / 42.5%

Measure_accuracy:

- Accuracy = 0.4880 = (48.80)
- Precision = 0.7935
- Recall = 0.4880
- F1 = 0.6043

evaluate:

- Accuracy = 0.4250 = (42.50)
- Precision = 0.1898
- Recall = 0.4250
- F1 = 0.2624

```
[985]   Pred: Q159260       Truth: ['Q7314030']     Match: False   (Q159260)
[986]   Pred: Q1669453      Truth: ['Q1669453']     Match: True    (Q1669453)
[987]   Pred: None          Truth: ['Q7670348']     Match: False   (None)
[988]   Pred: Q498410       Truth: ['Q498410']      Match: True    (Q498410)
[989]   Pred: Q132311       Truth: ['Q92437201']    Match: False   (Q132311)
[990]   Pred: Q9779         Truth: ['Q4721851']     Match: False   (Q9779)
[991]   Pred: Q28130206     Truth: ['Q28130206']    Match: True    (Q28130206)
[992]   Pred: Q7392916      Truth: ['Q7392916']     Match: True    (Q7392916)
[993]   Pred: Q6020637      Truth: ['Q6020637']     Match: True    (Q6020637)
[994]   Pred: Q7383544      Truth: ['Q7383544']     Match: True    (Q7383544)
[995]   Pred: Q5874938      Truth: ['Q5874938']     Match: True    (Q5874938)
[996]   Pred: Q921363       Truth: ['Q921363']      Match: True    (Q921363)
[997]   Pred: Q2503528      Truth: ['Q2503528']     Match: True    (Q2503528)
[998]   Pred: Q144105       Truth: ['Q144105']      Match: True    (Q144105)
[999]   Pred: Q7271689      Truth: ['Q7271689']     Match: True    (Q7271689)

========-EVAL METRICS-=========
Accuracy:  0.4880   48.80   (488/1000)
Precision: 0.7935
Recall:    0.4880
F1 Score:  0.6043
==============================
```

```
Dataset name: EVAL

****************
************
f1: 0.2624
accuracy: 0.4250
gold_recall: 0.5660
p: 0.1898
r: 0.4250
num_gold_spans: 1000
************
*******MD*****
MD_f1: 0.3688, (p: 0.2509, r: 0.6960)
*****************


******************************
```

Movies

Best = 53.9% / 56.2%

Measure_accuracy:

- Accuracy = 0.5390 = (53.90)
- Precision = 0.7926
- Recall = 0.5390
- F1 = 0.6417

evaluate:

- Accuracy = 0.5620 = (56.20)
- Precision = 0.1697
- Recall = 0.5620
- F1 = 0.2607

```
[985]   Pred: Q3488219      Truth: ['Q3488219']     Match: True    (Q3488219)
[986]   Pred: Q2190650      Truth: ['Q2190650']     Match: True    (Q2190650)
[987]   Pred: Q3212936      Truth: ['Q3212936']     Match: True    (Q3212936)
[988]   Pred: Q972771       Truth: ['Q972771']      Match: True    (Q972771)
[989]   Pred: Q1044944      Truth: ['Q1044944']     Match: True    (Q1044944)
[990]   Pred: Q1089809      Truth: ['Q1089809']     Match: True    (Q1089809)
[991]   Pred: None          Truth: ['Q2713426']     Match: False   (None)
[992]   Pred: None          Truth: ['Q3705949']     Match: False   (None)
[993]   Pred: Q542465       Truth: ['Q542465']      Match: True    (Q542465)
[994]   Pred: Q178417       Truth: ['Q3941726']     Match: False   (Q178417)
[995]   Pred: None          Truth: ['Q3212896']     Match: False   (None)
[996]   Pred: Q3066641      Truth: ['Q3066641']     Match: True    (Q3066641)
[997]   Pred: None          Truth: ['Q1057433']     Match: False   (None)
[998]   Pred: Q3602584      Truth: ['Q3602584']     Match: True    (Q3602584)
[999]   Pred: Q1162433      Truth: ['Q1162433']     Match: True    (Q1162433)

========-EVAL METRICS-==========
Accuracy:  0.5390   53.90   (539/1000)
Precision: 0.7926
Recall:    0.5390
F1 Score:  0.6417
==============================
```

```
Dataset name: EVAL

****************
************
f1: 0.2607
accuracy: 0.5620
gold_recall: 0.7670
p: 0.1697
r: 0.5620
num_gold_spans: 1000
************
*******MD*****
MD_f1: 0.3314, (p: 0.2064, r: 0.8410)
*****************


******************************
```

SN

Best = 16.4%

Measure_accuracy:

- Accuracy = 0.1640 = (16.40)
- Precision = 0.5857
- Recall = 0.1640
- F1 = 0.2562

evaluate:

- (not implemented)

```
[985]   Pred: None          Truth: ['NIL']          Match: False   (None)
[986]   Pred: None          Truth: ['Q2927109']     Match: False   (None)
[987]   Pred: None          Truth: ['Q118791749']   Match: False   (None)
[988]   Pred: None          Truth: ['Q12758']       Match: False   (None)
[989]   Pred: None          Truth: ['Q933681']      Match: False   (None)
[990]   Pred: None          Truth: ['NIL']          Match: False   (None)
[991]   Pred: Q7209967      Truth: ['Q7209967']     Match: True    (Q7209967)
[992]   Pred: None          Truth: ['NIL']          Match: False   (None)
[993]   Pred: Q918239       Truth: ['Q918239']      Match: True    (Q918239)
[994]   Pred: None          Truth: ['Q16988947']    Match: False   (None)
[995]   Pred: Q13576286     Truth: ['Q66232114']    Match: False   (Q13576286)
[996]   Pred: None          Truth: ['Q38609561']    Match: False   (None)
[997]   Pred: Q24055697     Truth: ['Q112160997']   Match: False   (Q24055697)
[998]   Pred: Q16997843     Truth: ['Q16997843']    Match: True    (Q16997843)
[999]   Pred: Q213560       Truth: ['Q5657624']     Match: False   (Q213560)

========-EVAL METRICS-=========
Accuracy:  0.1640   16.40   (164/1000)
Precision: 0.5857
Recall:    0.1640
F1 Score:  0.2562
==============================
```

wikipedia/wikidata

model_name **= ‘wikipedia_model’**
entity_set **= ‘wikidata’**

Companies

Best = 69.3% / 60.4%

Measure_accuracy:

- Accuracy = 0.6930 = (69.30)
- Precision = 0.8662
- Recall = 0.6930
- F1 = 0.7700

evaluate:

- Accuracy = 0.6040 = (60.40)
- Precision = 0.2502
- Recall = 0.6040
- F1 = 0.3538

```
[985]   Pred: Q159260       Truth: ['Q7314030']     Match: False    (Q159260)
[986]   Pred: Q1669453      Truth: ['Q1669453']     Match: True     (Q1669453)
[987]   Pred: None          Truth: ['Q7670348']     Match: False    (None)
[988]   Pred: Q498410       Truth: ['Q498410']      Match: True     (Q498410)
[989]   Pred: Q132311       Truth: ['Q92437201']    Match: False    (Q132311)
[990]   Pred: Q9779         Truth: ['Q4721851']     Match: False    (Q9779)
[991]   Pred: Q28130206     Truth: ['Q28130206']    Match: True     (Q28130206)
[992]   Pred: Q7392916      Truth: ['Q7392916']     Match: True     (Q7392916)
[993]   Pred: Q6020637      Truth: ['Q6020637']     Match: True     (Q6020637)
[994]   Pred: Q7383544      Truth: ['Q7383544']     Match: True     (Q7383544)
[995]   Pred: Q5874938      Truth: ['Q5874938']     Match: True     (Q5874938)
[996]   Pred: Q921363       Truth: ['Q921363']      Match: True     (Q921363)
[997]   Pred: Q2503528      Truth: ['Q2503528']     Match: True     (Q2503528)
[998]   Pred: Q144105       Truth: ['Q144105']      Match: True     (Q144105)
[999]   Pred: Q7271689      Truth: ['Q7271689']     Match: True     (Q7271689)

========-EVAL METRICS-=========
Accuracy:  0.6930   69.30   (693/1000)
Precision: 0.8662
Recall:    0.6930
F1 Score:  0.7700
==============================
```

```
Dataset name: EVAL

*****************
*************
f1: 0.3538
accuracy: 0.6040
gold_recall: 0.9220
p: 0.2502
r: 0.6040
num_gold_spans: 1000
*************
*******MD*****
MD_f1: 0.3688, (p: 0.2509, r: 0.6960)
******************

******************************
```

Movies

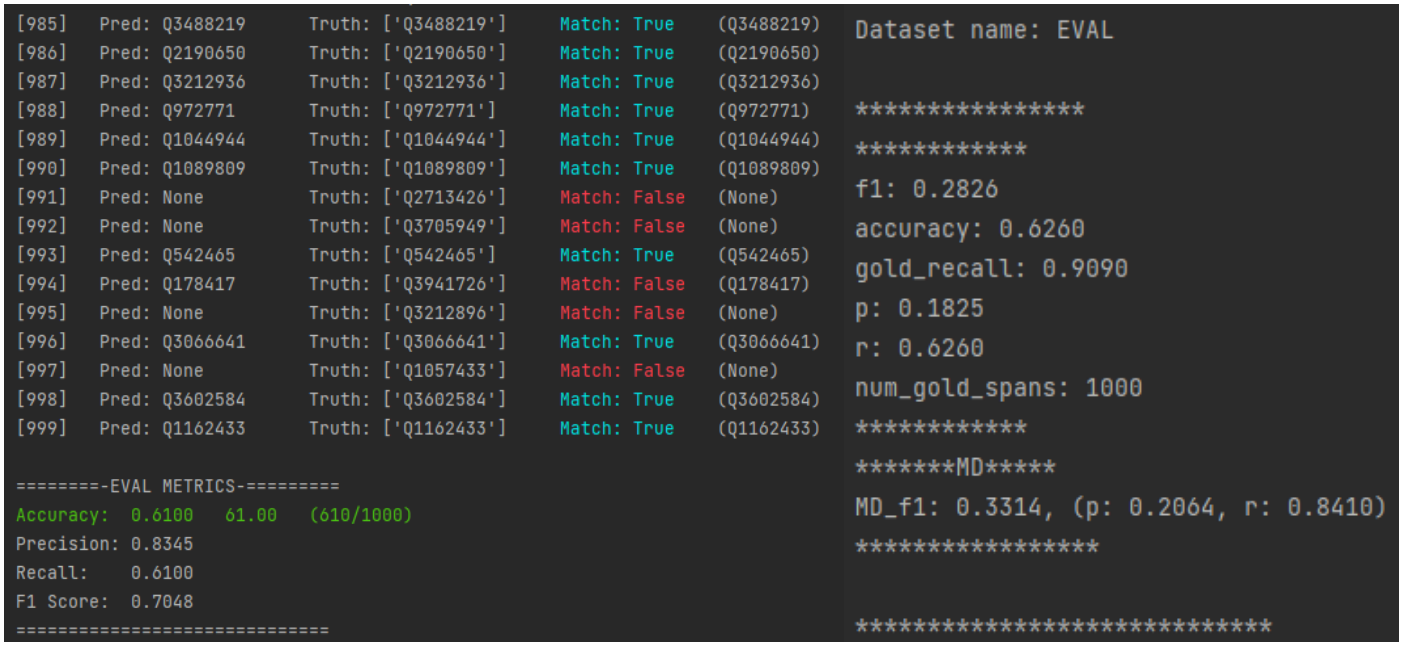
CSV = 61.0% / 62.6%

Measure_accuracy:

- Accuracy = 0.6100 = (61.00)
- Precision = 0.8345
- Recall = 0.6100
- F1 = 0.7048

evaluate:

- Accuracy = 0.6260 = (62.60)
- Precision = 0.1825
- Recall = 0.6260
- F1 = 0.2826



SN

Best = 21.5% /

Measure_accuracy:

- Accuracy = 0.2150 = (21.50)
- Precision = 0.6287
- Recall = 0.2150
- F1 = 0.3204

evaluate:

- (not implemented)

```
[985]   Pred: None          Truth: ['NIL']          Match: False   (None)
[986]   Pred: None          Truth: ['Q2927109']     Match: False   (None)
[987]   Pred: None          Truth: ['Q118791749']   Match: False   (None)
[988]   Pred: None          Truth: ['Q12758']       Match: False   (None)
[989]   Pred: None          Truth: ['Q933681']      Match: False   (None)
[990]   Pred: None          Truth: ['NIL']          Match: False   (None)
[991]   Pred: Q7209967      Truth: ['Q7209967']     Match: True    (Q7209967)
[992]   Pred: None          Truth: ['NIL']          Match: False   (None)
[993]   Pred: Q918239       Truth: ['Q918239']      Match: True    (Q918239)
[994]   Pred: None          Truth: ['Q16988947']    Match: False   (None)
[995]   Pred: Q13576286     Truth: ['Q66232114']    Match: False   (Q13576286)
[996]   Pred: None          Truth: ['Q38609561']    Match: False   (None)
[997]   Pred: Q24055697     Truth: ['Q112160997']   Match: False   (Q24055697)
[998]   Pred: Q16997843     Truth: ['Q16997843']    Match: True    (Q16997843)
[999]   Pred: Q213560       Truth: ['Q5657624']     Match: False   (Q213560)

========-EVAL METRICS-=========
Accuracy:  0.2150   21.50   (215/1000)
Precision: 0.6287
Recall:    0.2150
F1 Score:  0.3204
==============================
```

(BEST) numbers/wikidata

model_name **= ‘wikipedia_model_with_numbers’**
entity_set **= ‘wikidata’**

Companies

Best = 72.0% / 62.6%

Measure_accuracy:

- Accuracy = 0.7200 = (72.00)
- Precision = 0.8966
- Recall = 0.7200
- F1 = 0.7987

evaluate:

- Accuracy = 0.6260 = (62.60)
- Precision = 0.2474
- Recall = 0.6260
- F1 = 0.3547

```
[985]   Pred: Q289          Truth: ['Q7314030']     Match: False   (Q289)
[986]   Pred: Q1669453      Truth: ['Q1669453']     Match: True    (Q1669453)
[987]   Pred: None          Truth: ['Q7670348']     Match: False   (None)
[988]   Pred: Q498410       Truth: ['Q498410']      Match: True    (Q498410)
[989]   Pred: Q92437201     Truth: ['Q92437201']    Match: True    (Q92437201)
[990]   Pred: Q9779         Truth: ['Q4721851']     Match: False   (Q9779)
[991]   Pred: Q28130206     Truth: ['Q28130206']    Match: True    (Q28130206)
[992]   Pred: Q7392916      Truth: ['Q7392916']     Match: True    (Q7392916)
[993]   Pred: Q6020637      Truth: ['Q6020637']     Match: True    (Q6020637)
[994]   Pred: Q7383544      Truth: ['Q7383544']     Match: True    (Q7383544)
[995]   Pred: Q5874938      Truth: ['Q5874938']     Match: True    (Q5874938)
[996]   Pred: Q921363       Truth: ['Q921363']      Match: True    (Q921363)
[997]   Pred: Q2503528      Truth: ['Q2503528']     Match: True    (Q2503528)
[998]   Pred: Q144105       Truth: ['Q144105']      Match: True    (Q144105)
[999]   Pred: Q7271689      Truth: ['Q7271689']     Match: True    (Q7271689)

========-EVAL METRICS-=========
Accuracy:  0.7200   72.00   (720/1000)
Precision: 0.8966
Recall:    0.7200
F1 Score:  0.7987
==============================
```

```
Dataset name: EVAL

****************
************
f1: 0.3547
accuracy: 0.6260
gold_recall: 0.9220
p: 0.2474
r: 0.6260
num_gold_spans: 1000
************
*******MD*****
MD_f1: 0.3173, (p: 0.2038, r: 0.7160)
*****************


******************************
```

Movies

Best = 70.2% / 67.2%

Measure_accuracy:

- Accuracy = 0.7020 = (70.20)
- Precision = 0.8571
- Recall = 0.7020
- F1 = 0.7719

evaluate:

- Accuracy = 0.6720 = (67.20)
- Precision = 0.1933
- Recall = 0.6720
- F1 = 0.3003

```
[985]   Pred: Q3488219      Truth: ['Q3488219']     Match: True    (Q3488219)
[986]   Pred: Q2190650      Truth: ['Q2190650']     Match: True    (Q2190650)
[987]   Pred: Q3212936      Truth: ['Q3212936']     Match: True    (Q3212936)
[988]   Pred: Q972771       Truth: ['Q972771']      Match: True    (Q972771)
[989]   Pred: Q1044944      Truth: ['Q1044944']     Match: True    (Q1044944)
[990]   Pred: Q1089809      Truth: ['Q1089809']     Match: True    (Q1089809)
[991]   Pred: None          Truth: ['Q2713426']     Match: False   (None)
[992]   Pred: None          Truth: ['Q3705949']     Match: False   (None)
[993]   Pred: Q542465       Truth: ['Q542465']      Match: True    (Q542465)
[994]   Pred: Q3941726      Truth: ['Q3941726']     Match: True    (Q3941726)
[995]   Pred: None          Truth: ['Q3212896']     Match: False   (None)
[996]   Pred: Q3066641      Truth: ['Q3066641']     Match: True    (Q3066641)
[997]   Pred: Q1057433      Truth: ['Q1057433']     Match: True    (Q1057433)
[998]   Pred: Q17053893     Truth: ['Q3602584']     Match: False   (Q17053893)
[999]   Pred: Q1162433      Truth: ['Q1162433']     Match: True    (Q1162433)

========-EVAL METRICS-==========
Accuracy:  0.7020   70.20   (702/1000)
Precision: 0.8571
Recall:    0.7020
F1 Score:  0.7719
==============================
```

```
Dataset name: EVAL

****************
************
f1: 0.3003
accuracy: 0.6720
gold_recall: 0.9090
p: 0.1933
r: 0.6720
num_gold_spans: 1000
************
*******MD*****
MD_f1: 0.2814, (p: 0.1691, r: 0.8390)
****************

******************************
```

SN

Best = 22.5%

Measure_accuracy:

- Accuracy = 0.2250 = (22.50)
- Precision = 0.6696
- Recall = 0.2250
- F1 = 0.3368

evaluate:

- (not implemented)

```
[980|980] Predicted: 'Q137503     ' Truth: ['Q137503']     Match: True   (University of Łódź)
[981|981] Predicted: 'Q185246     ' Truth: ['Q185246']     Match: True   (Uppsala University)
[982|982] Predicted: 'None        ' Truth: ['NIL']         Match: False  (None)
[983|983] Predicted: 'None        ' Truth: ['Q393557']     Match: False  (None)
[984|984] Predicted: 'Q219        ' Truth: ['NIL']         Match: False  (Bulgaria)
[985|985] Predicted: 'None        ' Truth: ['NIL']         Match: False  (None)
[986|986] Predicted: 'None        ' Truth: ['Q2927109']    Match: False  (None)
[987|987] Predicted: 'None        ' Truth: ['Q118791749']  Match: False  (None)
[988|988] Predicted: 'None        ' Truth: ['Q12758']      Match: False  (None)
[989|989] Predicted: 'Q160556     ' Truth: ['Q933681']     Match: False  (Berlin Brandenburg Airport)
[990|990] Predicted: 'None        ' Truth: ['NIL']         Match: False  (None)
[991|991] Predicted: 'Q622532     ' Truth: ['Q7209967']    Match: False  (Policja (Poland))
[992|992] Predicted: 'None        ' Truth: ['NIL']         Match: False  (None)
[993|993] Predicted: 'Q918239     ' Truth: ['Q918239']     Match: True   (Gasunie)
[994|994] Predicted: 'None        ' Truth: ['Q16988947']   Match: False  (None)
[995|995] Predicted: 'Q13576286   ' Truth: ['Q66232114']   Match: False  (Health Education England)
[996|996] Predicted: 'None        ' Truth: ['Q38609561']   Match: False  (None)
[997|997] Predicted: 'Q24055697   ' Truth: ['Q112160997']  Match: False  (Cumberland City Council)
[998|998] Predicted: 'Q204234     ' Truth: ['Q16997843']   Match: False  (Milton Keynes)
[999|999] Predicted: 'Q84         ' Truth: ['Q5657624']    Match: False  (London)

Accuracy: 22.50% (225/1000 correct)

CUDA available? True
Running on GPU: NVIDIA GeForce RTX 4070

Results from file: SN_test.csv
Truth-values retrieved using CSV
```

Hard Table Retrieval

Benchmark Datasets for Entity Linking from Tabular Data

https://zenodo.org/records/17160156 (oppsummering fra linken ←)

Benchmark suite for evaluating entity linking algorithms on structured tabular data organised into two parts:

- Challenge dataset (HTR1, HTR2) from the SemTab Table-to-KG Challenge widely used in academic evaluations of table-to-KG alignment systems
- Each is a dataset (a collection of many tables) provided with ground truth and candidate mappings.

Contains:

- Real-world tables (Company, Movie, SN):

- **Company**
- one table constructed via SPARQL queries on wikidata, both wikidata and Crunchbase ground truths.

- **Movie**:
- One tabel constructed via SPARQL queries on wikidata

- **SN** (Spend Network)
- One procurement table from the enRichMyData (EMD) project, manually annotated and including NIL cases for mentions with no known wikidata match.

Sitering:

- This Zenodo record (Version 2):

Avogadro, R., & Rauniyar, A. (2025). Benchmark Datasets for Entity Linking from Tabular Data (Version 2). Zenodo. https://doi.org/10.5281/zenodo.15888942

- The SemTab Challenge (for HTR1/HTR2):

SemTab: Semantic Web Challenge on Tabular Data to Knowledge Graph Matching (Table-to-KG). (Cite the relevant SemTab overview paper for the year you reference.)

- Wikidata:

Data retrieved from Wikidata (public domain; CC0 1.0).

- enRichMyData (for SN / Spend Network):

Project resources from enRichMyData, licensed under CC BY 4.0.

(fra README.md)

Contents & Layout

Each challenge dataset folder typically includes:

- **tables/** - multiple input CSV tables
- **gt/** - ground-truth CSV(s) with the expected Wikidata QIDs (by cell/mention)
- **mention_to_qid/** - JSON mapping(s) from surface mentions -> candidate QIDs
- Optional JSONs in the dataset root:
    - **cell_to_qid.json**
    - **column_classifications.json**

Conventions

- String similarity statistics are computed on **lowercase** text
- Cell IDs follow the unified format: **tableName-idRow-idCol** (idRow is 0 indexed)
- Evaluators must score **only** cells present in the GT (ignore extra predictions)

Forklaring av mappe/fil-struktur

- Hver mappe (HTR1, HTR2) inneholder:
    - **tables/** - mange .CSV filer (hver = én table sample)
    - **gt/** - ground truth filer
    - **mention_to_qid/** - Kandidat entities per mention
- (valgfrie)
    - **cell_to_qid.json** og **column_classifications.json** - metadata hjelpefiler

Vårt hovedfokus:

- Mest lik vår tidligere testing er **CEA (Cell Entity Annotation)** filene.
  ("Predict entity for each mention in a cell")
- Fil: **`gt/cea_gt.csv`**
- Format: **tablename,idRow,idCol,http://www.wikidata.org/entity/Qxxxx**
- Mål: Predicte QID for hver mention cell

(resten)

- **gt/cpa_gt.csv (Column Pair Annotation)**
- mapper relasjoner mellom kolonner ("city" -> "country" = *located_in*)
  (relation linking task)

- **gt/cta_gt.csv (Column Type Annotation)**
- "assigner wikidata class/type (QID) til hver kolonne
  (Schema/type linking task)

- **mention_to_qid/*.json**
- Kandidat QID for hver "surface mention" i tablene
- For å restricte kandidat-rommet

- **cell_to_qid.json**
- Mapping fra hver "table-row-col" til QID

- **column_classifications.json**
- Høynivå metadata (typer, roller, …)

Trenger å gjøre:

- Egen funksjon for å prosesse ny data:
  - loop gjennom **tables/**
  - For hver table, les CSV
  - For hver entry i **cea_gt.csv**, finn tilsvarende celle med (idRow, idCol)
  - Sett sammen text med label QID (true entity)

- 2 lister som resultat:
  - ☐ “0FX36K78.csv”
  - ☐ texts = [“Runddeln”, “Varefjfll”, “Bukovc”, “Lftschiff”, “Lotterberh”]
  - ☐ truths = [
    [“Q7379764”], [“Q24254907”], [“Q23826100”], [“Q15118894”], [“Q23822585”]]

- Bruk av valgfrie filer:
  - ☐ **mention_to_qid**/0FX36K78.json
  - ☐ JSON med lik over ^

```
{
  "Runddeln": [
    "Q7379764"
  ],
  "Varefjfll": [
    "Q24254907"
  ],
  "Bukovc.": [
    "Q23826100"
  ],
  "Lftschiff": [
    "Q15118894"
  ],
  "Lotterberh": [
    "Q23822585"
  ]
}
```

  - **cell_to_qid**.json

```
{
  "NQK7B1JD": {
    "0-0": "Q7996268",
    "1-0": "Q7996260",
    "2-0": "Q7996231"
  },
```

  - gir GULL QID for hver celle - dvs trenger ikke iterere **table**

Results

tests done using “cell_to_qid.json”, measuring “Cell-level accuracy” using row and column in given file names in the form of:

“filename” : { row-col : truth QID, row-col : truth QID, … }

Run info:

GPU: rtx 4070
Batch: True
Batch size: 512

HTR1

↓ Model / Set →	wikidata	wikipedia
wikipedia_model_with_numbers	44.81%	31.65%
wikipedia_model	43.17%	**31.79%**
merged_full	35.85%	22.76%
merged_60k	37.06%	21.91%
merged_10k	**49.79%**	31.29%
companies_full	34.64%	21.41%
movies_full	26.03%	18.92%

HTR2

↓ Model / Set →	wikidata	wikipedia
wikipedia_model_with_numbers	31.33%	**26.79%**
wikipedia_model	28.54%	25.15%
merged_full	22.03%	18.15%
merged_60k	22.25%	16.95%
merged_10k	**32.97%**	25.15%
companies_full	20.78%	16.02%
movies_full	16.02%	12.96%

Summary

- Wikipedia-based models are the most robust, and generalize better than the domain-finetuned ones.

- Merged_10k gives the highest HTR1 accuracy of 49.8%, but does not generalize strongly to HTR2. Possibly due to mild overfitting due to finetuning.

- HTR2 is tougher overall, with lower accuracies across the board, suggesting a more ambiguous or sparse mention set.

- Larger or domain specific fine-tuned models (movies/companies) hurt the general performance due to domain shift.

(flere kommentarer?)

Semtab dataset

Datasets include

- 2T_Round4
- HardTablesR2
- HardTablesR3
- Round1_T2D
- Round3_2019
- Round4_2020

[Environment Info]

Python: 3.8.20 (default, Sep 7 2024, 18:35:07)

[**GCC** 13.2.0]

PyTorch: 2.4.1+cu121

NumPy: 1.24.4

Running on **GPU**: NVIDIA GeForce RTX 4070 Ti

System: Linux 6.6.87.2-microsoft-standard-WSL2

Machine: x86_64

Processor: x86_64

Architecture: 64bit

Cores: 12

Using ReFinED model: **'wikipedia_model_with_numbers**', entity set: '**wikidata**'

BATCH = 64

[**HTR1**]

[19:29:09] Starting processing: 'HTR1' ...
cell-level prediction mode
Prepared 1406 rows in time: 1.15s
Inference time for 1406 rows (batch 64) in time 11.32s

========-EVAL METRICS-=========
Accuracy: 0.4459   44.59   (627/1406)
Precision: 0.7037
Recall: 0.4459
F1 Score: 0.5459
==============================

[**HTR2**]

[19:29:21] Starting processing: 'HTR2' ...
cell-level prediction mode
Prepared 1983 rows in time: 2.45s
Inference time for 1983 rows (batch 64) in time 11.32s

========-EVAL METRICS-=========
Accuracy: 0.2935   29.35   (582/1983)
Precision: 0.5517
Recall: 0.2935
F1 Score: 0.3831
==============================

[**2T_Round4**]

[19:29:35] Starting processing: '2T_Round4' ...
cell-level prediction mode
Prepared 667244 rows in time: 33.82s
Inference time for 667244 rows (batch 64) in time 1142.00s

========-EVAL METRICS-=========
Accuracy: 0.4907   49.07   (327427/667244)
Precision: 0.8003
Recall: 0.4907
F1 Score: 0.6084
==============================

[**HardTablesR2**]

[19:48:38] Starting processing: 'HardTablesR2' ...
cell-level prediction mode
Prepared 47440 rows in time: 26.23s
Inference time for 47440 rows (batch 64) in time 118.64s

========-EVAL METRICS-=========
Accuracy: 0.4637   46.37   (21999/47440)
Precision: 0.7506
Recall: 0.4637
F1 Score: 0.5733
==============================

[**HardTablesR3**]

[19:50:59] Starting processing: 'HardTablesR3' ...
cell-level prediction mode
Prepared 58949 rows in time: 93.04s
Inference time for 58949 rows (batch 64) in time 142.78s

========-EVAL METRICS-=========
Accuracy: 0.2809   28.09   (16559/58949)
Precision: 0.4920
Recall: 0.2809
F1 Score: 0.3576
==============================

[**Round1_T2D**]

[19:54:47] Starting processing: 'Round1_T2D' ...
cell-level prediction mode
Prepared 37 rows in time: 0.54s
Inference time for 37 rows (batch 64) in time 0.12s

========-EVAL METRICS-=========
Accuracy: 0.5676   56.76   (21/37)
Precision: 0.8750
Recall: 0.5676++
F1 Score: 0.6885
==============================

[**Round3_2019**]

[19:54:48] Starting processing: 'Round3_2019' ...
cell-level prediction mode
Prepared 8530 rows in time: 25.02s
Inference time for 8530 rows (batch 64) in time 17.87s

========-EVAL METRICS-=========
Accuracy: 0.5613   56.13   (4788/8530)
Precision: 0.7698
Recall: 0.5613
F1 Score: 0.6492
==============================

[**Round4_2020**]

[19:55:31] Starting processing: 'Round4_2020' ...
cell-level prediction mode
Prepared 994921 rows in time: 338.98s
Inference time for 994921 rows (batch 64) in time 3032.32s

========-EVAL METRICS-=========
Accuracy: 0.5025   50.25   (499931/994921)
Precision: 0.7866
Recall: 0.5025
F1 Score: 0.6132
==============================

BATCH = 128

[**HTR1**]

[20:50:23] Starting processing: 'HTR1' ...
cell-level prediction mode
Prepared 1406 rows in time: 1.21s
Inference time for 1406 rows (batch 128) in time 8.73s

========-EVAL METRICS-=========
Accuracy: 0.4459   44.59   (627/1406)
Precision: 0.7037
Recall: 0.4459
F1 Score: 0.5459
==============================

[**HTR2**]

[20:50:32] Starting processing: 'HTR2' ...
cell-level prediction mode
Prepared 1983 rows in time: 2.32s
Inference time for 1983 rows (batch 128) in time 8.70s

========-EVAL METRICS-=========
Accuracy: 0.2940   29.40   (583/1983)
Precision: 0.5526
Recall: 0.2940
F1 Score: 0.3838
==============================

[**2T_Round4**]

[20:50:43] Starting processing: '2T_Round4' ...
cell-level prediction mode
Prepared 667244 rows in time: 30.76s
Inference time for 667244 rows (batch 128) in time 1103.25s

========-EVAL METRICS-=========
Accuracy: 0.4907   49.07   (327413/667244)
Precision: 0.8004
Recall: 0.4907
F1 Score: 0.6084
==============================

[**HardTablesR2**]

[21:09:14] Starting processing: 'HardTablesR2' ...
cell-level prediction mode
Prepared 47440 rows in time: 26.96s
Inference time for 47440 rows (batch 128) in time 122.71s

========-EVAL METRICS-=========
Accuracy: 0.4637   46.37   (21999/47440)
Precision: 0.7507
Recall: 0.4637
F1 Score: 0.5733
==============================

[**HardTablesR3**]

[21:11:39] Starting processing: 'HardTablesR3' ...
cell-level prediction mode
Prepared 58949 rows in time: 95.48s
Inference time for 58949 rows (batch 128) in time 169.23s

========-EVAL METRICS-=========
Accuracy: 0.2809   28.09   (16561/58949)
Precision: 0.4921
Recall: 0.2809
F1 Score: 0.3577
==============================

[**Round1_T2D**]

[21:15:57] Starting processing: 'Round1_T2D' ...
cell-level prediction mode
Prepared 37 rows in time: 0.55s
Inference time for 37 rows (batch 128) in time 0.18s

========-EVAL METRICS-=========
Accuracy: 0.5676   56.76   (21/37)
Precision: 0.8750
Recall: 0.5676
F1 Score: 0.6885
==============================

[**Round3_2019**]

[21:15:58] Starting processing: 'Round3_2019' ...
cell-level prediction mode
Prepared 8530 rows in time: 26.81s
Inference time for 8530 rows (batch 128) in time 25.34s

========-EVAL METRICS-=========
Accuracy: 0.5624   56.24   (4797/8530)
Precision: 0.7712
Recall: 0.5624
F1 Score: 0.6504
==============================

[**Round4_2020**]

[21:16:49] Starting processing: 'Round4_2020' ...
cell-level prediction mode
Prepared 994921 rows in time: 176.34s
Inference time for 994921 rows (batch 128) in time 2732.95s

========-EVAL METRICS-=========
Accuracy: 0.5025   50.25   (499938/994921)
Precision: 0.7866
Recall: 0.5025
F1 Score: 0.6132
==============================

BATCH = 256

[HTR1]

[22:04:23] Starting processing: 'HTR1' ...
cell-level prediction mode
Prepared 1406 rows in time: 1.30s
Inference time for 1406 rows (batch 256) in time 8.54s

========-EVAL METRICS-=========
Accuracy: 0.4452   44.52   (626/1406)
Precision: 0.7026
Recall: 0.4452
F1 Score: 0.5451
==============================

[HTR2]

[22:04:33] Starting processing: 'HTR2' ...
cell-level prediction mode
Prepared 1983 rows in time: 2.55s
Inference time for 1983 rows (batch 256) in time 8.48s

========-EVAL METRICS-=========
Accuracy: 0.2940   29.40   (583/1983)
Precision: 0.5526
Recall: 0.2940
F1 Score: 0.3838
==============================

[2T_Round4]

[22:04:44] Starting processing: '2T_Round4' ...
cell-level prediction mode
Prepared 667244 rows in time: 34.38s
Inference time for 667244 rows (batch 256) in time 2846.59s

========-EVAL METRICS-=========
Accuracy: 0.4907   49.07   (327411/667244)
Precision: 0.8003
Recall: 0.4907
F1 Score: 0.6084
==============================

[HardTablesR2]

[22:51:43] Starting processing: 'HardTablesR2' ...
cell-level prediction mode
Prepared 47440 rows in time: 27.23s
Inference time for 47440 rows (batch 256) in time 242.64s

========-EVAL METRICS-=========
Accuracy: 0.4637   46.37   (22000/47440)
Precision: 0.7507
Recall: 0.4637
F1 Score: 0.5733
==============================

[HardTablesR3]

[22:56:08] Starting processing: 'HardTablesR3' ...
cell-level prediction mode
Prepared 58949 rows in time: 92.68s
Inference time for 58949 rows (batch 256) in time 309.76s

========-EVAL METRICS-=========
Accuracy: 0.2811   28.11   (16569/58949)
Precision: 0.4922
Recall: 0.2811
F1 Score: 0.3578
==============================

[Round1_T2D]

[23:02:41] Starting processing: 'Round1_T2D' ...
cell-level prediction mode
Prepared 37 rows in time: 0.55s
Inference time for 37 rows (batch 256) in time 0.20s

========-EVAL METRICS-=========
Accuracy: 0.5676   56.76   (21/37)
Precision: 0.8750
Recall: 0.5676
F1 Score: 0.6885
==============================

[Round3_2019]

[23:02:41] Starting processing: 'Round3_2019' ...
cell-level prediction mode
Prepared 8530 rows in time: 25.18s
Inference time for 8530 rows (batch 256) in time 57.17s

========-EVAL METRICS-=========
Accuracy: 0.5623   56.23   (4796/8530)
Precision: 0.7711
Recall: 0.5623
F1 Score: 0.6503
==============================

[Round4_2020]

[23:04:03] Starting processing: 'Round4_2020' ...
cell-level prediction mode
Prepared 994921 rows in time: 204.23s
Inference time for 994921 rows (batch 256) in time 5229.76s

========-EVAL METRICS-=========
Accuracy: 0.5025   50.25   (499964/994921)
Precision: 0.7866
Recall: 0.5025
F1 Score: 0.6133
==============================

BATCH = 512

[HTR1]

[00:32:59] Starting processing: 'HTR1' ...
cell-level prediction mode
Prepared 1406 rows in time: 2.72s
Inference time for 1406 rows (batch 512) in time 13.04s

========-EVAL METRICS-=========
Accuracy: 0.4474   44.74   (629/1406)
Precision: 0.7059
Recall: 0.4474
F1 Score: 0.5477
==============================

[HTR2]

[00:33:15] Starting processing: 'HTR2' ...
cell-level prediction mode
Prepared 1983 rows in time: 5.47s
Inference time for 1983 rows (batch 512) in time 12.89s

========-EVAL METRICS-=========
Accuracy: 0.2935   29.35   (582/1983)
Precision: 0.5522
Recall: 0.2935
F1 Score: 0.3833
==============================

[2T_Round4]

[00:33:33] Starting processing: '2T_Round4' ...
cell-level prediction mode
Prepared 667244 rows in time: 31.36s
Inference time for 667244 rows (batch 512) in time 4915.48s

========-EVAL METRICS-=========
Accuracy: 0.4907   49.07   (327411/667244)
Precision: 0.8004
Recall: 0.4907
F1 Score: 0.6084
==============================

[HardTablesR2]

[01:55:05] Starting processing: 'HardTablesR2' ...
cell-level prediction mode
Prepared 47440 rows in time: 25.78s
Inference time for 47440 rows (batch 512) in time 284.56s

========-EVAL METRICS-=========
Accuracy: 0.4637   46.37   (21999/47440)
Precision: 0.7507
Recall: 0.4637
F1 Score: 0.5733
==============================

[HardTablesR3]

[02:00:16] Starting processing: 'HardTablesR3' ...
cell-level prediction mode
Prepared 58949 rows in time: 92.66s
Inference time for 58949 rows (batch 512) in time 346.05s

========-EVAL METRICS-=========
Accuracy: 0.2809   28.09   (16559/58949)
Precision: 0.4919
Recall: 0.2809
F1 Score: 0.3576
==============================

[Round1_T2D]

[02:07:36] Starting processing: 'Round1_T2D' ...
cell-level prediction mode
Prepared 37 rows in time: 0.57s
Inference time for 37 rows (batch 512) in time 0.31s

========-EVAL METRICS-=========
Accuracy: 0.5676   56.76   (21/37)
Precision: 0.8750
Recall: 0.5676
F1 Score: 0.6885
==============================

[Round3_2019]

[02:07:37] Starting processing: 'Round3_2019' ...
cell-level prediction mode
Prepared 8530 rows in time: 25.06s
Inference time for 8530 rows (batch 512) in time 57.01s

========-EVAL METRICS-=========
Accuracy: 0.5623   56.23   (4796/8530)
Precision: 0.7711
Recall: 0.5623
F1 Score: 0.6503
==============================

[Round4_2020]

[02:09:01] Starting processing: 'Round4_2020' ...
cell-level prediction mode
Prepared 994921 rows in time: 342.26s
Inference time for 994921 rows (batch 512) in time 6722.05s

========-EVAL METRICS-=========
Accuracy: 0.5025   50.25   (499925/994921)
Precision: 0.7866
Recall: 0.5025
F1 Score: 0.6132
==============================

Report

Experimental Setup

The ReFinED entity linking tool, configured with the ‘wikipedia_model_with_numbers’ model using the ‘wikidata’ entity set, was evaluated across multiple datasets from the SemTab challenge from various different years. These include: 2T_Round4, HardTablesR2, HardTablesR3, Round3_2019, and Round4_2020. The Round2_T2D dataset was also included, but only a small subset of rows could be processed successfully due to discrepancies between table files and ground truth files. Additional datasets ‘HTR1’ and HTR2 were also included, although analyzed separately elsewhere due to previous testing of the model and due to more precise analysis with different models. The experiments were conducted on a system equipped with an NVIDIA GeForce RTX 4070 Ti GPU, using Python 3.8.20 and PyTorch 2.4.1, with batch sizes varying from 64 to 512.

For all experiments, ReFinED operated in cell-level prediction mode. Metrics used for evaluation included accuracy, precision, recall, and F1 score. Inference times were recorded to assess computational performance under varying batch sizes.

Results

Across all the datasets and different batch sizes, the accuracy, precision, and F1 scores remained mostly consistent, with only small differences between runs. For instance, the large dataset 2T_Round4 achieved a consistent accuracy of approximately 49%, precision of around 0.80, and an F1 score of roughly 0.61, regardless of big or small batch size used. This stability arises due to the use of cell-level entity linking, where each row is processed independently, meaning that increasing batch size primarily affects processing efficiency rather than the outcome of predictions. Minor differences in metrics (as small as 0.001 in F1) are likely attributable to floating point rounding or slight variations in batch handling.

Performance varied primarily between the different datasets. Tests on datasets such as ‘Round3_2019’ resulted in higher accuracies (around 56-57%) and F1 scores (0.65-0.69), whereas more challenging datasets such as HTR1 and HardTablesR2 produced lower

accuracies (28-30%) and F1 scores (0.35-0.38). These differences reflect varying levels of ambiguity, naming inconsistencies, and novel entity occurrences across datasets.

The Round1_T2D dataset represents a special case in this evaluation. While the ground truth contains 8418 entities across 64 tables, only 64 rows were successfully processed in our tests. This limitation is due to inconsistencies between the table files and the CEA ground truth files. As a result, the metrics obtained for this subset - Accuracy of 56.76%, Precision of 87.5%, Recall of 56.76%, and F1 of 0.6885 - reflects only a small fraction of the dataset and should be interpreted with caution. These results are presented for completeness but are not included in aggregated performance comparisons.

Across all datasets, precision has consistently been higher than by recall and accuracy. This indicates that ReFinED operates conservatively, favoring avoiding false positives over capturing every correct entity. While this approach minimizes incorrect links, it also limits coverage, particularly in larger datasets where recall lags behind precision. This observation highlights a potential area of improvement, such as strategies that incorporate additional context or row-level predictions.

Runtime and Scalability

Inference time naturally increased with dataset size, but the impact of batch size varied depending on the dataset. For smaller datasets, increasing batch size reduced runtime due to more efficient GPU utilization. For very large datasets, such as Round4_2020 and 2T_Round4, runtime increased most notably at the largest batch sizes. These results indicate that while ReFinED’s predictive performance is largely stable across batch sizes, selecting an appropriate batch size is critical to optimize throughput for large-scale data processing.

The Round1_T2D dataset is an exception in this analysis. Despite containing 8418 entities across 64 tables, only 37 rows were successfully processed, resulting in an unusually short runtime. This highlights the importance of validating dataset structure and completeness before running large-scale evaluations.

Discussion

Overall, ReFinED exhibits consistent predictive performance across batch sizes in cell-level mode. Dataset characteristics, particularly ambiguity, naming variations, and the presence of novel entities, are the primary determinants of variation in accuracy and F1 scores. Conservative prediction behavior is evident across datasets, with precision consistently exceeding recall and accuracy. This suggests that ReFinED favors avoiding false positives over maximizing coverage, which can limit recall in complex or highly ambiguous datasets.

The observed higher precision over recall on most datasets can be interpreted as ReFinED's reliance on confidence-based filtering during entity assignment. This means candidate entities are only linked when sufficient evidence is available, which reduces the likelihood of incorrect links but also causes many ambiguous or weakly supported mentions to remain unlinked. In datasets with high name ambiguity or limited context, this behavior leads to a high amount of missed entity links, lowering recall. This also means that datasets with clearer mentions and strong context benefit from this strategy, achieving high accuracy and F1 scores.

The increase in runtime observed for very large datasets at higher batch sizes can be due to reasons such as hardware and memory constraints, rather than model behavior. Large batches increase workload on the GPU memory and can cause internal buffering overheads, which then can reduce the efficiency of parallel processing and introduce delays. Because of this, beyond a certain batch size, the benefits of increased parallelism are outweighed by hardware costs, leading to long runtimes.

The partial results from Round1_T2D illustrate the potential impact of dataset anomalies. Although the metrics for the processed subset appear reasonable (Accuracy = 56.76%, Precision = 87.5%, F1 = 0.6885), these results cannot be interpreted as representative of the full dataset. This puts weight on the importance of dataset verification and careful preprocessing in entity linking evaluations.

Future work could explore row-level prediction, which incorporates surrounding cell context and may increase recall. Error analysis could further categorize failures according to entity ambiguity, missing context, or novel entity occurrence, providing insights for targeted model improvements. Additionally, visual summaries of performance across datasets, such as bar charts of accuracy/F1 or scatterplots of precision vs recall, could effectively communicate patterns and outliers in the results. Finally, examining batch size efficiency in combination

with dataset size can inform best practices for high-throughput deployment of ReFinED in practical applications.

Data tables and visualization

WIP

https://docs.google.com/spreadsheets/d/1Iemt3-J4HgdoZn-eT7IM9tCRTax501-XGEbfJjGHmVg/edit?usp=sharing

	A	B	C	D	E	F	G	H	I
1	Dataset	Batch Size	Rows Processed	Accuracy	Precision	Recall	F1 Score	Inference Time (s	Note
2	HTR1	64	1406	0.4459	0.7037	0.4459	0.5459	11.32	-
3	HTR2	64	1983	0.2935	0.5517	0.2935	0.3831	11.32	-
4	2T_Round4	64	667244	0.4907	0.8003	0.4907	0.6084	11.42	-
5	HardTablesR2	64	47440	0.4637	0.7506	0.4637	0.5733	118.64	-
6	HardTablesR3	64	58949	0.2809	0.4920	0.2809	0.3576	142.78	-
8	Round3_2019	64	8530	0.5613	0.7698	0.5613	0.6492	17.87	-
9	Round4_2020	64	994921	0.5025	0.7866	0.5025	0.6132	3032.32	-
10									
11	HTR1	128	1406	0.4459	0.7037	0.4459	0.5459	8.73	-
12	HTR2	128	1983	0.2940	0.5526	0.2940	0.3838	8.70	-
13	2T_Round4	128	667244	0.4907	0.8004	0.4907	0.6084	1103.25	-
14	HardTablesR2	128	47440	0.4637	0.7507	0.4637	0.5733	122.71	-
15	HardTablesR3	128	58949	0.2809	0.4921	0.2809	0.3577	169.23	-
17	Round3_2019	128	8530	0.5624	0.7712	0.5624	0.6504	25.34	-
18	Round4_2020	128	994921	0.5025	0.7866	0.5025	0.6132	2732.95	-
19									
20	HTR1	256	1406	0.4452	0.7026	0.4452	0.5451	8.54	-
21	HTR2	256	1983	0.2940	0.5526	0.2940	0.3838	8.48	-
22	2T_Round4	256	667244	0.4907	0.8003	0.4907	0.6084	2846.59	-
23	HardTablesR2	256	47440	0.4637	0.7507	0.4637	0.5733	242.64	-
24	HardTablesR3	256	58949	0.2811	0.4922	0.2811	0.3578	309.76	-
26	Round3_2019	256	8530	0.5623	0.7711	0.5623	0.6503	57.17	-
27	Round4_2020	256	994921	0.5025	0.7866	0.5025	0.6133	5229.76	-
28									
29	HTR1	512	1406	0.4474	0.7059	0.4474	0.5477	13.04	-
30	HTR2	512	1983	0.2935	0.5522	0.2935	0.3833	12.89	-
31	2T_Round4	512	667244	0.4907	0.8004	0.4907	0.6084	4915.48	-
32	HardTablesR2	512	47440	0.4637	0.7507	0.4637	0.5733	284.56	-
33	HardTablesR3	512	58949	0.2809	0.4919	0.2809	0.3576	346.05	-
35	Round3_2019	512	8530	0.5623	0.7711	0.5623	0.6503	57.01	-
36	Round4_2020	512	994921	0.5025	0.7866	0.5025	0.6132	6722.05	-

Green shows a threshold over 0.7
Red shows a threshold below 0.4

Round1_T2D is excluded due to inconsistencies.

Table 1: Inference time vs. Batch size

Table 1: Inference time vs. batch size

Dataset	Batch 64	Batch 128	Batch 256	Batch 512
2T_Round4	11.42	1103.25	2846.59	4915.48
Round4_2020	3032.32	2732.95	5229.76	6722.05
HardTablesR3	142.78	169.23	309.76	346.05

notes:
- few tables with most notable differences chosen

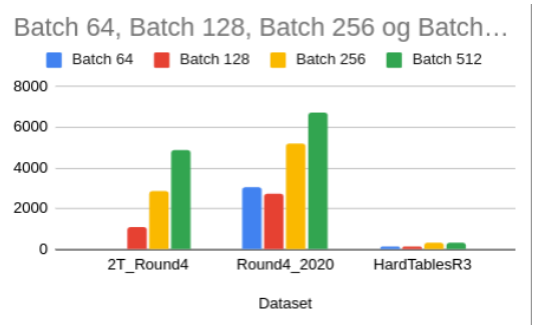
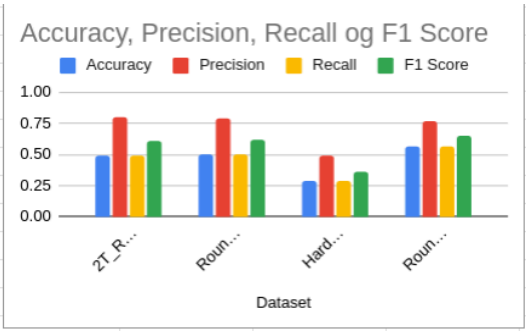


Table 2: Accuracy / F1 comparison on batch size 128

Table 2: Accuracy / F1 comparison on batch size 128

Dataset	Accuracy	Precision	Recall	F1 Score
2T_Round4	0.4907	0.8004	0.4907	0.6084
Round4_2020	0.5025	0.7866	0.5025	0.6132
HardTablesR3	0.2809	0.4921	0.2809	0.3577
Round3_2019	0.5624	0.7712	0.5624	0.6504



Data verification

Telling av antall rader i de ulike datasettene, brukt for tabell i kapittel 3.3

Dataset	Files	Avg Rows	Max Rows
2T_Round4	180	1080.21	15477
HardTablesR2	1750	16.73	57
HardTablesR3	7206	8.18	20
Round1_T2D	64	142.02	585
Round3_2019	2161	70.69	206
Round4_2020	22207	21.43	59

Ollama

Approach 1

zero-shotte den med prompts linje for linje, be om QID only som svar

resultat
den er ganske ass, nesten uansett modell, den kan finne super basic ting som hovedstaden av tyskland osv, men den kan ikke faktisk gjøre wikidata queries så den lowkey bare gjetter..

Approach 2

Lage et system som først gir den en linje, så lar den lage en sparql query som vi kjører, så velge QID utifra sparql svar

resultat 1
eeeh den funka litt rart, ga ikke noe særlig bedre svar

prøvde igjen med sparql likte ikke så force avslutta?

etter litt rework så funker det men må fikse litt på resultatene jeg får ..:

--- SPARQL RESULTS ---
[1] {'film': 'http://www.wikidata.org/entity/Q830191'}

FINAL RESULT → QID: None

riktig QID men LLM sier den ikke finner noe

kan hende approachen ikke 100% følger https://wikitech.wikimedia.org/wiki/Robot_policy så må eventuelt tweake den litt spesielt denne delen https://foundation.wikimedia.org/wiki/Policy:Wikimedia_Foundation_User-Agent_Policy

men usikker om det er riktig type approach uansett…

API

Side for informasjon om backend-service / API som skal integrere ReFinED med Koala-UI

https://github.com/enRichMyData/koala_ui

Ide:

- Koala-UI og Backend-service (API) er to programmer som kjøres separat
- De kommuniserer med hverandre via http requests (?) vha FastAPI

Basic pipeline:

==========

1. **Data input**
    a. bruker begynner med CSV fil med data
    b. laster opp via Koala (?) (eller direkte via POST request til API)

2. **API backend**
    a. POST /datasets/{dataset_name}/tables –> laste opp CSV
    b. GET /datasets/{dataset_name}/tables/{table_name} –> henting av table

    ansvar:
    - leser CSV
    - bruker ReFinED for linking
    - lager Koala JSON
    - returnerer JSON

3. **Koala frontend**
    a. henter JSON via GET request til backend
    b. Viser resultater

==========

Kode

- Branch --> 'API'
- folder --> 'API'
- fil --> 'API_test.py'

- for øyeblikket en "proof-of-concept" testkode som verifiserer funksjonaliteten og viser riktig output

Må gjøres

1. **Sette opp Koala lokalt**
    - krever LamAPI
    - krever Crocodile

- Instruksjon står skrevet på Koala Readme

2. **Integrere egen modell (ReFinED) med Koala**
- Instruksjoner står på koala:
3. **Lage modell for å vise pipeline**
- En modell / tegning / diagram som kan settes inn i masteren for å vise hvordan hele prosessen fungerer

(fra koala README)

Example Integration

To integrate your entity linking system with Koala UI:

1. Implement the required API endpoints
2. Format your data according to this specification
3. Configure Koala UI to point to your backend URL
4. Optionally integrate with external knowledge bases (Wikidata, etc.)

For reference implementation, see the Crocodile entity linking system.

(FERDIG)

Arbeidsoversikt

Børge:

(Tirsdag) 22. April 2025
- Begynte på essay / thesis
- Skrev 3-4 sider om introduksjon til semantiske teknologier

(Tirsdag) 29. April 2025
- Lagt inn kilder
- Fikset småfeil på essay

(Tirsdag) 6. Mai 2025
- Begynte lesing av ReFinED artikkel. Skrev ned nyttige punkt på side: ReFinED (Mention detection + Entity disambiguation)
- Ryddet opp i kilder
- Lagt til flere delkapitler med forslag

(Tirsdag) 13. Mai 2025
- Fortsatt notering av ReFinEd Artikkel ReFinED
- Integrerte thesis-description/statement i introduksjonsteksten.
- Skriver videre og utforsker problemer med entity linking som essay-tema
    - entity linking forklaring
    - ambiguity
        - homonymy + polysemy
    - Nevnte og skrev om noen par utfordringer
        - naming, context, novel entities

(august en gang)
- Laget test-repo for ReFinED. (https://github.com/borgebj/ReFinED_Master)
- Fått den til å kjøre, funnet ut problemet osv (python versjon, transformers)
- Laget testfiler, både for prosessering av CSV vi fikk og for å kjøre faktisk entity linking fra ReFinED.
-

(Fredag) 05. September 2025
- Byttet over endringer fra test-repo til main-repo (https://github.com/enRichMyData/ReFined)

(Tirsdag) 9. September 2025
- Opprettet fil for benchmarking i ReFinED/my_tests/benchmark.py
    - inkluderer kjøretidsresultater gjennom manuell timing av kjøring, og detaljert profilering gjennom cProfile modulen.
    - Lagt til lang kommentar som forklarer hele filen samtidig som hva som er viktig, hva som tar tid, bottlenecks etc
    - + timing for Ifi-pc på bunnen
    - https://github.com/enRichMyData/ReFinED/blob/main/my_tests/benchmark.py

(Mandag) 15. September 2025

- Begynt med benchmarking og måling av resultater. Skrevet ned info og resultat av to kjøringer (kort og lang) og vurdert resultatene i en mini-rapport.
- https://docs.google.com/document/d/1Nm_H4IEWSPvTUbH5uymfzyQEMES3cAboq3LqJaFXwOE/edit?tab=t.p0cft2sh2j72
-

(Onsdag-lørdag + Mandag) 17-20. + 22. September 2025

- Stor refaktorering av testkode
- nye filer (accuracy.py, refined_utils.py, data-filer)
- Måling av nøyaktighet med ground truth values fra både CSV og JSON fil
- Trening av nye modeller
- Lagt til batching-option (i refined_utils)
- Tillatt gpu/cpu switching og sammenligning (i refined_utils, valg i hver fil)

(Tirsdag-Torsdag) 23.25 September 2025

- Fine tuning av modell m/ data for 'movies'' og 'companies'
- test-fil : finetune.py
    - kode skal til fil: src/dataset_reading/entity_linking/ dataset_factory.py

(Mandag) 29. September 2025

- Kjøretids-testing av alle 3 filer
- m/ GPU ulike batch size: 512, 256, 128, 64, 32
- data + diagrammer på nye sider: benchmark/companies, movies, SN, summary

(Torsdag- Lørdag 02-04 oktober 2025

- Testing og kjøring av fine tuning for store datafiler
- Test-modeller spesifisert på Fine tuning siden (link)
- Inkluderer testing for 5k linjer hver fil, 30k linjer hver fil, full companies, full movies, full begge (manger timer hver)
- Rapportert om resultater og skrevet om det: FINE TUNING
  (summary delen nederst er viktigst, og kan brukes i tesen)

(Onsdag) 22. Oktober 2025

- Startet dokumentering av HTR

(Søndag-Mandag) 26-27 Oktober 2025

- Startet testing av HTR (Hard Table Retrieval)
- oppdatert HTR side
- Laget enkel fil for testing (trenger mer) (my_tests/htr_test.py)
- github branch: "**evaluation**"

(Mandag-Onsdag) 3-5. November 2025

- Måling og kort kommentar av htr1 + htr2, ligger under “Hard Table Retrieval”
- Utvidet accuarcy måleren i “accuracy.py” - måler F1, recall, precision
- kjørt tester på nytt og oppdatert “Accuarcy (%)” siden

(idk) Sent november / tidlig desember 2025

- Laget kode for kjøring og testing av nye dataset
- Kjørt og testet at fungerer

(Tirsdag) 6. Januar 2026

- Fullført kjøring av testing av dataset
- Dokumentert og skrevet om resultat
- Startet mal og lagt inn småfillers av selve masteroppgaven på www.overleaf.com
- https://www.overleaf.com/project/695cdb068df15c6d11197121

(Tirsdag) 13. Januar 2026

- Bearbeidet og videre analysert data fra semtab kjøring.
- Laget diagrammer og tabeller inne på google sheets

(Tirsdag) 20. Januar 2026

- Fylt opp Overleaf masteroppgaven med mye tekst fra essayet
- Også satt inn generell “introduksjoner / motivasjoner” rundt omkring i forskjellige kapitler
- Omskrevet noen kapitler / delkapitler
- Fått opp til 40 sider !

(Torsdag) 29. januar 2026

- Startet arbeid på API branch
    - laget “proof-of-concept” kode som test av JSON - mangler FastAPI
- Lagt inn mer og ryddet tekst på overleaf masteroppgaven
    - bl.a. introduksjon og background (chapter 1 og 2)

Vegard:

(Lørdag-Mandag)27-29 September 2025

- Fine tune funksjon for ‘companies’ laget
- lagt til get_companies_docs og get_movies_docs funksjonene i dataset_factory.py
- tester finetune.py, prosess kjører…: 900/1049(måtte starte på nytt…)
- den “evaluatet”så startet på en ny runde elns? 61/1049